인공지능

Artificial Intelligence

컨볼루션 신경망과 컴퓨터 비전

본 강의자료는 한빛아카데미 〈파이썬으로 만드는 인공지능〉을 내용을 기반으로 제작 및 보완되었음을 알립니다.

지금까지

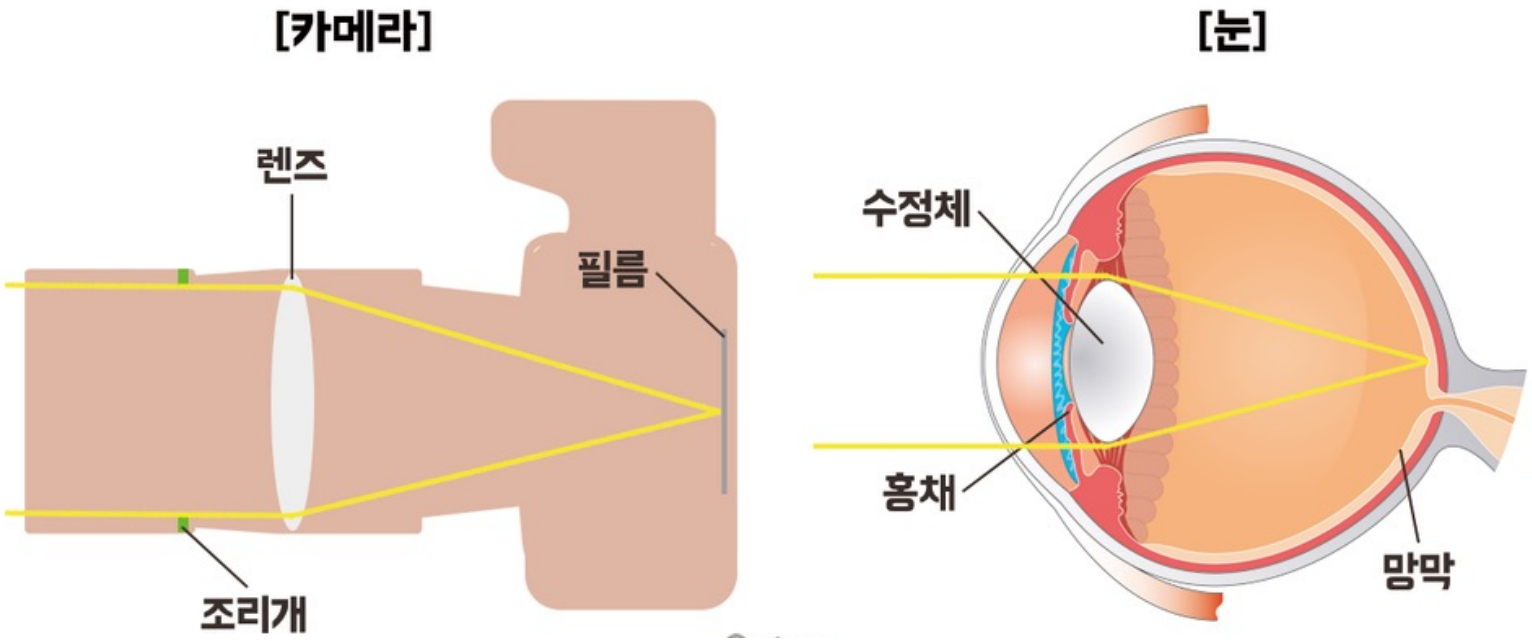
- **신경망 기초**
  - Perception with scikit-learn
  - Multi-Layer Perception (MLP) with scikit-learn

- **딥러닝과 파이토치**
  - Multi-Layer Perception (MLP) with Pytorch

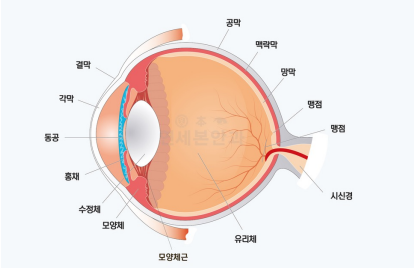
- **컨볼루션 신경망과 컴퓨터 비전**
  - Convolution Neural Network (CNN) with Pytorch
    - 인간 뇌의 시각 시스템을 모사한 신경망

6.1 컨볼루션 신경망의 동기와 전개

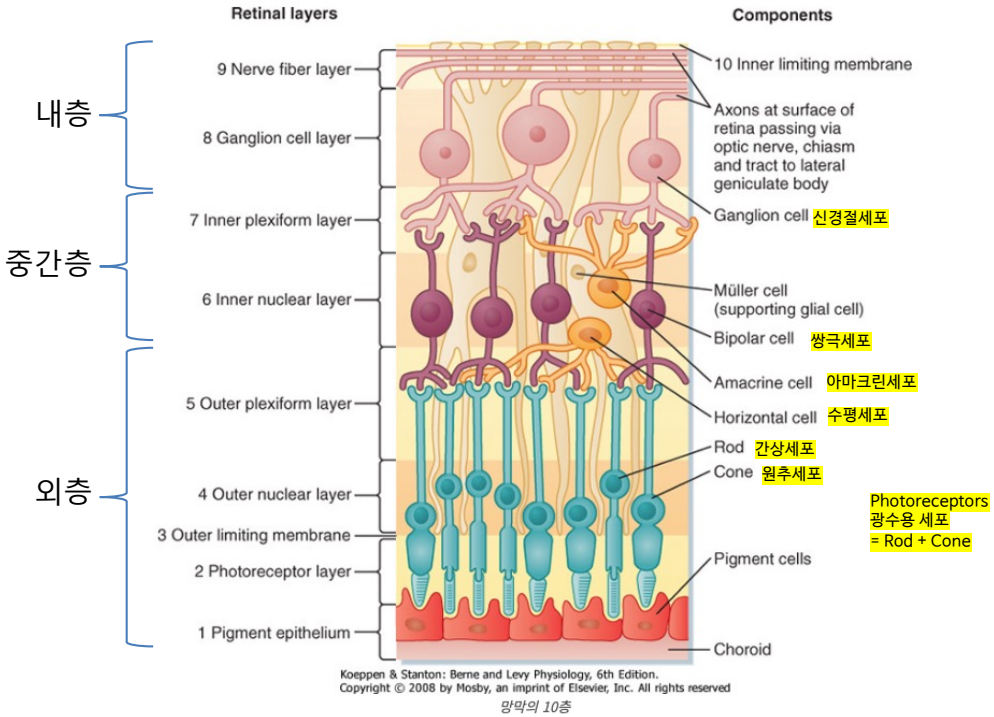
- **망막이란?**
  - 안구의 가장 안쪽에 있으면서 시신경이 분포되어 있는 투명하고 얇은 막
  - 카메라와 비교했을 때 카메라 필름에 해당함
  - 망막은 우리 눈에 들어온 빛에 대한 정보를 전기적 정보로 전환하여 신경을 통해 뇌로 전달해주는 역할을 담당
  - 우리가 사물을 볼 때 그 상이 망막에 맺혀서 뇌에 전달되어 사물을 인식하기 때문에 눈에서 가장 중요한 부분임



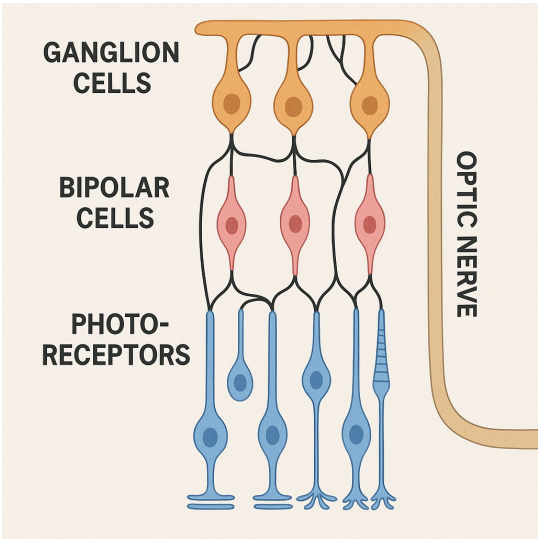
6.1 컨볼루션 신경망의 동기와 전개



- **망막이란?**
  - 망막은 광수용세포 + 중간 연결세포 + 신경절세포로 이루어진 다층 신경망(Neural Network) 구조 임
    - 망막의 층별 신경회로 (예) photoreceptor → bipolar cell → ganglion cell → optic nerve
    - 신경절 세포는 망막의 여러 광수용세포로부터 입력을 받는데, 이들이 형성하는 영역이 세포의 수용장임

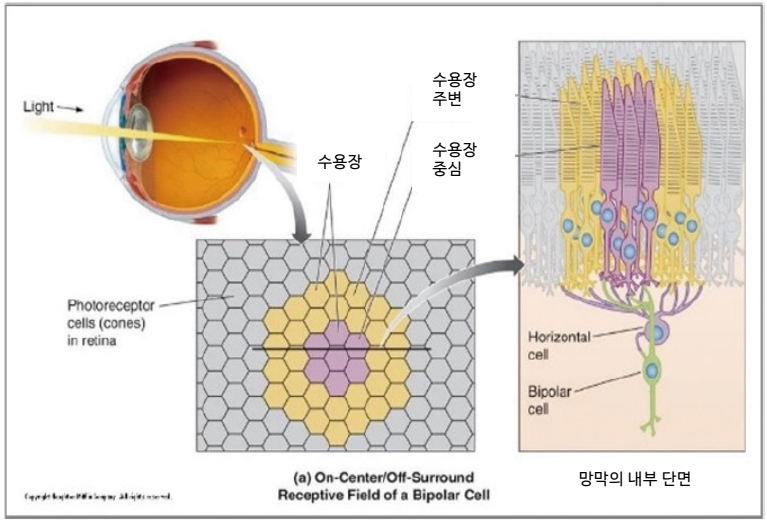


망막의 10층



6.1 컨볼루션 신경망의 동기와 전개

- **동물의 감각 기관은 수용장receptive field으로 구성(1900년대 초에 발견)**
  - 광수용 세포photoreceptor cell의 뭉치가 수용장receptive field 구성
  - 광수용 세포는 센싱한 빛의 강도를 화학 신호로 변환하여 수평 세포horizontal cell 와 양극 세포Bipolar Cell에 전달
    - 수평 세포 : 주변 신호를 억제,
    - 양극 세포 : 수용 영역 중심부의 흥분 신호를 전달
  - 신경절세포ganglion cell 는 수평 세포와 양극 세포의 신호를 모아 발화(세포막의 전위가 일정 임계값을 넘어서면서 전기적 신호가 축삭을 따라 전달되는 현상) 여부 결정



(a) 인간 시각의 수용장

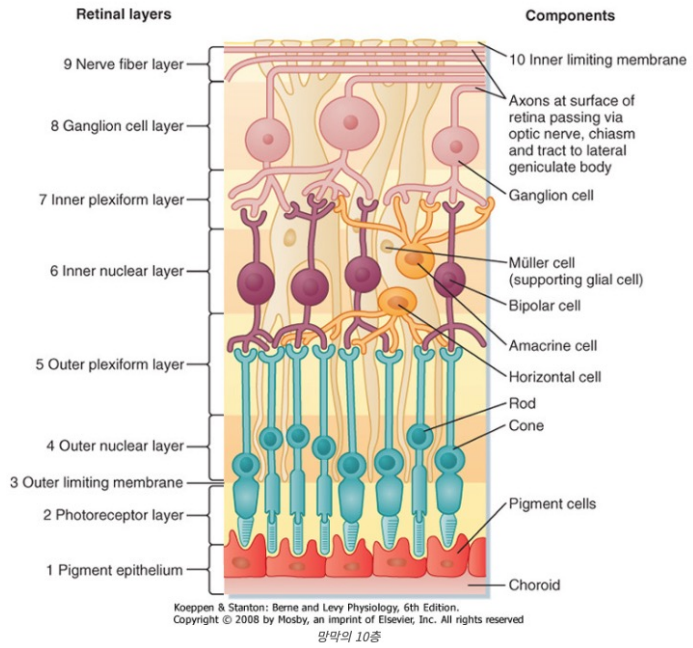
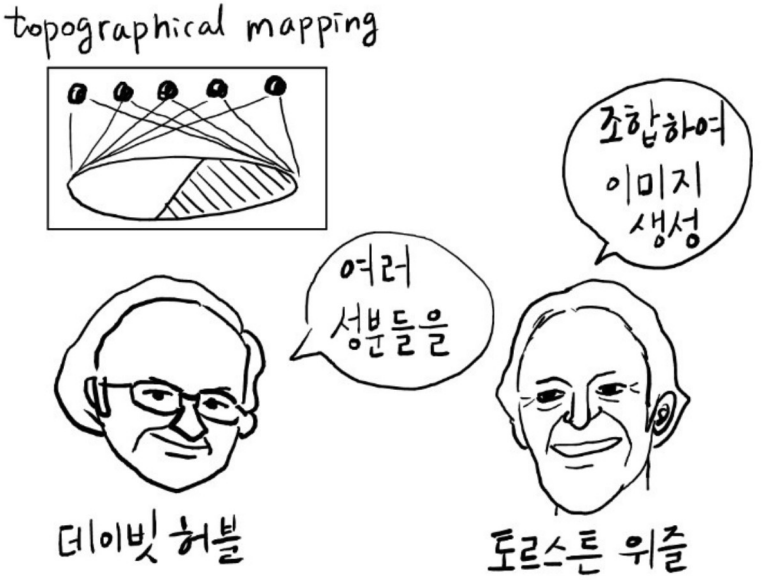


그림 6-2 컨볼루션 신경망(CNN)의 동기와 발전 과정

6.1 컨볼루션 신경망의 동기와 전개

- **허블Hubel과 비셀Wiesel의 고양이 실험**
  - 1차 시각 피질에 마이크로 전극을 삽입하고 반응을 관찰 (1958-1959년)
  - 에지 방향에 따라 반응하는 뉴런이 정해져 있고 같은 방향에 반응하는 뉴런은 같은 열에 배치되어 있다는 사실 발견 (2005년)
  - 생물 신경망의 과학적 발견은 인공 신경망 발전에 영향을 미침

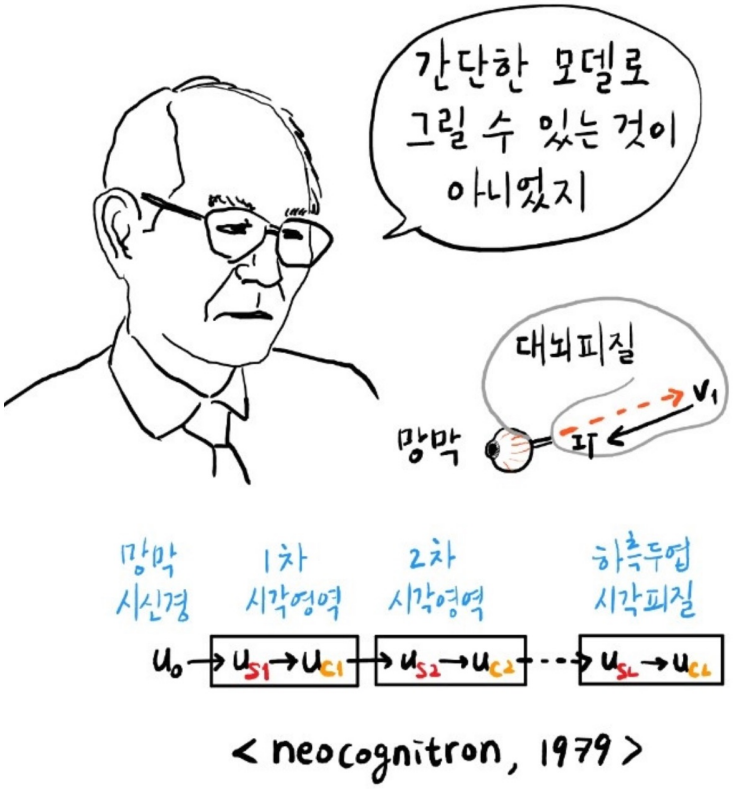


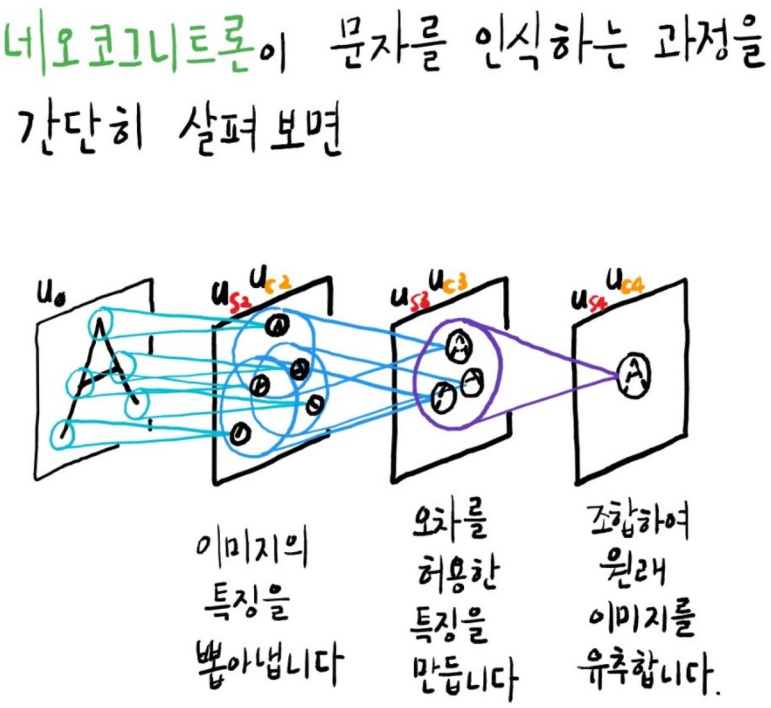


야사와 만화로 배우는 인공지능 https://brunch.co.kr/@hvnpoet/52

6.1 컨볼루션 신경망의 동기와 전개

- **후쿠시마의 네오코그니트론Neocognitron (1980년)**
  - 고양이 시각연구에 영향을 받아 “시각시스템” 연구에 몰입함
  - S-CellSimple 과 C-CellComplex의 여러 층을 통해 네오코그니트론을 만듬

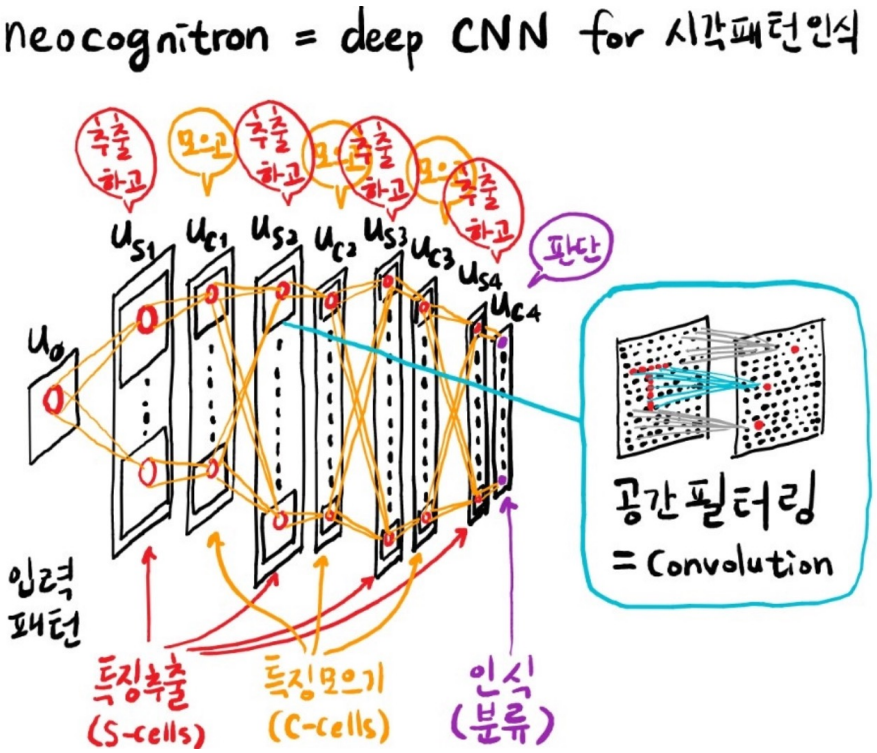




야사와 만화로 배우는 인공지능 https://brunch.co.kr/@hvnpoet/52

6.1 컨볼루션 신경망의 동기와 전개

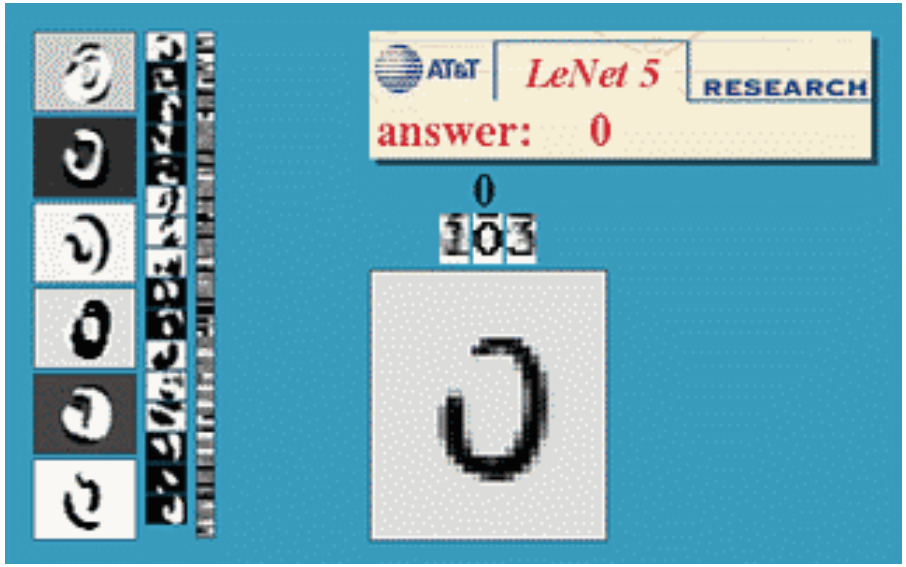
- **후쿠시마의 네오코그니트론Neocognitron (1980년)**
  - 고양이 시각연구에 영향을 받아 “시각시스템” 연구에 몰입함
  - S-CellSimple 과 C-CellComplex의 여러 층을 통해 네오코그니트론을 만듦



야사와 만화로 배우는 인공지능 https://brunch.co.kr/@hvnpoet/52

6.1 컨볼루션 신경망의 동기와 전개

- **르쿤의 1998년 〈Proceedings of the IEEE〉 논문**
  - 네오코그니트론 과 LeCun 교수님의 Back-propagation 이 더해지면서 CNN이 고안됨
  - LeNet-5는 필기 숫자에 대해 혁신적인 성능 얻음 (미국에서 쓰이는 수표 인식기에 활용)
  - 실용적인 성능을 입증한 최초의 컨볼루션 신경망 모델



http://yann.lecun.com/exdb/lenet/index.html

6.1 컨볼루션 신경망의 동기와 전개

- **ILSVRC2012 우승한 AlexNet**
  - 자연 영상을 1000개 부류로 분류하는 문제
  - AlexNet은 5순위 오류율 15.3%로 우승

    (이전 해 대회는 고전적인 컴퓨터 비전 기술로 25.8%가 우승)
  - 이후 대부분 컴퓨터 비전 연구자가 고전적 방법에서 딥러닝으로 전환

- **컨볼루션 신경망이 순환 신경망 또는 강화 학습과 결합 ➔ 인공지능 응용에 도전!**
  - 자연 영상 내용을 파악하여 10단어 정도로 설명하는 시스템(Image Captioning)
  - 영상 분석은 컨볼루션 신경망, 문장 생성은 순환 신경망이 담당
  - 비디오 게임을 하는 인공지능
  - 비디오 화면 분석은 컨볼루션 신경망, 게임 전략 학습은 강화 학습이 맡음
  - 알파고는 컨볼루션 신경망과 강화 학습 사용

6.2 컨볼루션 신경망의 구조와 동작

- 6.2절에서는
  - 컨볼루션 연산의 원리와 특성을 소개
  - 컨볼루션 신경망이 어떻게 복잡한 컴퓨터 비전 문제를 푸는지 설명
  - 컨볼루션 신경망을 구성하는 빌딩 블록과 빌딩 블록을 쌓아 신경망을 만드는 원리 설명

복습

6.2 컨볼루션 신경망의 구조와 동작

6.2.1 컨볼루션 연산으로 특징 맵 추출

- 컨볼루션convolution은 신호에서 “특징을 추출”하거나 “신호를 변환”하는 데 사용하는 연산
- 〈컨볼루션〉은 〈수용장 receptive field〉과 〈커널 kernel〉의 선형 결합 linear combination
  - 선형 결합이란 해당하는 요소끼리 곱한 결과를 더하는 연산
  - 예) 1차원 신호 [5]의 위치에서 선형결합 결과는 2*(−1)+9*0+9*1=7
- 컨볼루션 연산을 적용한 결과를 〈특징 맵 feature map〉 이라 부름
  - 단, [0]과 [7]의 위치에서는 커널의 특성상 특징맵을 구할 수 없음



그림 6-4 컨볼루션 연산

CNN에서의 수용장이란?
출력 뉴런에 영향을 미치는 입력 영역

6.2 컨볼루션 신경망의 구조와 동작

6.2.1 컨볼루션 연산으로 특징 맵 추출

- 식(6.1)은 1차원 컨볼루션을 식으로 쓴 것이며, z는 입력 신호이고 u는 커널이며, h는 커널의 크기임
  - 보통 커널 크기는 3, 5, 7, … 등 을 사용하는데, 홀 수를 사용해 대칭을 이룸
  - $$f(i) = z(i) \circledast u = \sum_{x=-(h-1)/2}^{(h-1)/2} z(i+x)u(x) \qquad (6.1)$$

- [그림 6-4(b)]는 2차원 컨볼루션을 설명하는 그림이며, 커널과 수용장도 2차원 구조를 가짐
  - 그림은 3x3커널을 예시하고, 위치 [5,2] 에서 계산하는 예를 보여줌
  - $$f(j,i) = z(j,i) \circledast u = \sum_{x=-(h-1)/2}^{(h-1)/2} \sum_{y=-(h-1)/2}^{(h-1)/2} z(j+y, i+x)u(y,x) \qquad (6.2)$$

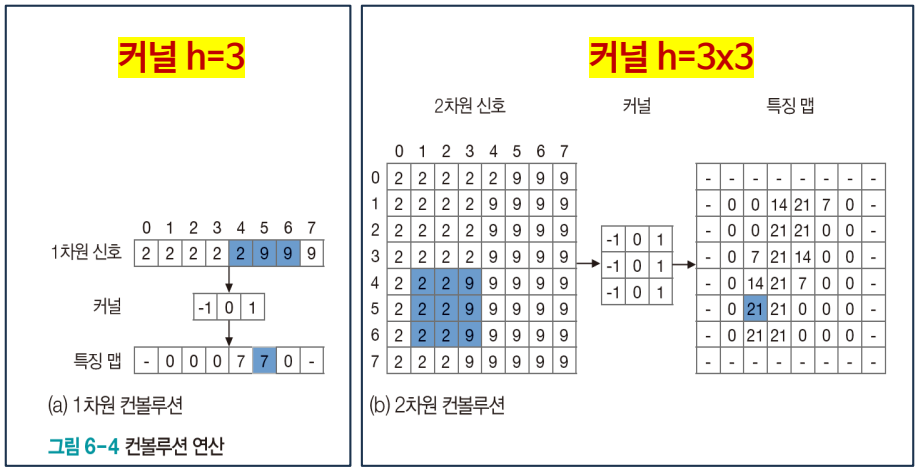


그림 6-4 컨볼루션 연산

6.2 컨볼루션 신경망의 구조와 동작

6.2.1 컨볼루션 연산으로 특징 맵 추출

- 그림 6-4(b) 2차원 커널에서 오른쪽 열은 모두 1, 왼쪽 열은 모두 -1 이며, 이 커널은 수용장의 오른쪽에서 왼쪽의 밝기 값을 빼는 계산을 하는 셈임. *수용장 : 컨볼루션 연산이 이루어지는 입력 데이터 영역
- 오른쪽이 밝고 왼쪽이 어두운 수용장은 양수가 되고, 반대라면 음수가 되며, 명암 변화가 없으면 수용장은 0이 됨.
- 해당 커널은 수직 에지(vertical edge)라는 특징을 추출하는 역할을 수행함

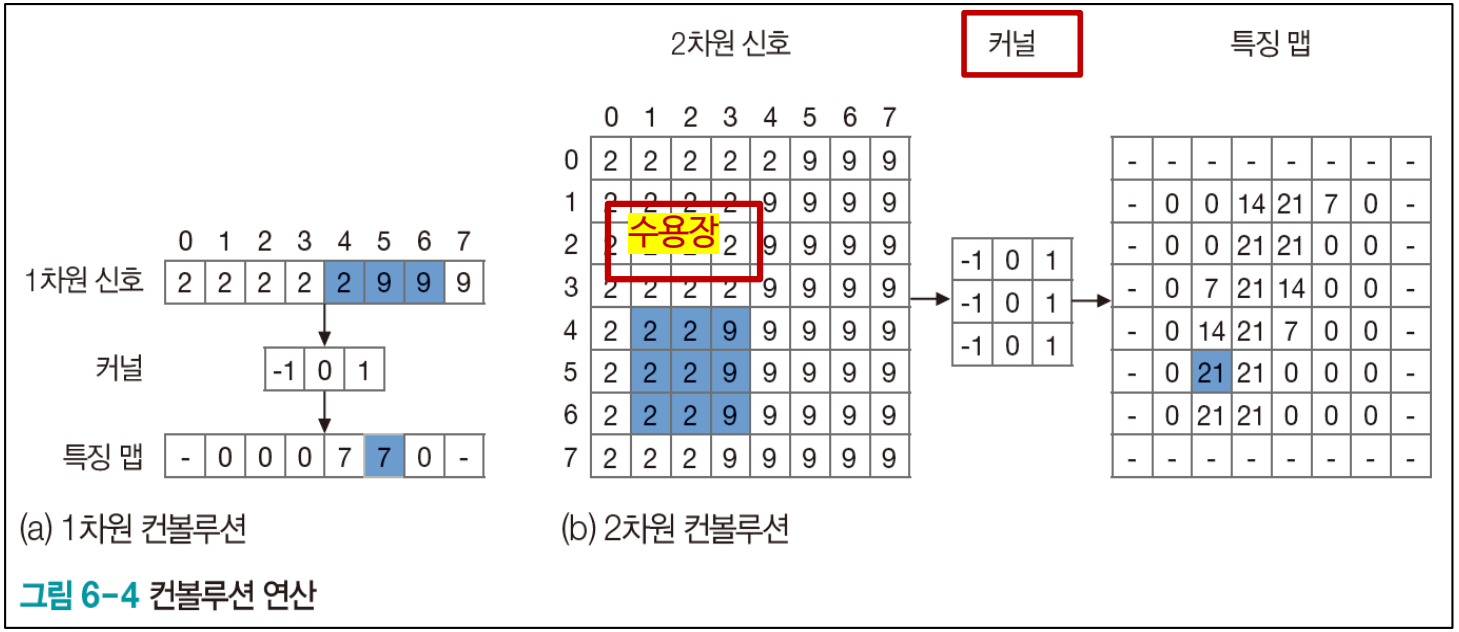


그림 6-4 컨볼루션 연산

6.2 컨볼루션 신경망의 구조와 동작

6.2.1 컨볼루션 연산으로 특징 맵 추출

- [그림 6-5]에서 보는 바와 같이 원본 영상에 컨볼루션을 적용하면 원본 영상과 형태와 크기가 같은 특징맵에 생성됨
- 이런 장점이 있는 컨볼루션을 신경망에 도입하면 입력 **데이터의 형태를 고스란히 유지한 채** 특징을 추출할 수 있게 됨
- 깊은 다층 퍼셉트론(Deep Multi-Layer Perceptron, Deep Neural Network)에 영상을 입력하려면 2차원 모양이 영상을 1차원으로 펼쳐야 하며, 픽셀은 주위 픽셀 8개와 이웃인데, 1차원으로 펼치면 이웃 정보가 사라져서 정보 손실이 큼.

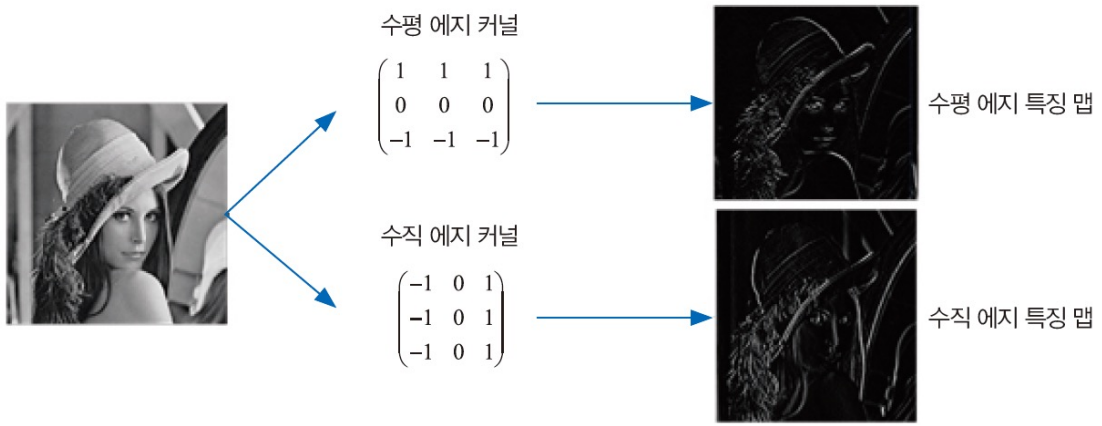


그림 6-5 에지 검출(수평 에지와 수직 에지 특징 맵)

복습

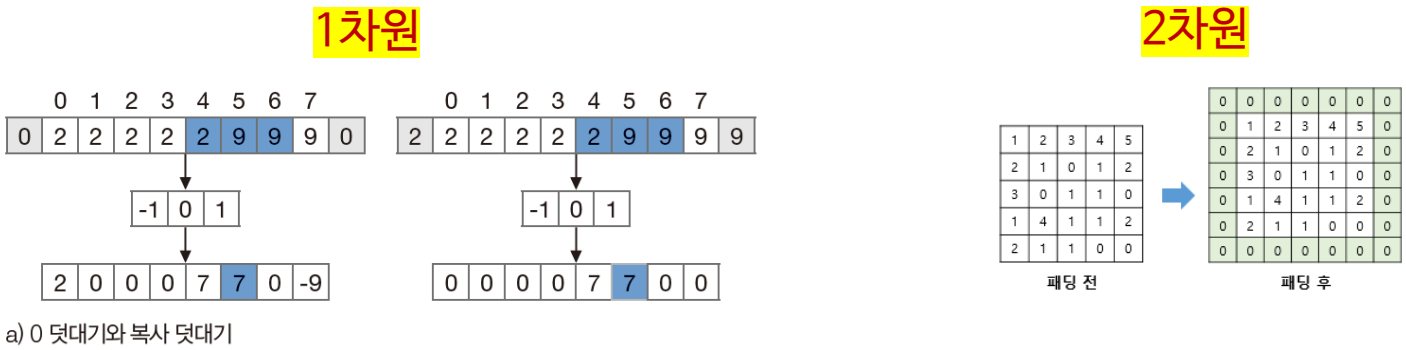
6.2 컨볼루션 신경망의 구조와 동작

6.2.2 컨볼루션층과 풀링층

컨볼루션 신경망은 〈컨볼루션으로 특징 맵을 추출하는 컨볼루션층〉과 〈요약 통계량을 구하는 풀링층〉으로 구성됨

6.2.2.1 컨볼루션층

- 가장자리가 없어지는 문제 및 해결법
  - 딥러닝에서는 컨볼루션층을 많이 쌓기 때문에 데이터의 크기가 점점 작아지는 문제가 심각함
  - 컨볼루션 커널의 크기가 클수록 빨리 작아짐
    - 예를 들어, 11x11 크기의 커널을 사용한다면 상하좌우 모두 5 만큼 씩 줄어드는데 원래 영상이 256x256이라면 컨볼루션을 한번 적용하면 246x246이 됨
  - 아래의 그림과 같이 〈0 덧대기 zero padding〉 혹은 이웃 화솟값을 복사하는 〈복사 덧대기〉를 통해 크기 유지 가능



a) 0 덧대기와 복사 덧대기

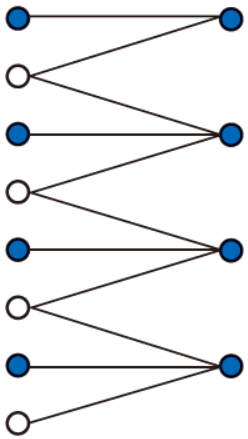
6.2 컨볼루션 신경망의 구조와 동작

복습

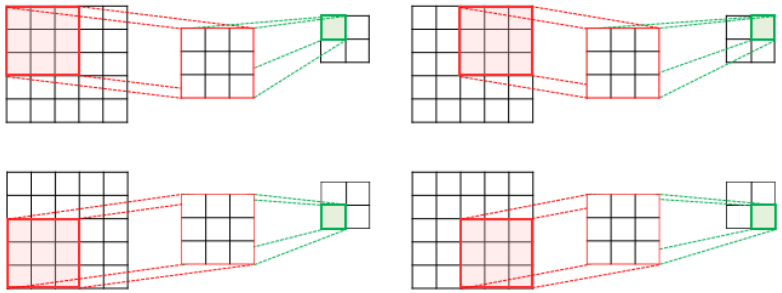
6.2.2.1 컨볼루션층

- 입력영상이 아주 큰 경우 연산량을 줄이기 위해 사용하는 보폭 stride
  - 아래의 그림은 보폭을 2로 설정한 예로, 커널을 두 화소에 한 번씩 적용함
  - 특징 맵은 입력 영상에 비해 반으로 줄었음
  - 일반적으로 보폭을 K로 설정하면 특징 맵의 크기는 1/K로 줄어듬

1차원



2차원

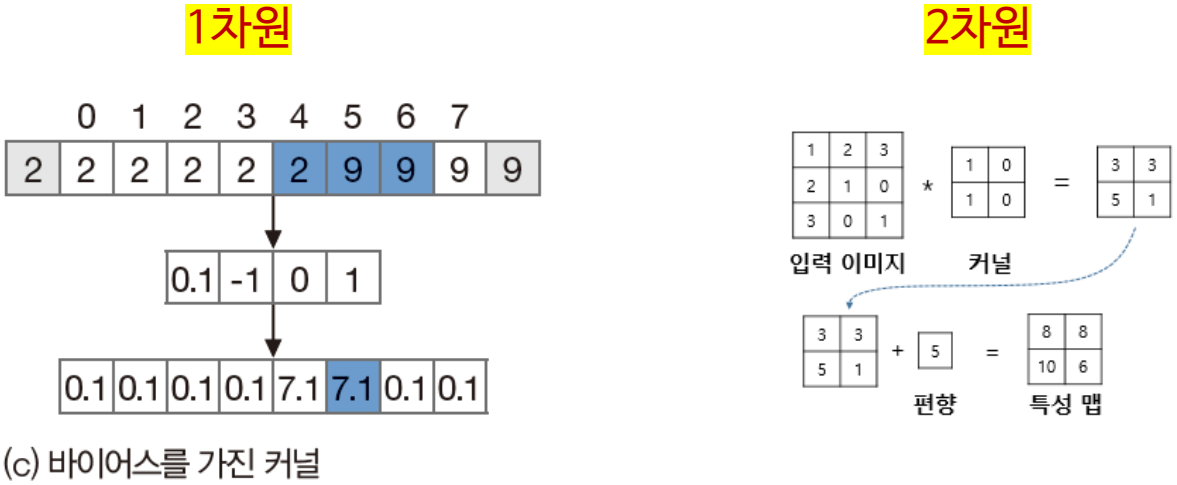


(b) 보폭=2(파란색 화소에 컨볼루션 적용)

6.2 컨볼루션 신경망의 구조와 동작

6.2.2.1 컨볼루션층

- 커널에는 바이어스 bias 가 있음
  - 아래이 그림이 보여주는 바와 같이 커널에는 바이어스가 있음



(c) 바이어스를 가진 커널

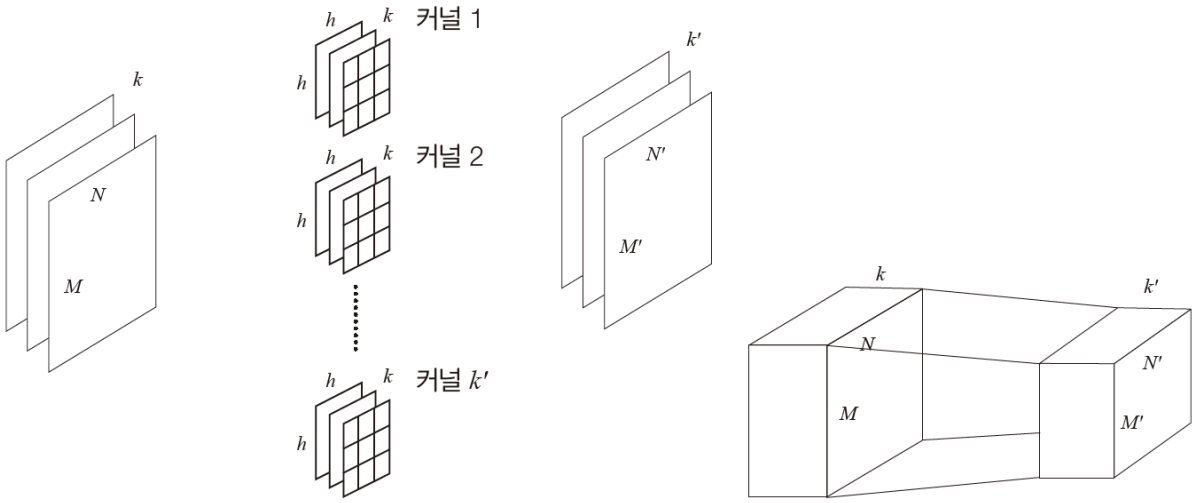
6.2 컨볼루션 신경망의 구조와 동작

복습

6.2.2.1 컨볼루션층

- 다중 커널의 사용
  - 하나의 커널은 한 종류의 특징만 추출하므로 이런 상황에서는 매우 빈약한 특징이 생성됨
  - 영상에서는 변화무쌍한 변화가 나타나므로 풍부한 특징 추출은 컨볼루션 신경망의 성공 여부를 가르는데 매우 중요한 일임
  - 컨볼루션층은 커널을 64개 또는 128개와 같이 아주 많이 사용하여 풍부한 특징 맵을 생성함
  - 커널의 개수가 K`이면 특징맵도 K` 개가 생성됨

예시) 3채널을 가진 1개의 커널



(a) 다중 커널 (b) 블록 다이어그램

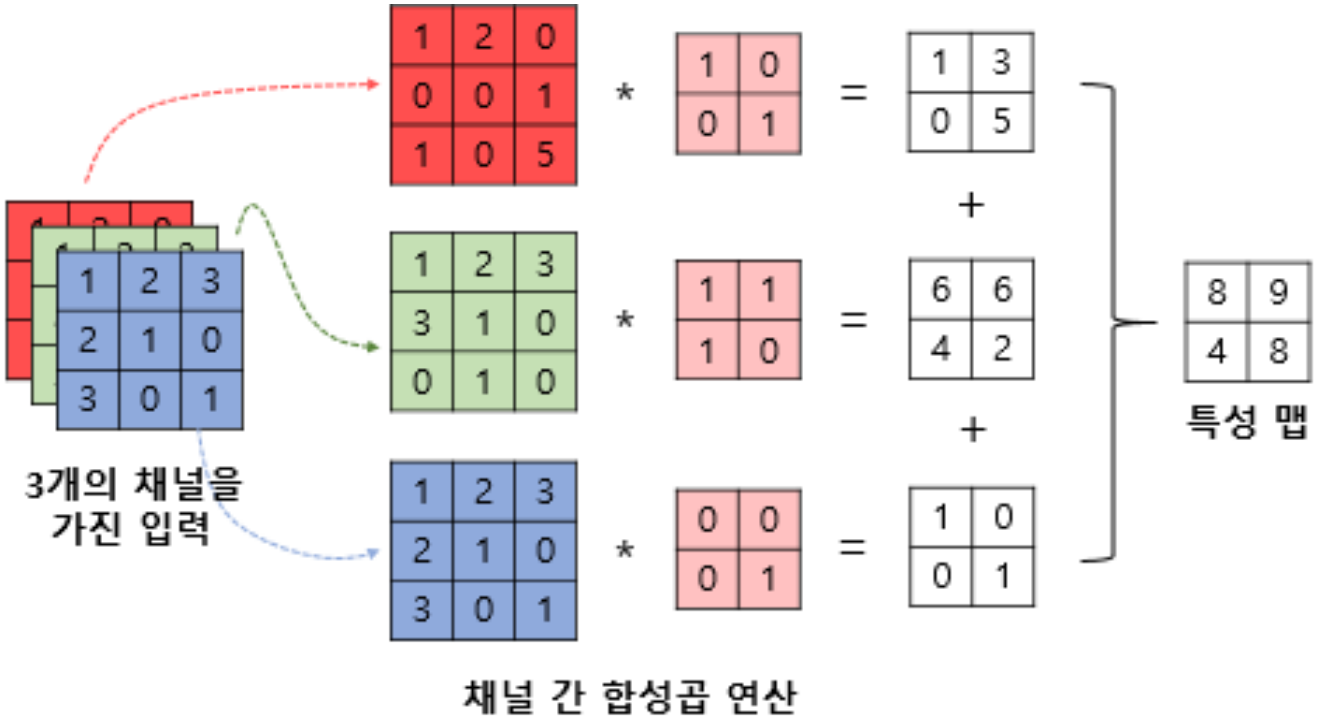
그림 6-7 컨볼루션층

6.2 컨볼루션 신경망의 구조와 동작

복습

6.2.2.1 컨볼루션층

- 다중 커널의 사용



https://wikidocs.net/64066

6.2 컨볼루션 신경망의 구조와 동작

복습

6.2.2.1 컨볼루션층

- 컨볼루션층이 수행하는 연산의 정의
  - 컨볼루션층은 식(6.2)로 구한 **특징맵 f** 에 **활성함수 τ** 를 적용함
  - 활성함수는 특징 맵의 화소 각각에 적용함

컨볼루션층의 연산: $\tau\left(f\left(j,\ i\right)\right)=\tau\left(z\left(j,\ i\right)\circledast u\right)$ (6.3)

$$f\left(j,i\right)=z\left(j,i\right)\circledast u=\sum_{x=-(h-1)/2}^{(h-1)/2}\sum_{y=-(h-1)/2}^{(h-1)/2}z\left(j+y,i+x\right)u\left(y,x\right)$$ (6.2)

6.2 컨볼루션 신경망의 구조와 동작

복습

6.2.2.2 풀링층

- 컨볼루션층 다음에는 풀링층이 위치함
- [그림6-8]은 2차원 텐서에 2x2 크기의 커널로 〈최대 풀링max pooling〉을 적용한 사례를 보여줌
- 최대 풀링은 커널 안에 있는 화수 중에서 최댓값을 취하는 연산임
- 풀링은 특징 맵에 나타난 지나친 상세함을 줄여 요약 통계량 summary statistics 을 추출함
- 따라서 보통 보폭을 1보다 크게 하는데, [그림6-8]은 보폭을 2로 설정한 경우임
- 보폭을 s로 설정하면 특징 맵의 크기는 s배 만큼 줄어듬

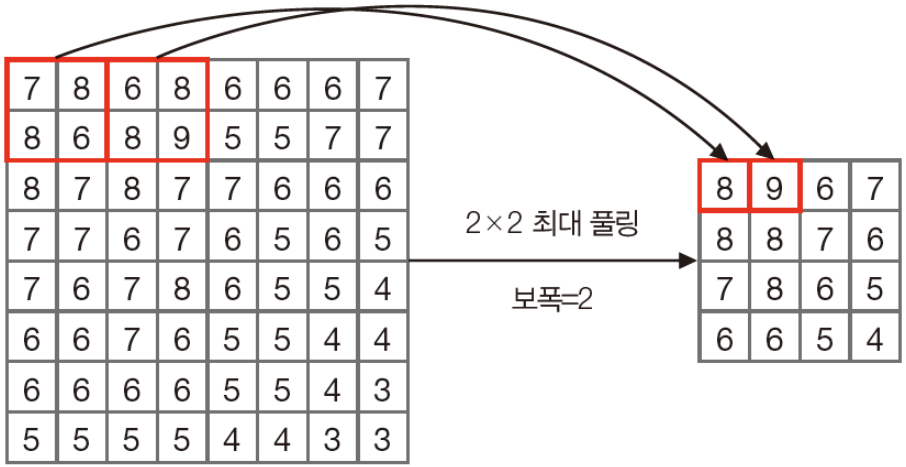


그림 6-8 최대 풀링(보폭이 2인 경우)

6.2 컨볼루션 신경망의 구조와 동작

6.2.2.2 풀링층

- 풀링에는 〈최대 풀링max pooling〉과 〈평균 풀링average pooling〉을 주로 사용함
- 최대 풀링은 커널 안에 있는 화소의 최댓값을 취하고, 평균 풀링은 평균을 취함
- 풀링층은 지나치게 상세한 특징 맵에서 핵심을 추출해 성능을 높여주는 역할뿐 아니라 특징 맵의 크기를 줄여 메모리 효율과 계산 속도를 끌어올리는 효과까지 제공함

6.2 컨볼루션 신경망의 구조와 동작

6.2.2.3 부분 연결성과 가중치 공유

- 컨볼루션층에는 부분 연결성과 가중치 공유라는 좋은 특성이 있음
- [그림6-9(b)]는 오른쪽 층에 있는 노드가 왼쪽 층 노드 3개만 에지로 연결되어 있으며, 이는 [그림5-8]의 깊은 다층 퍼셉트론에서 모든 노드 쌍이 연결된 〈완전 연결 FC: fully connected 구조〉와 확연히 다름
- 왼쪽 층과 오른쪽 층에 n_1과 n_2개의 노드가 있다면, 완전연결의 경우는 $n_1 x n_2$개의 에지가 있는 반면에 컨볼루션 신경망에는 $h x n_2$개의 에지가 있음. 커널의 크기 h는 보통 3, 5, 7 정도이므로 컨볼루션 신경망의 가중치가 훨씬 적음. 이런 특성을 부분 연결성이라고 부름

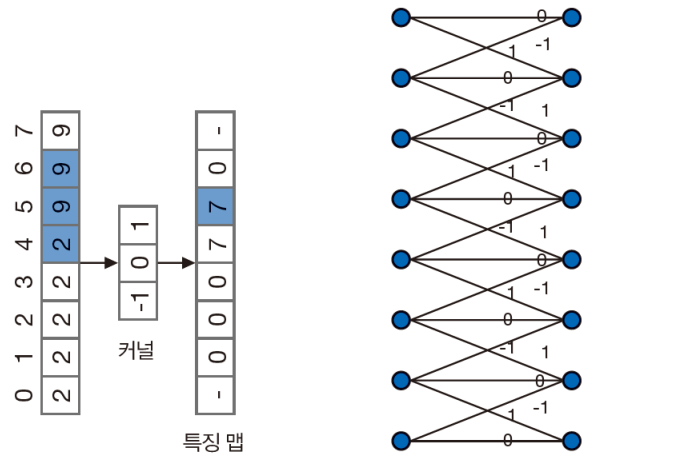


그림 6-9 컨볼루션층의 부분 연결성과 가중치 공유

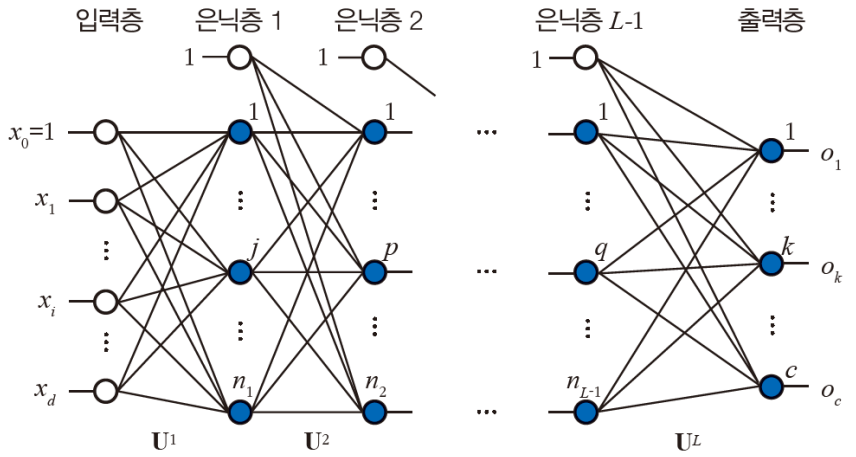


그림 5-8 깊은 다층 퍼셉트론의 구조

6.2 컨볼루션 신경망의 구조와 동작

6.2.2.3 부분 연결성과 가중치 공유

- 부분 연결성 도입 이유

필요성	설명
공간적 인접성(Locality)	이미지의 의미는 공간적으로 가까운 픽셀들 사이의 관계에 의해 형성됩니다. 멀리 떨어진 픽셀 간의 직접적인 연결은 불필요합니다.
파라미터 절감(Parameter sharing)	모든 입력 노드와 연결할 필요가 없으므로 학습해야 할 가중치 수가 대폭 줄어듭니다.
학습 효율 향상	파라미터가 줄어들면 과적합을 방지하고, 학습 속도가 빨라집니다.
시각적 패턴 탐지	필터가 이미지 내의 엣지, 코너, 질감 등의 국소적 특징(local feature)을 잘 포착합니다.

6.2 컨볼루션 신경망의 구조와 동작

6.2.2.3 부분 연결성과 가중치 공유

- 컨볼루션 신경망에서는 커널을 구성하는 값이 가중치(커널1x3)에 해당하는데,
  깊은 다층 퍼셉트론에서는 인접한 두 층의 노드 쌍을 연결하는 에지에 가중치가 있음
- 따라서 컨볼루션 신경망과 깊은 다층 퍼셉트론은 학습 방식이 크게 다름
- [그림6-9]는 학습에 관련된 〈가중치 공유 weight sharing 〉라는 컨볼루션 신경망의 또 다른 장점을 설명함

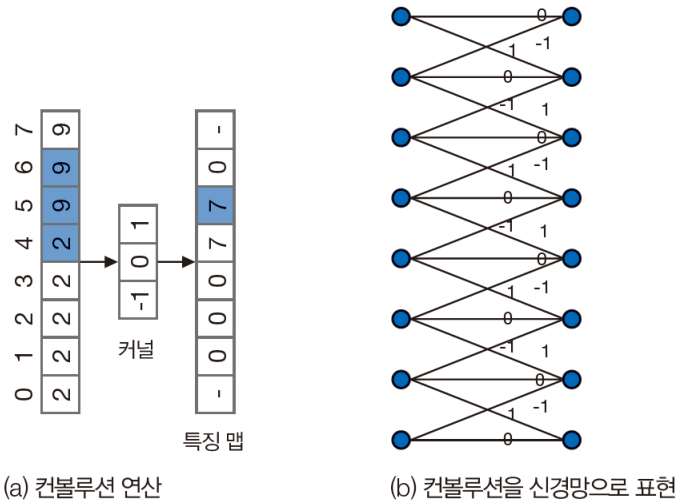


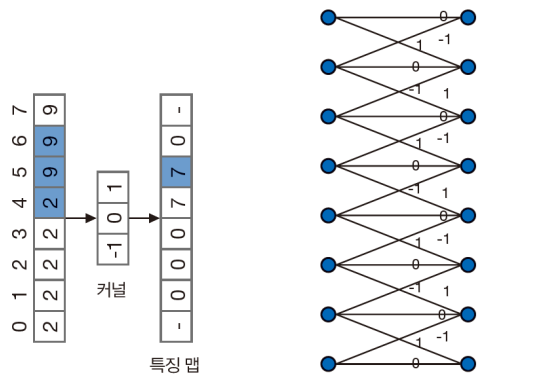
그림 6-9 컨볼루션층의 부분 연결성과 가중치 공유

CNN의 가중치 수 〈〈 MLP의 가중치 수

6.2 컨볼루션 신경망의 구조와 동작

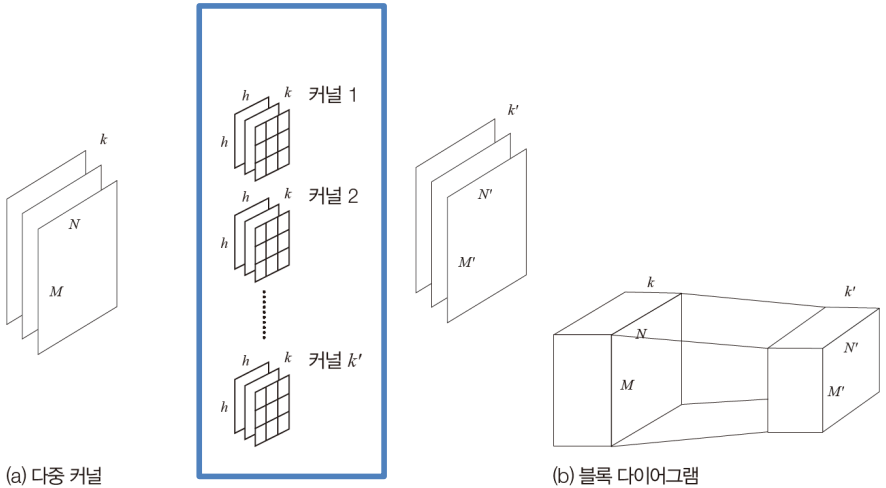
6.2.2.3 부분 연결성과 가중치 공유

- 입력 영상의 모든 곳에 같은 커널을 적용하므로 모든 화소가 **하나의 커널을 공유**하는 셈임
- 가중치 공유는 학습이 최적화해야 하는 매개변수의 개수를 획기적으로 줄여줌
- [그림6-9(b)]에는 hxn_2개의 에지가 있지만, 가중치 공유에 따라 학습이 알아내야 하는 매개변수는 h개에 불과함
- [그림6-7(a)]가 보여주는 컨볼루션층은 hxhxk 크기의 커널 k`개 를 사용하므로,
  하나의 컨볼루션층에서 학습 알고리즘이 최적화해야 하는 매개변수는 k`xhxhxk 개 임
  - k 는 3, k`은 보통 32, 64, 128 이고, h는 3, 5, 7 이므로, 다층 퍼셉트론의 완전연결층에 비하면 매개변수 개수가 획기적으로 줄어듬



(a) 컨볼루션 연산 (b) 컨볼루션을 신경망으로 표현

그림 6-9 컨볼루션층의 부분 연결성과 가중치 공유



(a) 다중 커널 (b) 블록 다이어그램

그림 6-7 컨볼루션층

6.2 컨볼루션 신경망의 구조와 동작

복습

6.2.3 빌딩 블록을 쌓아 만드는 컨볼루션 신경망

- 컨볼루션층과 풀링층을 가지고 컨볼루션 신경망을 구축하며, 빌딩 블록을 쌓아가는 방식으로 만듦

6.2.3.1 빌딩 블록

- [그림5-8]의 깊은 다층 퍼셉트론은 은닉층을 쌓아서 만들며, 은닉층 하나가 빌딩 블록에 해당함
- [그림 6-10(a)]는 컨볼루션 신경망이 사용하는 빌딩 블록임
  - 빌딩 블록의 단위 [컨볼루션-활성함수-풀링] : 예)conv-relu-maxpooling
- 빌딩 블록의 출력 텐서는 다음 빌딩 블록의 입력 텐서로 사용됨

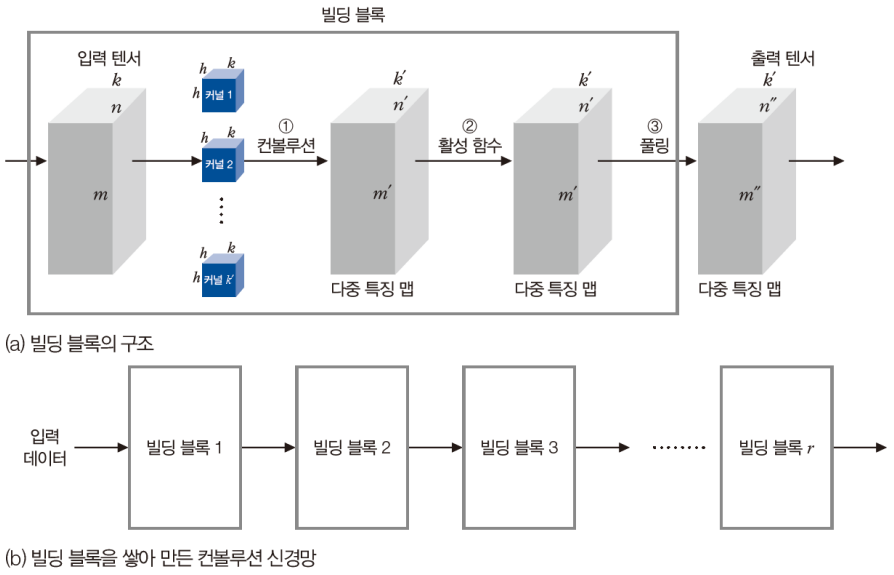


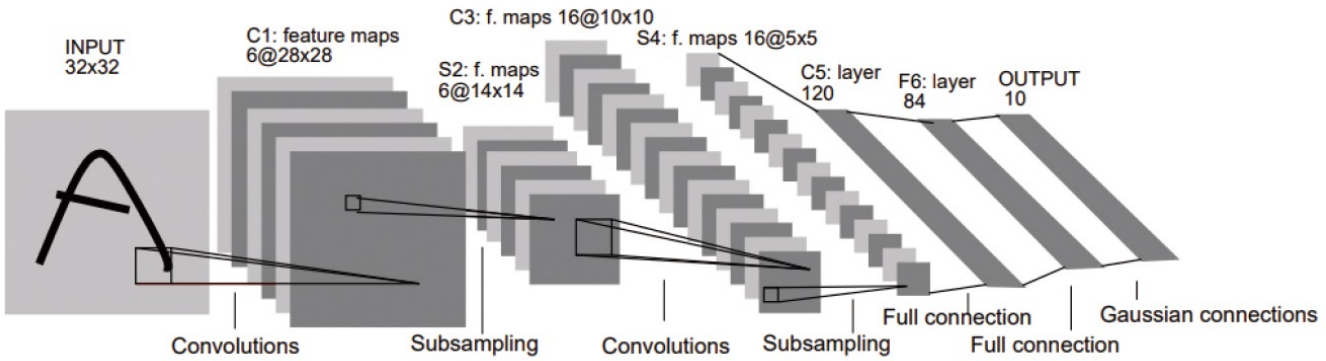
그림 6-10 컨볼루션 신경망의 구조

6.2 컨볼루션 신경망의 구조와 동작

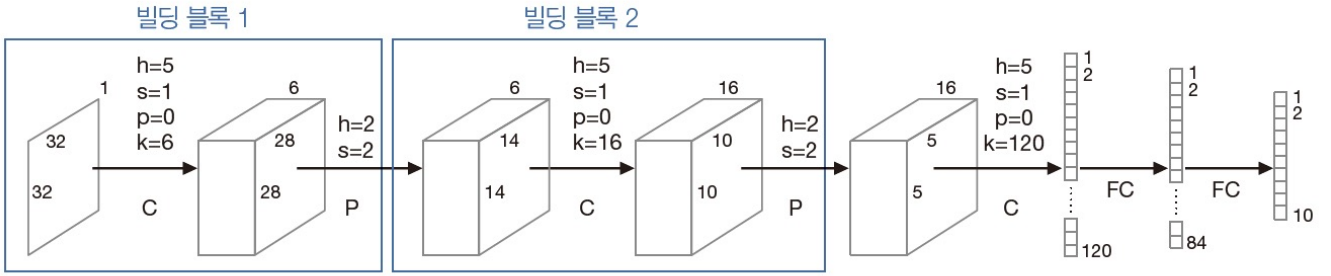
복습

6.2.3.2 LeNet-5 사례

- [그림6-11]은 LeNet-5인데, 현재 기준으로 보면 아주 간단한 구조의 컨볼루션 신경망임
- LeNet-5는 필기 숫자 인식 문제에서 컨볼루션 신경망의 가능성을 최초로 입증했다는 점에서 의미가 큼
- 필기 숫자 영상은 명암 영상이므로 입력 데이터이 깊이는 1이고, C는 컨볼루션층, P는 풀링층, FC는 완전연결 층임



(a) [LeCun1998]의 그림



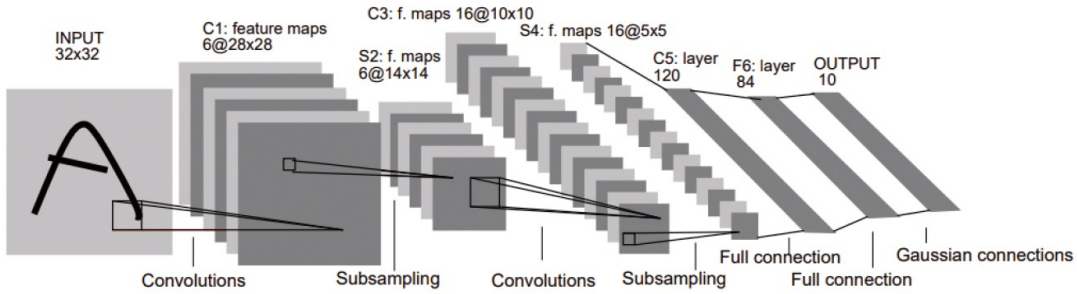
(b) 이 책의 방식에 따른 그림

그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)

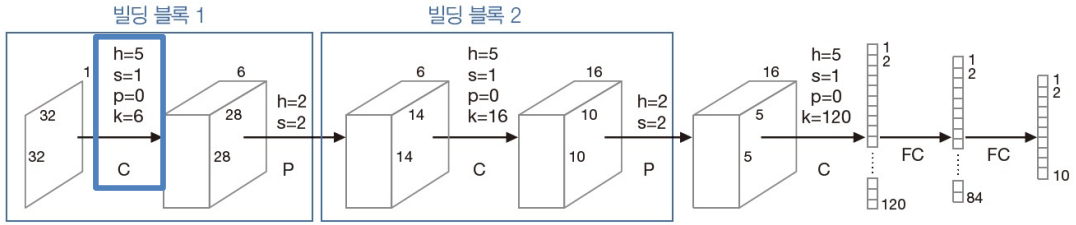
6.2 컨볼루션 신경망의 구조와 동작

6.2.3.2 LeNet-5 사례

- 첫 번째 층은 컨볼루션층인데 5x5커널을 6개 사용하고 보폭은 1, 덧대기는 하지 않음
  - 1x32x32 의 텐서에서 28x28 크기의 특징 맵을 6개 추출해 6x28x28의 텐서를 출력함
- 다음은 풀링층인데 2x2커널을 보폭 2로 적용해 6x14x14의 텐서가 출력되며, 여기까지 빌딩 블록 1임
- 다음은 컨볼루션층인데 입력은 풀링층에서 받은 6x14x14의 텐서이고 출력은 16x10x10텐서임
- 이어지는 풀링층은 2x2커널을 보폭2로 적용해 16x5x5텐서를 출력되며, 여기까지 빌딩 블록 2임
- 이후에 컨볼루션 연산을 한 번 더 적용하고, 완전연결(FC)을 두 번 적용함



(a) [LeCun1998]의 그림



(b) 이 책의 방식에 따른 그림

5*5*16=400 (flatten x)

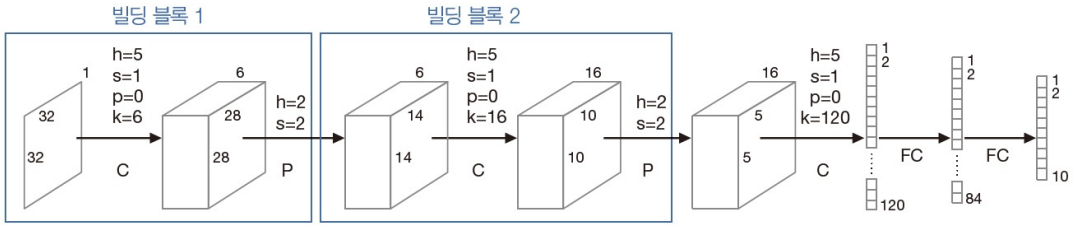
그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)

6.2 컨볼루션 신경망의 구조와 동작

6.2.3.2 LeNet-5 사례

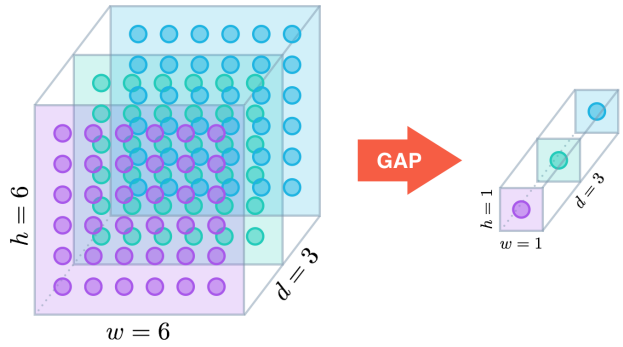
- 전체적으로 LeNet-5는 C-P-C-P-C-FC-FC 층으로 구성되며, 2개의 FC는 다층 퍼셉트론으로 볼 수 있음
- LeNet-5의 학습 알고리즘이 최적화해야 하는 매개변수의 개수는 3,692 + 11,014
  - **3개의 컨볼루션층**은 왼쪽에서 오른쪽으로 가면서 5x5커널 6개, 5x5커널 16개, 5x5커널 120개를 사용하므로 매개변수는 총 (5x5+1)x6+(5x5+1)x16+(5x5+1)x120 = 3,692개
  - **완전연결층**의 매개변수는 총 (120+1)x84+(84+1)x10=11,014개 ➔ 컨볼루션층의 3배 이상

- 초창기 컨볼루션 신경망은 완전연결층이 과다한 매개변수를 가지는 구조를 사용했는데, 이후에는 완전연결층을 〈전역 평균 풀링 GAP: global average pooling〉과 같은 구조로 대체하여 매개변수 수를 줄이는 방향으로 발전함



(b) 이 책의 방식에 따른 그림

그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)



6.3 컨볼루션 신경망의 학습

6.3.1 손실 함수와 옵티마이저

- 손실 함수는 신경망이 범하는 오류를 측정함
- 컨볼루션 신경망의 내부구조는 깊은 다층 퍼셉트론과 다르지만 출력층은 같음
  - 예) 숫자 인식을 하는 신경망이라면 출력층에 노드가 10개 있고 softmax 또는 tanh 활성함수의 결과 출력

- 옵티마이저는 손실 함수의 최저점을 찾아감
- 컨볼루션 신경망은 커널이 매개변수에 해당하고, 깊은 다층 퍼셉트론은 에지의 가중치가 매개변수라는 점만 다르고 미분으로 방향을 알아내는 원리는 동일함
  - 따라서 컨볼루션 신경망과 깊은 다층 퍼셉트론은 같은 옵티마이저를 사용함

6.3 컨볼루션 신경망의 학습

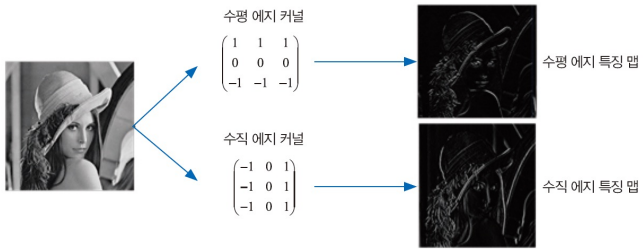


그림 6-5 에지 검출(수평 에지와 수직 에지 특징 맵)

6.3.2 통째 학습 end-to-end learning

- 수작업 특징 hand-crafted features
  - 고전적인 컴퓨터 비전에서는 [그림6-5]와 같이 수평 에지와 수직 에지 특징을 추출하는 커널을 사람이 직접 설계했음
  - 수작업 특징은 사람의 직관으로 설계되므로 어느 정도 수준의 성능을 제공하지만, 자연 영상과 같이 복잡한 데이터에서는 아주 낮은 성능에 머물러 있음
  - [그림6-12(a)] 는 수작업 특징을 사용하는 고전적인 컴퓨터 비전 방식을 보여 줌
- 특징 학습 feature learning
  - 딥러닝은 특징을 추출하는 방식에서 패러다임 변화를 일으킴
  - [그림 6-12(b)]는 특징 추출과 분류를 동시에 학습으로 알아내는 방식이며, 딥러닝은 특징을 학습으로 알아내기 때문에 특징 학습을 한다고 말함
  - 또한 입력 패턴에서 최종 출력에 이르는 전체 과정을 한꺼번에 학습한다는 의미에서 통째 학습end-to-end learning 을 한다고 함
  - 통째 학습은 딥러닝의 월등한 성능을 설명하는 가장 중요한 요임임

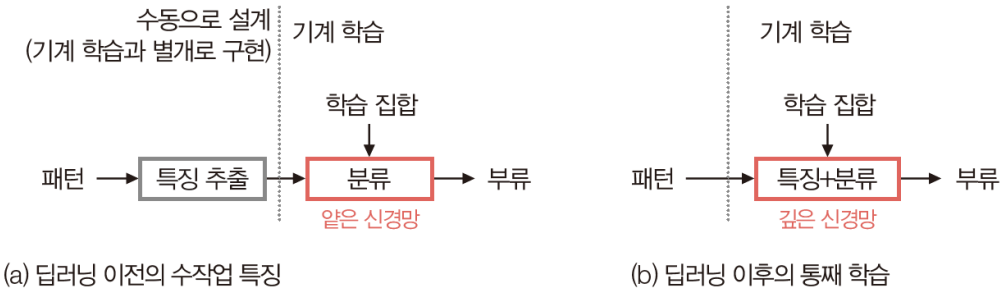


그림 6-12 딥러닝에 의한 컴퓨터 비전 패러다임 변화

6.3 컨볼루션 신경망의 학습

6.3.2 통째 학습 end-to-end learning

- 컨볼루션 신경망은 딥러닝의 일종이므로 딥러닝 패러다임을 따름
- LeNet-5에 [그림6-12(b)]의 패러다임을 씌워 딥러닝의 원리를 좀 더 구체적으로 설명하면, 앞부분의 컨볼루션층과 풀링층은 특징 추출을, 뒷부분의 완전연결층은 분류를 담당함
- 컨볼루션 신경망의 학습 알고리즘은 깊은 다층 퍼셉트론과 마찬가지로 오류 역전파 알고리즘을 사용함
  - 1986년 Hinton 교수님이 네이처에 발표한 논문을 통해 현대적 역전파 알고리즘의 대중화를 이루었음
- 단, 최적화해야 할 매개변수가 완전연결층의 가중치 뿐만 아니라 컨볼루션층이 커널도 포함된다는 점만 다름

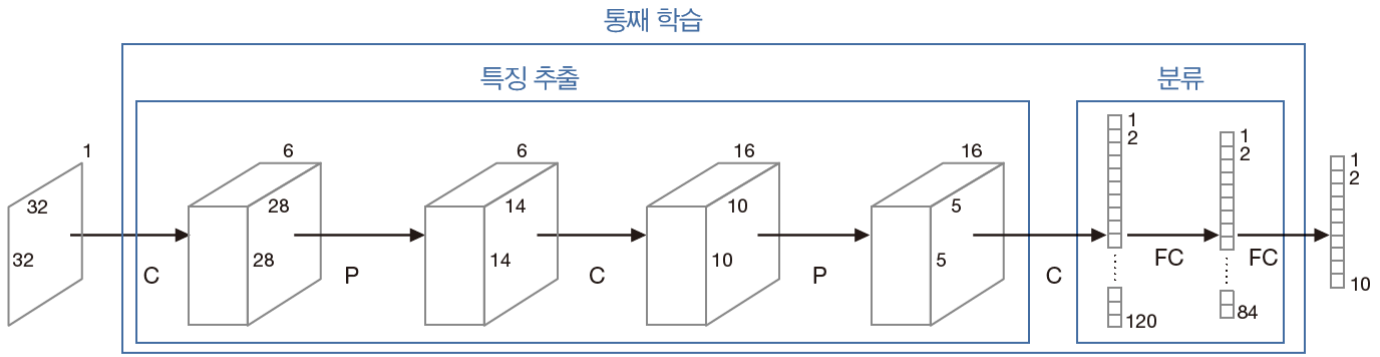


그림 6-13 특징 추출과 분류를 동시에 학습하는 딥러닝의 통째 학습

6.3 컨볼루션 신경망의 학습

6.3.3 컨볼루션 신경망의 성능이 월등한 이유

- “통째 학습을 한다”
  - 특징 추출과 분류를 동시에 최적화하는 학습 방법은 따로 최적화한 후에 결합하는 학습방법보다 뛰어남

- “특징 학습을 한다”
  - 주어진 데이터셋에 최적인 특징 추출 알고리즘을 학습으로 알아내기 때문에 상당한 성능향상이 있음

- “신경망의 깊이를 더욱 깊게 하여 풍부한 특징을 추출한다”
  - 컨볼루션층은 부분 연결성과 가중치 공유로 인해 매개변수 개수가 적음. 따라서 신경망의 깊이를 충분히 깊게 해도 학습이 잘됨

- “컨볼루션 연산은 데이터의 원래 구조를 그대로 유지한 채 특징을 추출한다”
  - 입력 데이터가 컬러 영상의 3차원 텐서라면 컨볼루션층이 출력하는 특징 맵도 3차원 텐서임
  - 영상의 한 화소는 상하좌우에 있는 이웃 화소와 상관관계가 큰데, 컨볼루션 연산은 상관 관계 정보를 그대로 유지하는 강점이 있음.
  - 깊이 다층 퍼셉트론에서는 영상을 1차원으로 펼쳐 입력하기 때문에 이웃 정보가 사라짐

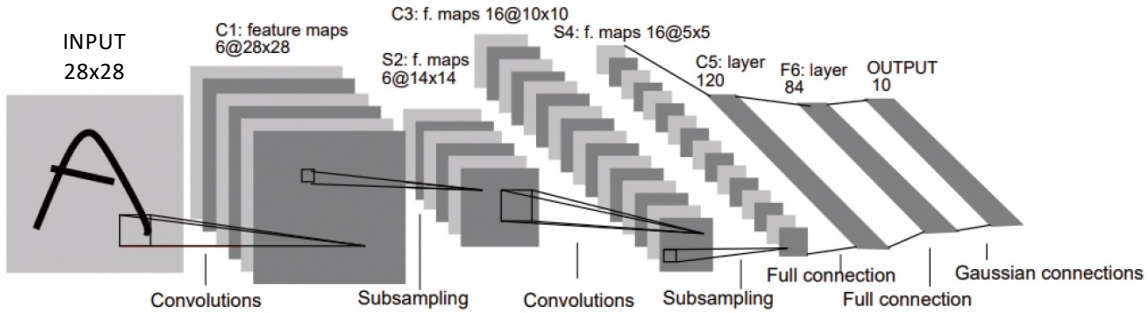
6.4 컨볼루션 신경망 프로그래밍

6.4.1 필기 숫자 인식

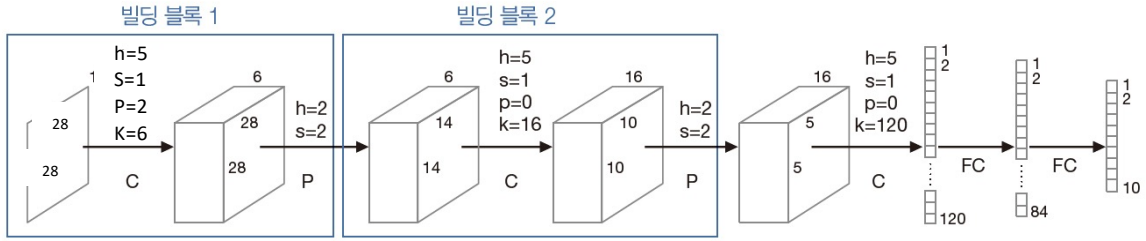
6.4.1.1 LeNet-5

[프로그램 6-1] LeNet-5로 MNIST 인식 (C-P-C-P-C-FC-FC)

주의: 앞장과 영상 입력 사이즈가 상이함



(a) [LeCun1998]의 그림



(b) 이 책의 방식에 따른 그림

그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p1-codingnote

6.4 컨볼루션 신경망 프로그래밍

6.4.1 필기 숫자 인식

6.4.1.1 LeNet-5

[프로그램 6-1] LeNet-5로 MNIST 인식

파이토치를 이용한 DNN 모델 학습 코드를 LeNet-5로 수정하여 사용해보자

5.7 딥러닝이 사용하는 손실 함수

5.7.3 손실 함수의 성능 비교 실험 (MNIST)

[프로그램5-10] 손실 함수의 성능 비교: 평균제곱오차와 교차 엔트로피

**Torch.nn.MSELoss()의 정답 라벨은 one-hot 형태로 입력되어야 함

```
def train_epoch(train_dataloader, optimizer, model, loss_fn, log_dict, loss_type):
    acc_train = 0
    loss_train = 0
    model.train()

    for X, y in train_dataloader: #학습 데이터 배치단위 로드
        X = X.view(-1, 784).cuda()
        y = y.cuda()

        pred = model(X)           # 모델 forward

        target = y if loss_type == "ce" else F.one_hot(y, num_classes=10).float()
        cost = loss_fn(pred,target) # 손실함수 계산
```

```
# 손실함수 비교 실험을 위한 정의
loss_mse = torch.nn.MSELoss()
loss_ce = torch.nn.CrossEntropyLoss()

# 모델 학습
for iter in tqdm.tqdm(range(30)):
    DNN_adam, log_ce = train_epoch(train_dataloader, optimizer_adam, DNN_adam, loss_ce, log_ce, "ce")
    # 모델 테스트
    with torch.no_grad():
        DNN_adam, log_ce = test_epoch(test_dataloader, optimizer_adam, DNN_adam, loss_ce, log_ce, "ce")

# 모델 학습
for iter in tqdm.tqdm(range(30)):
    DNN_adam, log_mse = train_epoch(train_dataloader, optimizer_adam, DNN_adam, loss_mse, log_mse, "mse")
    # 모델 테스트
    with torch.no_grad():
        DNN_adam, log_mse = test_epoch(test_dataloader, optimizer_adam, DNN_adam, loss_mse, log_mse, "mse")
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w5-p7-codingnote

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-1] LeNet-5로 MNIST 인식 (모델 구현부)

```
# 입력 사이즈만 변경한 오리지널 LeNet5
class LeNet5v2(torch.nn.Module):
    def __init__(self):
        super(LeNet5v2, self).__init__()

        self.input_layer = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=1, out_channels=6, kernel_size=(5,5), stride=1, padding=2),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=(2, 2))
        )
        self.hidden_layer_1 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=6, out_channels=16, kernel_size=(5,5), stride=1, padding=0),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=(2, 2))
        )
        self.hidden_layer_2 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=16, out_channels=120, kernel_size=(5,5), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.Flatten = torch.nn.Flatten()  # 3차원 feature map을 1차원 벡터로 펼쳐주는(Flatten) 레이어
        self.hidden_layer_3 = torch.nn.Sequential(
            torch.nn.Linear(in_features=120*1*1, out_features=84),
            torch.nn.ReLU()
        )
        self.classifier = torch.nn.Linear(in_features=84, out_features=10)

    def forward(self, input):
        """
        input shape: (1, 28, 28)
        output shape: (10)
        """
        x = self.input_layer(input)  # (1, 28, 28) -> (6, 14, 14)
        x = self.hidden_layer_1(x)   # (6, 14, 14) -> (16, 5, 5)
        x = self.hidden_layer_2(x)   # (16, 5, 5)  -> (120, 1, 1)
        x = self.Flatten(x)          # (120, 1, 1) -> (120)
        x = self.hidden_layer_3(x)   # (120) -> (84)
        x = self.classifier(x)       # (84) -> (10)
        return x
```

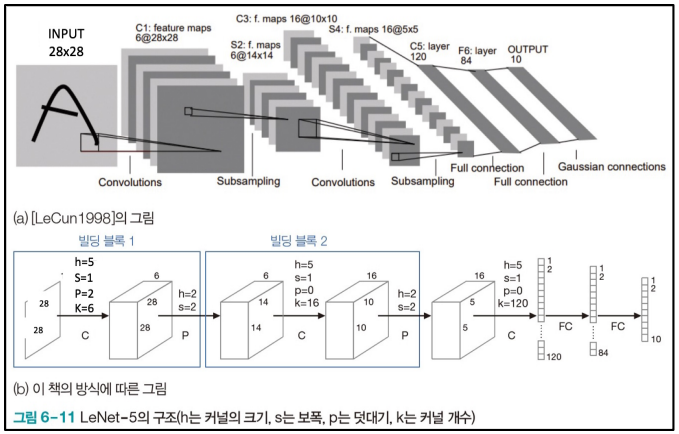


그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)

1채널의 이미지를 입력받는 첫 컨볼루션 계층 정의

(채널, 너비, 높이) 3채널의 특징 맵을 1차원 벡터로 변환하는 계층

모든 은닉층을 통과한 특징 벡터를 이용해 예측을 수행하는 계층

실제로 입력 데이터가 각 계층을 통과하는 과정인 forward 함수 정의

각 계층을 통과하며 변화하는 데이터의 형태(shape)에 주목

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-1] LeNet-5로 MNIST 인식 (데이터 처리부)

```
127 def train_epoch(train_dataloader, optimizer, model, loss_fn, log_dict, loss_type):
128     acc_train = 0
129     loss_train = 0
130     model.train()
131 
132     for X, y in train_dataloader: #학습 데이터 배치단위 로드
133         # DNN 입력 [128,1,28,28]=>[128,784]
134         # X = X.view(-1, 784).cuda()
135         # CNN은 입력 [128, 1, 28, 28] = [Batch, Ch, H, W]
136         X = X.cuda()
137         y = y.cuda()
138 
139         pred = model(X)           # 모델 forward
140 
141         target = y if loss_type == "ce" else F.one_hot(y, num_classes=10).float()
142         cost = loss_fn(pred,target) # 손실함수 계산
143 
144         optimizer.zero_grad()     # 기울기 초기화 필요 (파이토치가 기울기를 축적하여 사용하다보니..)
145         cost.backward()           # 역전파 계산
146         optimizer.step()          # 가중치 갱신
147 
148         acc_train += torch.sum(torch.argmax(pred, dim=1)==y).item()
149         loss_train += cost.item()
150 
151     # 학습 로그 기록
152     log_dict['train_acc'].append(acc_train/len(train_dataloader.dataset))
153     log_dict['train_loss'].append(loss_train)
154 
155     return model, log_dict
```

136행 데이터 입력 부분 수정 없이 CNN 입력으로 사용

6.4 컨볼루션 신경망 프로그래밍

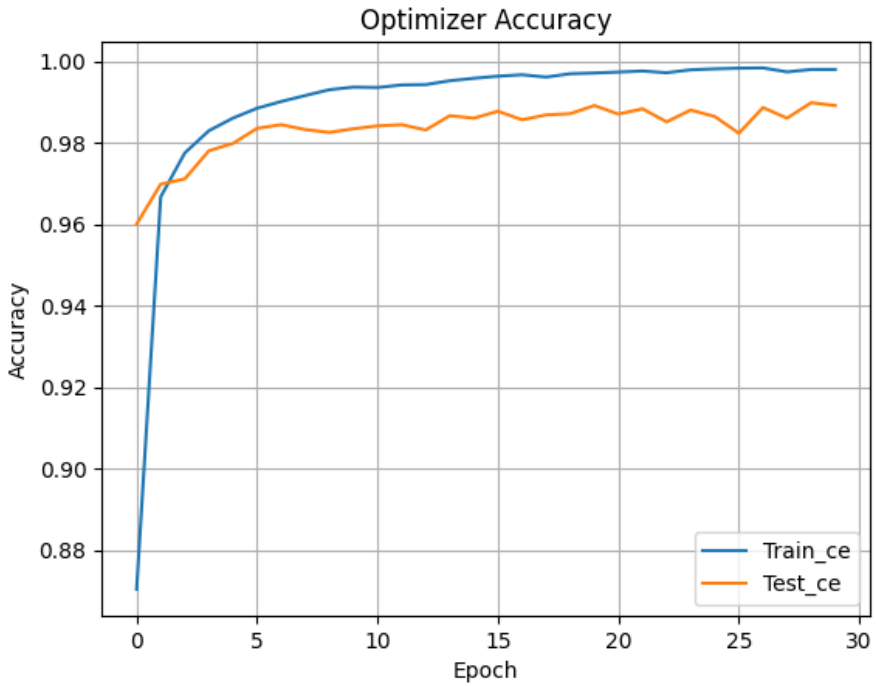
[프로그램 6-1] LeNet-5로 MNIST 인식 (데이터 처리부)

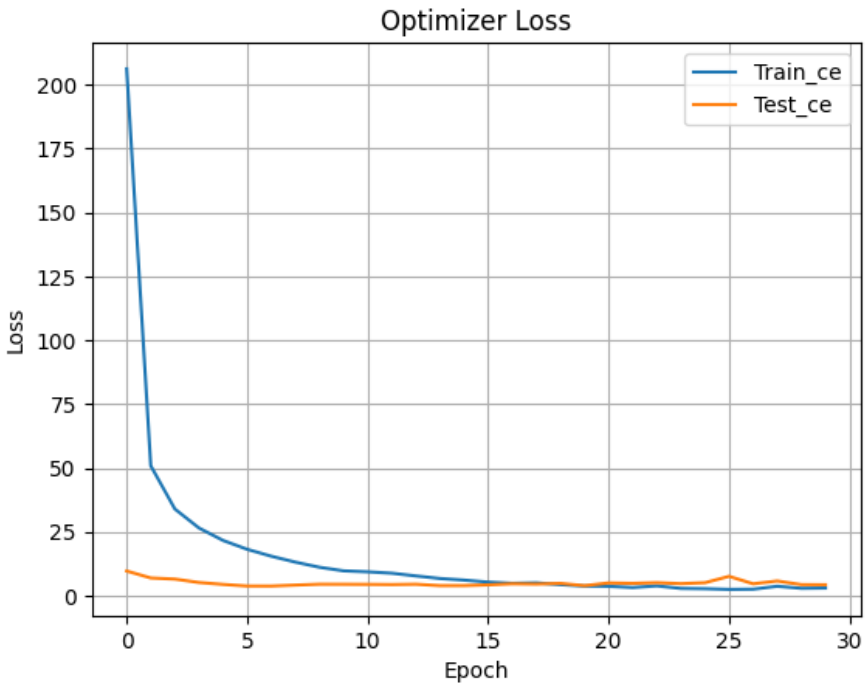
```
159 def test_epoch(test_dataloader, optimizer, model, loss_fn, log_dict, loss_type): # train_epoch와 패턴 유사
160     acc_test = 0
161     loss_test = 0
162     model.eval()
163 
164     for X, y in test_dataloader: #테스트 데이터 배치단위 받기
165         # DNN 입력 [128,1,28,28]=>[128,784]
166         # X = X.view(-1, 784).cuda()
167         # CNN은 입력 [128, 1, 28, 28] = [Batch, Ch, H, W]
168         X = X.cuda()
169         y = y.cuda()
170 
171         pred = model(X)
172         target = y if loss_type == "ce" else F.one_hot(y, num_classes=10).float()
173         cost = loss_fn(pred,target) # 손실함수 계산
174 
175         # 학습 단계가 아니므로, 아래의 단계 생략
176         #optimizer.zero_grad()
177         #cost.backward()
178         #optimizer.step()
179 
180         acc_test += torch.sum(torch.argmax(pred, dim=1)==y).item()
181         loss_test += cost.item()
182 
183     log_dict['test_acc'].append(acc_test/len(test_dataloader.dataset))
184     log_dict['test_loss'].append(loss_test)
185 
186     return model, log_dict
```

168행 데이터 입력 부분 수정 없이 CNN 입력으로 사용

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-1] LeNet-5로 MNIST 인식 학습을 마친 후 정확률과 손실 값을 시각화



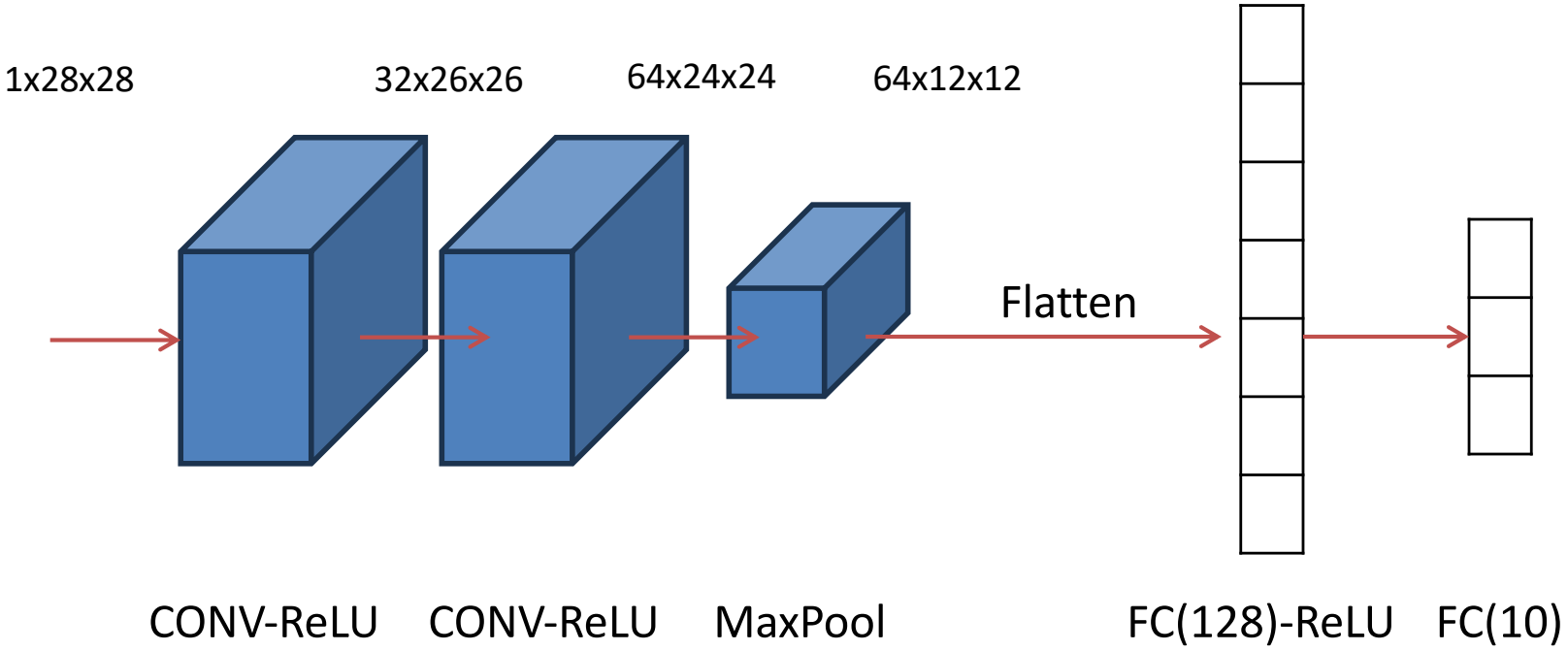


6.4 컨볼루션 신경망 프로그래밍

6.4.1 필기 숫자 인식

6.4.1.2 컨볼루션 신경망의 유연한 구조

[프로그램 6-2] 컨볼루션 신경망으로 MNIST 인식 (C-C-P-dropout-FC-dropout-FC)



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p2-codingnote

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-2] 컨볼루션 신경망으로 MNIST 인식 (모델 구현)

```
166 # 교재에 있는 CNNv2
167 class CNNv2(torch.nn.Module):
168     def __init__(self):
169         super(CNNv2, self).__init__()
170 
171         self.input_layer = torch.nn.Sequential(
172             torch.nn.Conv2d(in_channels=1, out_channels=32, kernel_size=(3,3), stride=1, padding=0),
173             torch.nn.ReLU(),
174         )
175         self.hidden_layer_1 = torch.nn.Sequential(
176             torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
177             torch.nn.ReLU(),
178         )
179         self.dropout_1 = torch.nn.Dropout(p=0.25)
180         self.maxpool = torch.nn.MaxPool2d(kernel_size=(2, 2))
181         self.dropout_1 = torch.nn.Dropout(p=0.25)
182         self.flatten = torch.nn.Flatten()
183         self.hidden_layer_2 = torch.nn.Sequential(
184             torch.nn.Linear(in_features=64*12*12, out_features=128),
185             torch.nn.ReLU()
186         )
187         self.dropout_2 = torch.nn.Dropout(p=0.5)
188         self.classifier = torch.nn.Linear(in_features=128, out_features=10)
189 
190     def forward(self, input):
191         """
192         input shape: (1, 28, 28)
193         output shape: (10)
194         """
195         x = self.input_layer(input)
196         x = self.hidden_layer_1(x)
197         x = self.maxpool(x)
198         x = self.dropout_1(x)
199         x = self.flatten(x)
200         x = self.hidden_layer_2(x)
201         x = self.dropout_2(x)
202         x = self.classifier(x)
203         return x
204 
```

LeNet-5와 다른 구조의 CNN을 설계하여 MNIST 인식 수행
(모델 정의 부분을 제외하고 [프로그램 6-1]과 같은 코드 사용

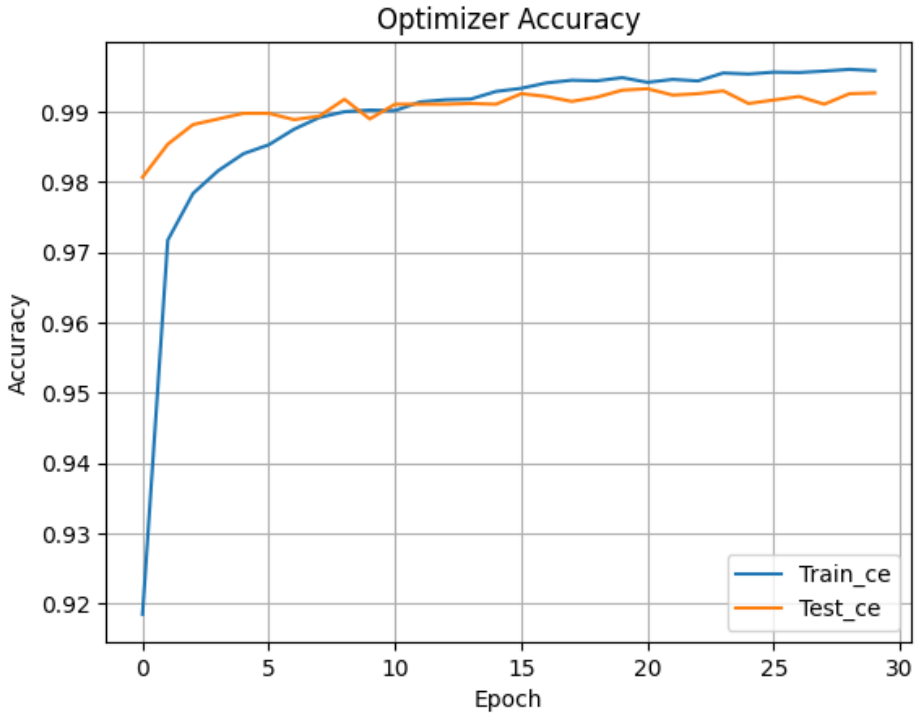
서로 다른 드롭아웃 확률을 갖는 드롭아웃 계층 사용

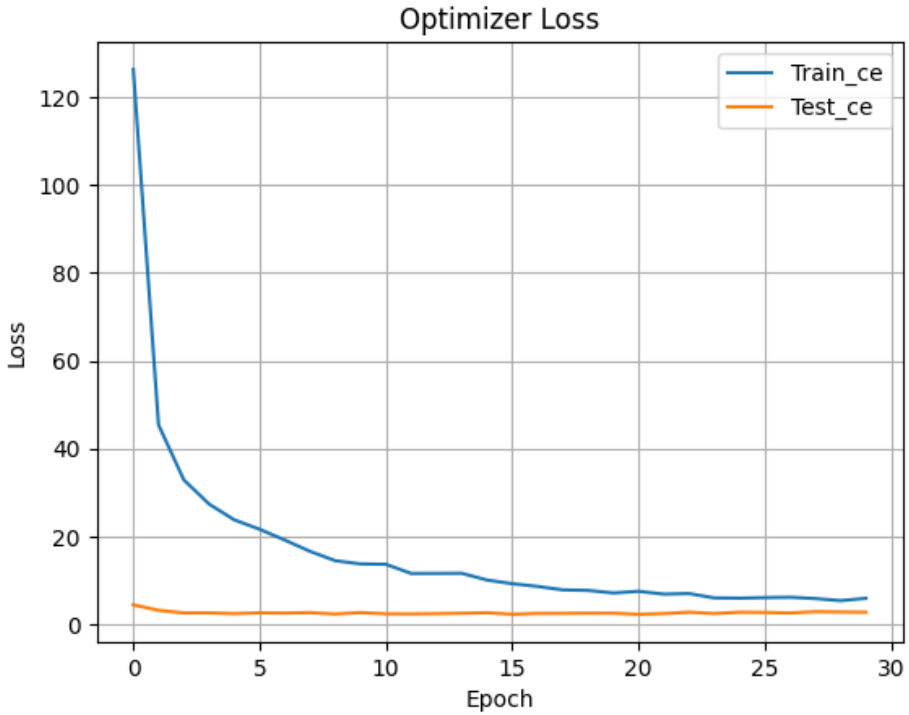
컨볼루션 층 사이에 풀링을 매번 수행한 LeNet과 다르게
컨볼루션 두 층을 연달아 사용하고 풀링 수행

드롭아웃 계층의 적용

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-2] 컨볼루션 신경망으로 MNIST 인식 (모델 구현)



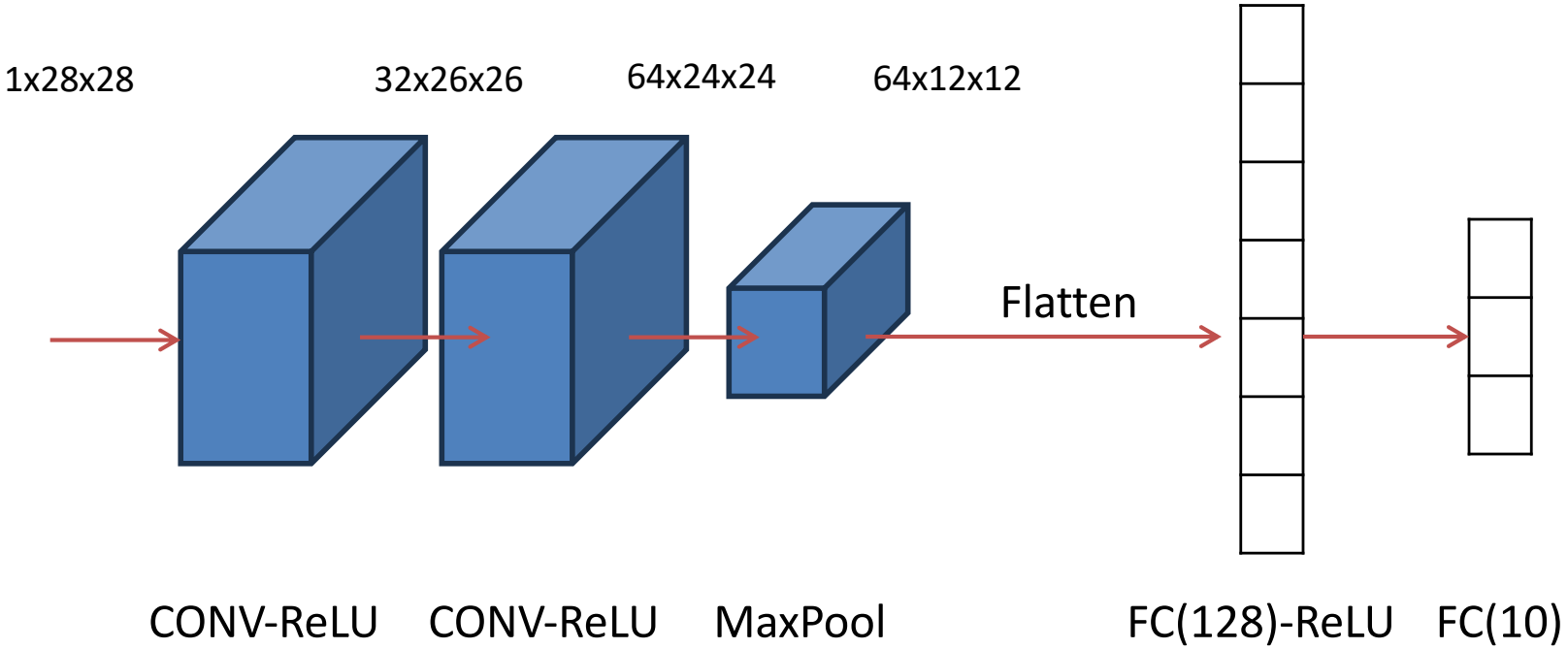


6.4 컨볼루션 신경망 프로그래밍

6.4.3 패션 데이터 인식

[프로그램 6-3] 컨볼루션 신경망으로 Fashion MNIST 인식

(C-C-P-dropout-FC-dropout-FC)



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p3-codingnote

6.4 컨볼루션 신경망 프로그래밍

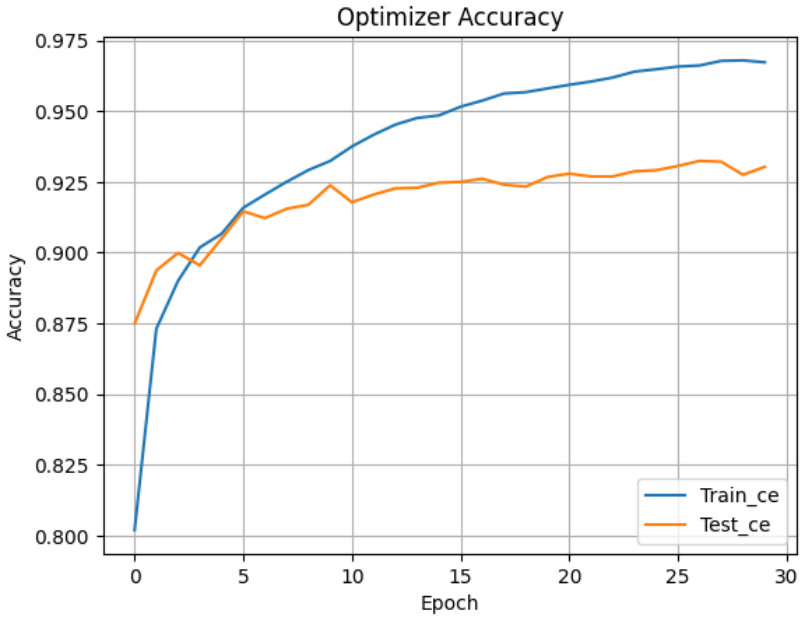
[프로그램 6-3] 컨볼루션 신경망으로 Fashion MNIST 인식

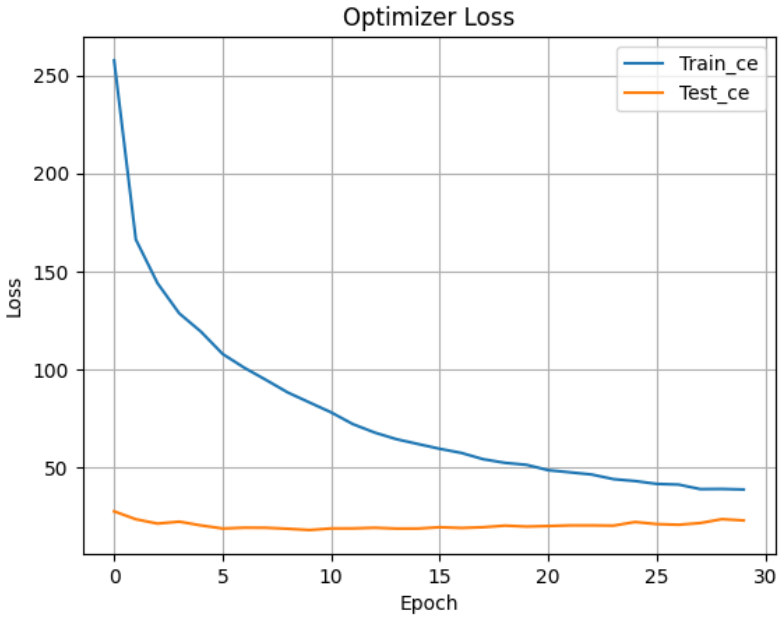
패션 관련 이미지를 다루는 Fashion MNIST에 대한 예측 실습해보기
(데이터셋 로드 부분을 제외하고 [프로그램 6-2]와 같은 코드 사용)

```
17
18      ## 데이터셋 준비 - W5P1에서 배움
19      FashionMNIST_training_data = datasets.FashionMNIST(root='data',train=True,download=True,transform=ToTensor())
20      FashionMNIST_test_data = datasets.FashionMNIST(root='data',train=False,download=True,transform=ToTensor())
21
22      ## 배치 단위 데이터셋 준비
23      train_dataloader = torch.utils.data.DataLoader(FashionMNIST_training_data, batch_size=128)
24      test_dataloader = torch.utils.data.DataLoader(FashionMNIST_test_data, batch_size=128, shuffle=False)
25
26
27      # 모델 정의
28      # model = LeNet5v2().cuda()
29      # model = CNN().cuda()
30      model = CNNv2().cuda()
31
```

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-3] 컨볼루션 신경망으로 Fashion MNIST 인식





6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-3] 컨볼루션 신경망으로 Fashion MNIST 인식

자신이 설계한 모델 확인

[4]:
```
1 print(model)
```

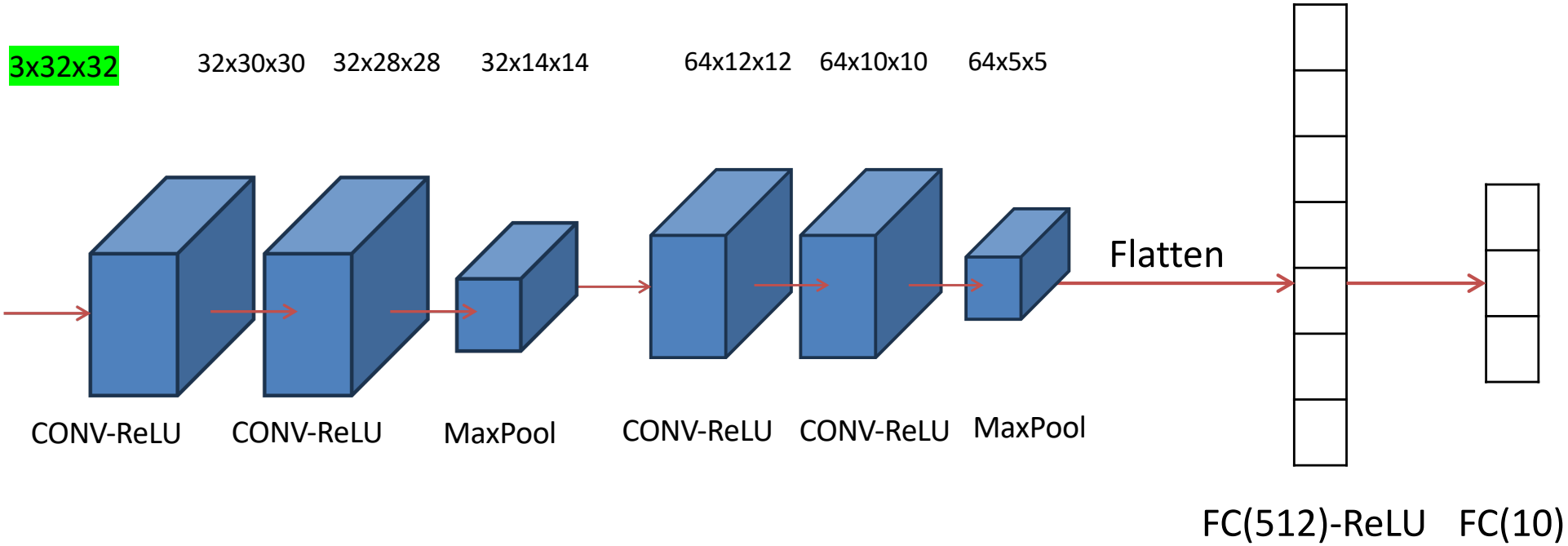
```
CNNv2(
  (input_layer): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))
    (1): ReLU()
  )
  (hidden_layer_1): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): ReLU()
  )
  (dropout_1): Dropout(p=0.25, inplace=False)
  (maxpool): MaxPool2d(kernel_size=(2, 2), stride=(2, 2), padding=0, dilation=1, ceil_mode=False)
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (hidden_layer_2): Sequential(
    (0): Linear(in_features=9216, out_features=128, bias=True)
    (1): ReLU()
  )
  (dropout_2): Dropout(p=0.5, inplace=False)
  (classifier): Linear(in_features=128, out_features=10, bias=True)
)
```

6.4 컨볼루션 신경망 프로그래밍

6.4.4 자연 영상 데이터 인식

[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 (입력 3채널) 인식

(C-C-P-dropout-C-C-P-dropout-FC-dropout-FC)



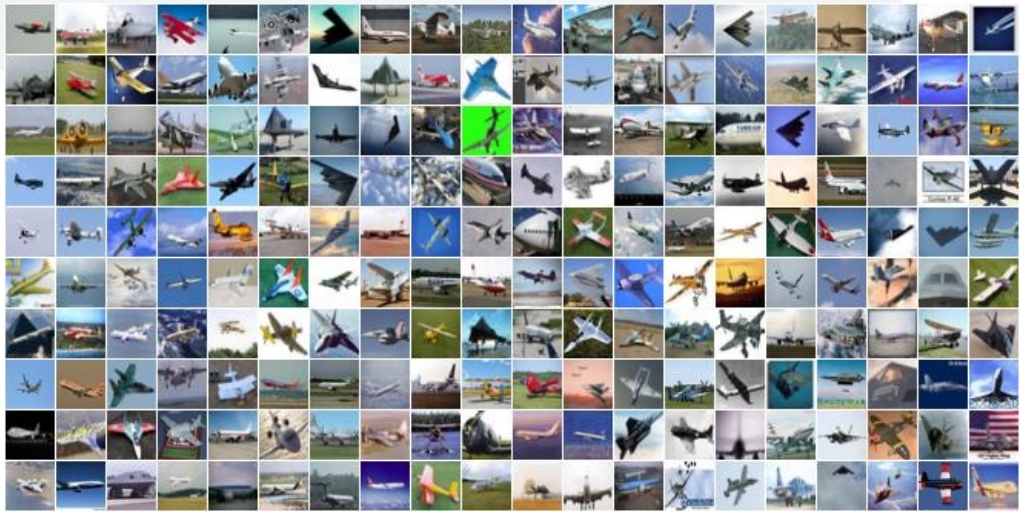
https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p4-codingnote

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 인식 (데이터 로드 변경)

CIFAR-10 데이터셋 로드하기

```
16
17
18      ## 데이터셋 준비 - W5P1에서 배움
19      CIFAR_training_data = datasets.CIFAR10(root='data',train=True,download=True,transform=ToTensor())
20      CIFAR_test_data = datasets.CIFAR10(root='data',train=False,download=True,transform=ToTensor())
21
22      ## 배치 단위 데이터셋 준비
23      train_dataloader = torch.utils.data.DataLoader(CIFAR_training_data, batch_size=128)
24      test_dataloader = torch.utils.data.DataLoader(CIFAR_test_data, batch_size=128, shuffle=False)
25
26
27      # 모델 정의
28      # model = LeNet5v2().cuda()
29      # model = CNN().cuda()
30      model = CNNv3().cuda()
```



6.4 컨볼루션 신경망 프로그래밍

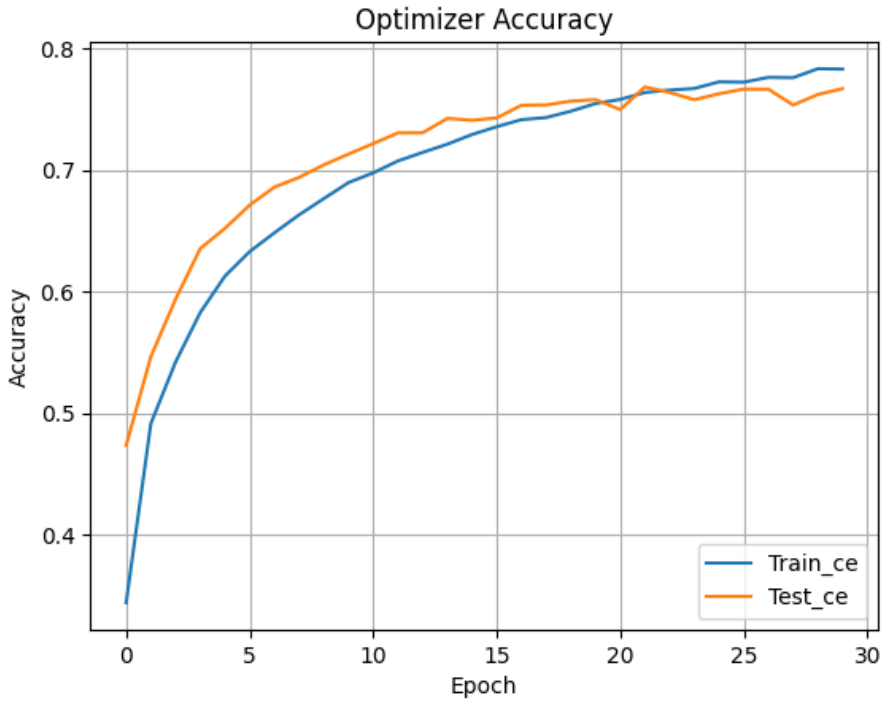
[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 인식 (모델 구현)

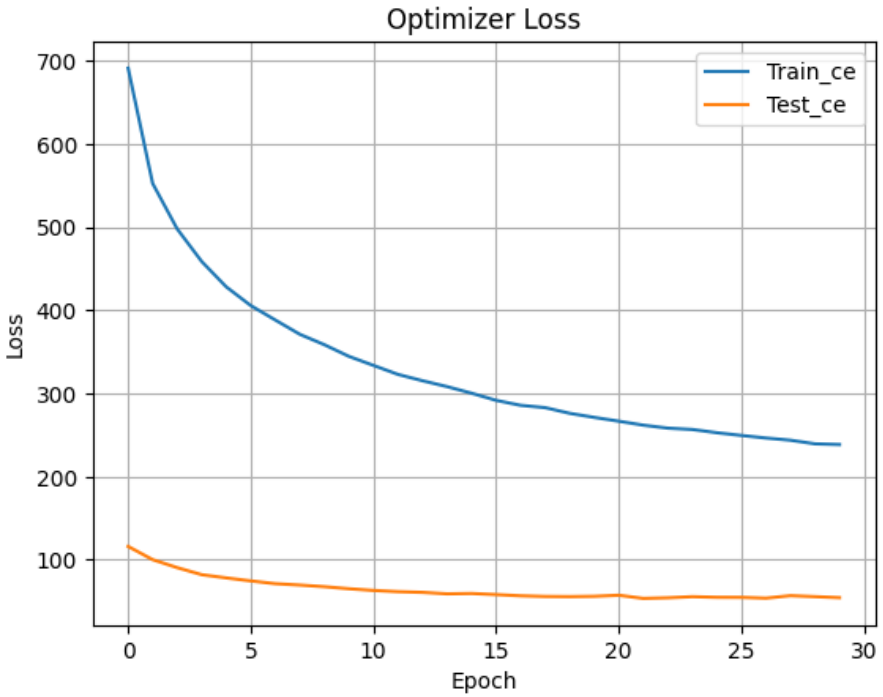
```
206 # 교재에 있는 CNNv3
207 class CNNv3(torch.nn.Module):
208     def __init__(self):
209         super(CNNv3, self).__init__()
210 
211         self.input_layer = torch.nn.Sequential(
212             torch.nn.Conv2d(in_channels=3, out_channels=32, kernel_size=(3,3), stride=1, padding=0),
213             torch.nn.ReLU(),
214         )
215         self.hidden_layer_1 = torch.nn.Sequential(
216             torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=(3, 3), stride=1, padding=0),
217             torch.nn.ReLU(),
218         )
219         self.maxpool = torch.nn.MaxPool2d(kernel_size=(2, 2))
220         self.dropout_1 = torch.nn.Dropout(p=0.25)
221         self.hidden_layer_2 = torch.nn.Sequential(
222             torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
223             torch.nn.ReLU(),
224             torch.nn.Conv2d(in_channels=64, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
225             torch.nn.ReLU(),
226         )
227         self.flatten = torch.nn.Flatten()
228         self.hidden_layer_3 = torch.nn.Sequential(
229             torch.nn.Linear(in_features=64*5*5, out_features=512),
230             torch.nn.ReLU()
231         )
232         self.dropout_2 = torch.nn.Dropout(p=0.5)
233         self.classifier = torch.nn.Linear(in_features=512, out_features=10)
234 
235     def forward(self, input):
236         """
237         input shape: (3, 32, 32)
238         output shape: (10)
239         """
240         x = self.input_layer(input)
241         x = self.hidden_layer_1(x)
242         x = self.maxpool(x)
243         x = self.dropout_1(x)
244         x = self.hidden_layer_2(x)
245         x = self.maxpool(x)
246         x = self.dropout_1(x)
247         x = self.flatten(x)
248         x = self.hidden_layer_3(x)
249         x = self.dropout_2(x)
250         x = self.classifier(x)
251         return x
252 
```

입력이 RGB 컬러 이미지 이므로 입력 채널은 3으로 설정

6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 인식





6.4 컨볼루션 신경망 프로그래밍

[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 인식

모델 구조 확인하기

[9]:

```
#print(model)

!pip install torchsummary
from torchsummary import summary
summary(model, input_size=(3, 32, 32))
```

```
Requirement already satisfied: torchsummary in /usr/local/lib/python3.11/dist-packages (1.5.1)
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 32, 30, 30]             896
              ReLU-2           [-1, 32, 30, 30]               0
            Conv2d-3           [-1, 32, 28, 28]           9,248
              ReLU-4           [-1, 32, 28, 28]               0
         MaxPool2d-5           [-1, 32, 14, 14]               0
           Dropout-6           [-1, 32, 14, 14]               0
            Conv2d-7           [-1, 64, 12, 12]          18,496
              ReLU-8           [-1, 64, 12, 12]               0
            Conv2d-9           [-1, 64, 10, 10]          36,928
             ReLU-10           [-1, 64, 10, 10]               0
        MaxPool2d-11             [-1, 64, 5, 5]               0
          Dropout-12             [-1, 64, 5, 5]               0
          Flatten-13                 [-1, 1600]               0
           Linear-14                  [-1, 512]         819,712
             ReLU-15                  [-1, 512]               0
          Dropout-16                  [-1, 512]               0
           Linear-17                   [-1, 10]           5,130
================================================================
Total params: 890,410
Trainable params: 890,410
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 1.20
Params size (MB): 3.40
Estimated Total Size (MB): 4.61
----------------------------------------------------------------
```

6.4 컨볼루션 신경망 프로그래밍

6.4.5 학습된 모델 저장과 재활용 (학습된 모델을 불러서 평가 방식 중요**)

[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 인식

```
1 torch.save(model.state_dict(), "./my_cnn_state_dict.pth")
```

학습이 완료된 모델의 state_dict를 파일로 저장

[프로그램 6-5] 학습된 모델 불러 쓰기

[프로그램 6-4]와 동일하게 모델 정의까지 진행

학습을 처음부터 진행하는 대신, 학습된 모델의 state_dict 불러오기

```
1 model.load_state_dict(torch.load("/kaggle/working/my_cnn_state_dict.pth"))
```

```
1  from torch.utils.data import DataLoader
2
3  val_dataloader = DataLoader(CIFAR_test_data, batch_size=256, shuffle=False, drop_last=False)
4
5  model.eval()
6  acc = 0
7  for x, y in val_dataloader:
8      x = x.cuda()
9      y = y.cuda()
10
11     with torch.no_grad():
12         pred = model(x)
13     acc += torch.sum(torch.argmax(pred, dim=1) == y).item()
14
15 acc /= len(val_dataloader.dataset)
16 print("테스트 정확률:", acc*100)
```

불러온 모델로 평가 진행 시 동일한 결과를 얻을 수 있음

Pytorch 를 이용한 CIFAR10 CNN 실습 과제

- 제출 방식 : 이메일로 보고서 제출
  - 마감 : 10월 14일 (수) 23시 59분
  - 이메일 주소 : admin@rcv.sejong.ac.kr
  - 이메일 제목 : [2026-2] [인공지능] 6주차 과제 제출 (학번_이름)
  - 보고서 분량 제한 없음

- [문제1] CIFAR-10
  - LeNet-5, CNN (6-4) 모델의 성능을 비교하시오
  - 가장 적절한 옵티마이저와 파라미터 값을 찾아 현재 설정된 베이스라인을 해결하시오.
  - https://www.kaggle.com/t/bc2571b6881b462c87a7630925688724
  - 참고해서 리더보드 만들기

6.5 컨볼루션 신경망의 시각화

- 시각화는 딥러닝의 동작을 잘 이해하는데 매우 효과적임
- 앞 절에서는 딥러닝 학습 알고리즘의 수렴 특성을 관찰할 수 있도록 〈학습 곡선을 시각화〉 했음
- 이번 절에서는 1) 컨볼루션층에 있는 〈커널을 시각화〉 하고, 2) 컨볼루션층이나 풀링층이 추출해주는 〈특징 맵을 시각화〉 할 예정임

6.5 컨볼루션 신경망의 시각화

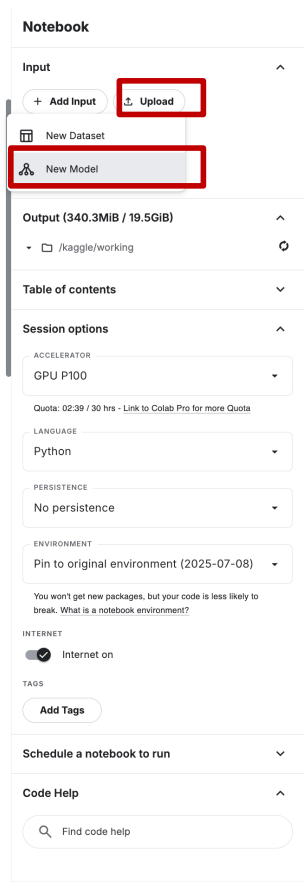
학습목표1:
학습했던 모델(가중치)을 파일로 저장하고, 다시 로드 하여 사용할 수 있다.

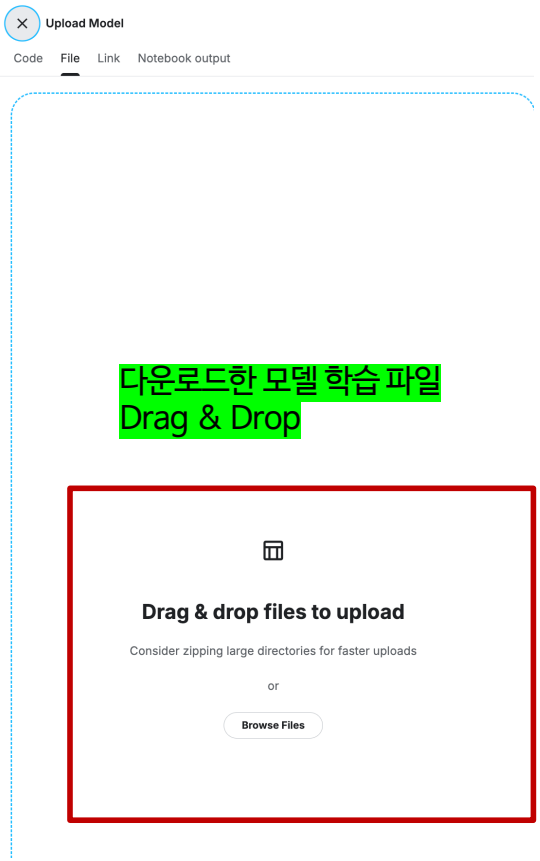
학습목표2:
모델 가중치와 특징맵을 시각화 할 수 있다.

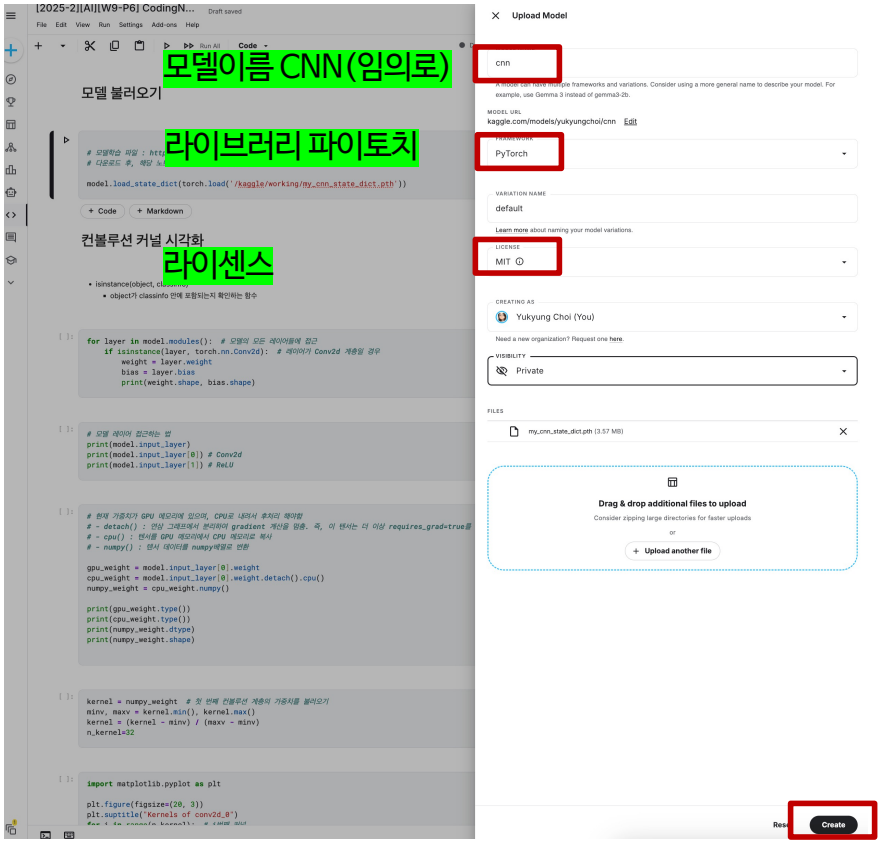
6.5.1 컨볼루션 커널의 시각화 프로그래밍

[프로그램 6-6] 컨볼루션 신경망으로 CIFAR-10 인식

노트북에 학습된 AI모델 불러오기







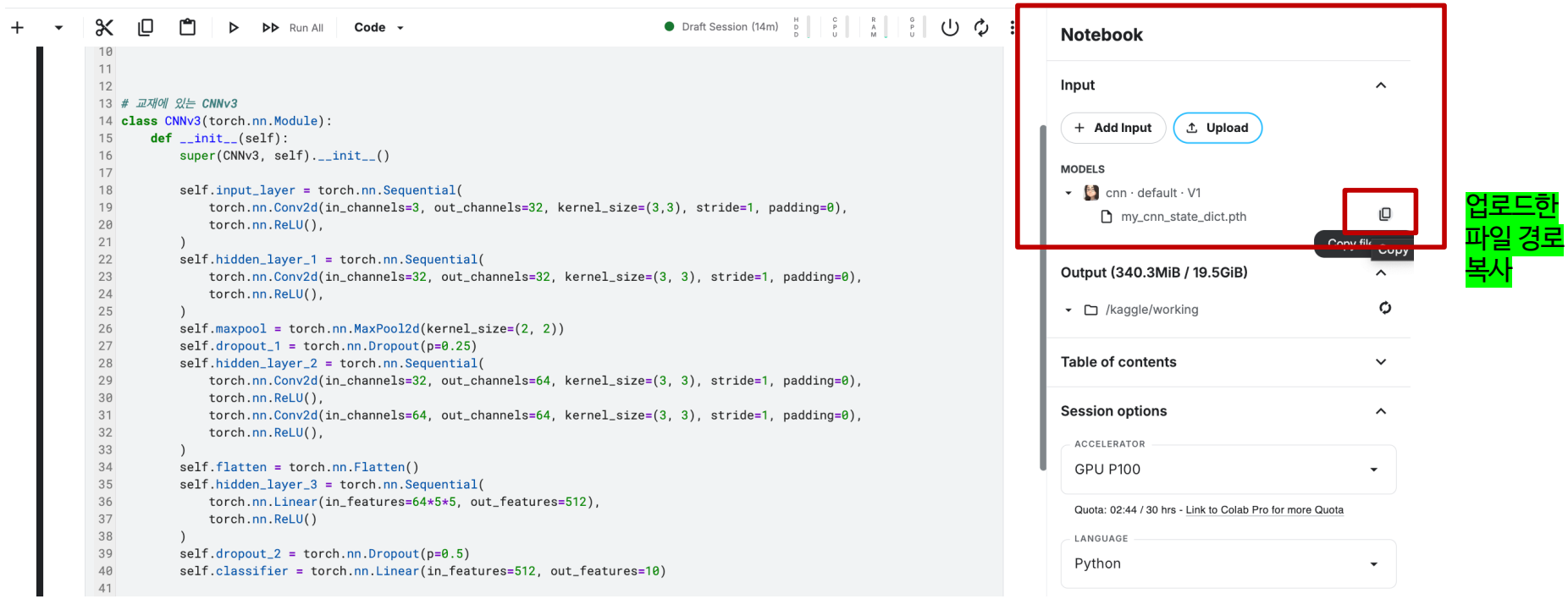
https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p6-codingnote

6.5 컨볼루션 신경망의 시각화

6.5.1 컨볼루션 커널의 시각화 프로그래밍

[프로그램 6-6] 컨볼루션 신경망으로 CIFAR-10 인식

노트북에 학습된 AI모델 불러오기




```python
# 교재에 있는 CNNv3
class CNNv3(torch.nn.Module):
    def __init__(self):
        super(CNNv3, self).__init__()

        self.input_layer = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=3, out_channels=32, kernel_size=(3,3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.hidden_layer_1 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.maxpool = torch.nn.MaxPool2d(kernel_size=(2, 2))
        self.dropout_1 = torch.nn.Dropout(p=0.25)
        self.hidden_layer_2 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=64, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.flatten = torch.nn.Flatten()
        self.hidden_layer_3 = torch.nn.Sequential(
            torch.nn.Linear(in_features=64*5*5, out_features=512),
            torch.nn.ReLU()
        )
        self.dropout_2 = torch.nn.Dropout(p=0.5)
        self.classifier = torch.nn.Linear(in_features=512, out_features=10)
```

업로드한 파일 경로 복사

6.5 컨볼루션 신경망의 시각화

6.5.1 컨볼루션 커널의 시각화 프로그래밍

[프로그램 6-6] 컨볼루션 신경망으로 CIFAR-10 인식

[프로그램 6-5]와 동일하게 모델 정의 후 학습된 가중치 불러오기

```
# 교재에 있는 CNNv3
class CNNv3(torch.nn.Module):
    def __init__(self):
        super(CNNv3, self).__init__()

        self.input_layer = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=3, out_channels=32, kernel_size=(3,3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.hidden_layer_1 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.maxpool = torch.nn.MaxPool2d(kernel_size=(2, 2))
        self.dropout_1 = torch.nn.Dropout(p=0.25)
        self.hidden_layer_2 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=64, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.flatten = torch.nn.Flatten()
        self.hidden_layer_3 = torch.nn.Sequential(
            torch.nn.Linear(in_features=64*5*5, out_features=512),
            torch.nn.ReLU()
        )
        self.dropout_2 = torch.nn.Dropout(p=0.5)
        self.classifier = torch.nn.Linear(in_features=512, out_features=10)

    def forward(self, input):
        """
        input shape: (3, 32, 32)
        output shape: (10)
        """
        x = self.input_layer(input)
        x = self.hidden_layer_1(x)
        x = self.maxpool(x)
        x = self.dropout_1(x)
        x = self.hidden_layer_2(x)
        x = self.maxpool(x)
        x = self.dropout_1(x)
        x = self.flatten(x)
        x = self.hidden_layer_3(x)
        x = self.dropout_2(x)
        x = self.classifier(x)
        return x
```

+ Code + Markdown

```
# 모델학습 파일 : https://github.com/sejongresearch/2025.AI/tree/main/Code
# 다운로드 후, 해당 노트북에 업로드 하여 사용할 것

model = CNNv3().cuda()
model.load_state_dict(torch.load('/kaggle/input/cnn/pytorch/default/1/my_cnn_state_dict.pth'))
```

업로드한 파일 경로 붙여넣기

6.5 컨볼루션 신경망의 시각화

6.5.1 컨볼루션 커널의 시각화 프로그래밍

[프로그램 6-6] 컨볼루션 신경망으로 CIFAR-10 인식

컨볼루션 계층의 가중치와 편향 값의 형태(shape) 살펴보기

```
for layer in model.modules():  # 모델의 모든 레이어들에 접근
    if isinstance(layer, nn.Conv2d):  # 레이어가 Conv2d 계층일 경우
        weight = layer.weight
        bias = layer.bias
        print(weight.shape, bias.shape)
```

```
torch.Size([32, 3, 3, 3]) torch.Size([32])
torch.Size([32, 32, 3, 3]) torch.Size([32])
torch.Size([64, 32, 3, 3]) torch.Size([64])
torch.Size([64, 64, 3, 3]) torch.Size([64])
```

```
# 교재에 있는 CNNv3
class CNNv3(torch.nn.Module):
    def __init__(self):
        super(CNNv3, self).__init__()

        self.input_layer = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=3, out_channels=32, kernel_size=(3,3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.hidden_layer_1 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.maxpool = torch.nn.MaxPool2d(kernel_size=(2, 2))
        self.dropout_1 = torch.nn.Dropout(p=0.25)
        self.hidden_layer_2 = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=64, out_channels=64, kernel_size=(3, 3), stride=1, padding=0),
            torch.nn.ReLU(),
        )
        self.flatten = torch.nn.Flatten()
        self.hidden_layer_3 = torch.nn.Sequential(
            torch.nn.Linear(in_features=64*5*5, out_features=512),
            torch.nn.ReLU()
        )
        self.dropout_2 = torch.nn.Dropout(p=0.5)
        self.classifier = torch.nn.Linear(in_features=512, out_features=10)
```

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p6-codingnote

6.5 컨볼루션 신경망의 시각화

6.5.1 컨볼루션 커널의 시각화 프로그래밍

[프로그램 6-6] 컨볼루션 신경망으로 CIFAR-10 인식

시각화 하고자 하는 첫번째 계층의 가중치 가져오고 값을 스케일링 해주기

[49]:
```
# 모델 레이어 접근하는 법
print(model.input_layer)
print(model.input_layer[0]) # Conv2d
print(model.input_layer[1]) # ReLU
```

```
Sequential(
  (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1))
  (1): ReLU()
)
Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1))
ReLU()
```

[50]:
```
# 현재 가중치는 GPU메모리에 있음. CPU로 내려서 후처리 해야함

gpu_weight = model.input_layer[0].weight
cpu_weight = model.input_layer[0].weight.cpu().detach()
numpy_weight = cpu_weight.numpy()

print(gpu_weight.type())
print(cpu_weight.type())
print(numpy_weight.dtype)
print(numpy_weight.shape)

```

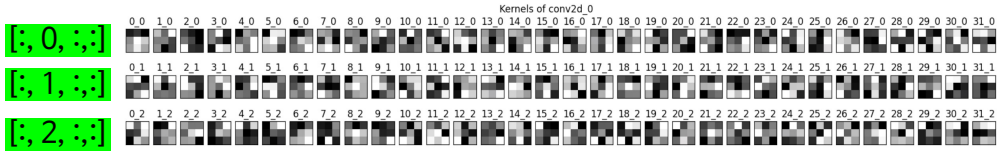
```
torch.cuda.FloatTensor
torch.FloatTensor
float32
(32, 3, 3, 3)
```

[51]:
```
kernel = numpy_weight  # 첫 번째 컨볼루션 계층의 가중치를 불러오기
minv, maxv = kernel.min(), kernel.max()
kernel = (kernel - minv) / (maxv - minv)
n_kernel=32
```

+ Code + Markdown

[52]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 3))
plt.suptitle("Kernels of conv2d_0")
for i in range(n_kernel):  # i번째 커널
    f = kernel[i, :, :, :]
    for j in range(3):  # j번째 채널
        plt.subplot(3, n_kernel, j * n_kernel + i + 1)
        plt.imshow(f[:, :, j], cmap='gray')
        plt.xticks([]); plt.yticks([])
        plt.title(str(i)+"_"+str(j))
plt.show()
```



6.5 컨볼루션 신경망의 시각화

6.5.2 특징 맵의 시각화 프로그래밍

[프로그램 6-6] 컨볼루션 신경망으로 CIFAR-10 인식

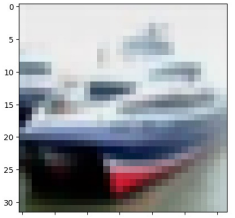
CIFAR10의 이미지를 입력하였을 때, 생성되는 특징 맵의 시각화

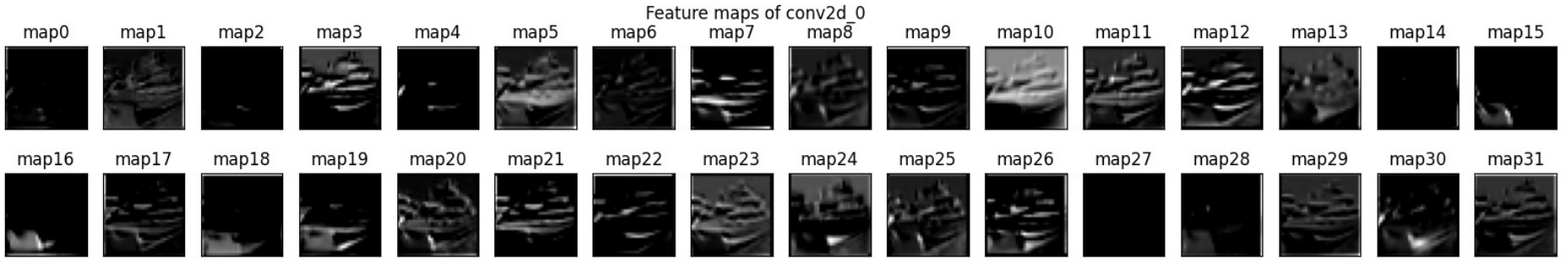
```
from torch.utils.data import DataLoader
val_dataloader = DataLoader(CIFAR_test_data, batch_size=256, shuffle=False, drop_last=False)

layer = model.input_layer  # 모델에서 첫 번째 합성곱 계층만 떼어냄
partial_x = next(iter(val_dataloader))[0]  # 테스트 데이터의 첫 배치만 사용

with torch.no_grad():
    feature_map = layer(partial_x)
```

```
plt.figure(figsize=(20, 3))
plt.suptitle("Feature maps of conv2d_0")
for i in range(32):
    plt.subplot(2, 16, i+1)
    plt.imshow(fm[i, :, :], cmap="gray")
    plt.xticks([]); plt.yticks([])
    plt.title("map"+str(i))
```





6.6 딥러닝의 규제

- 딥러닝 학습 알고리즘이 최적화해야 하는 매개변수는 방대함
  - 예를들어 [프로그램 6-4]을 실행한 결과는 매개변수가 89만 개 이상임
  - 컨볼루션층 4개와 완전연결층 2개로 구성된 깊지 않은 신경망이지만 방대한 수의 매개변수가 있음

모델 구조 확인하기

[9]:
```
#print(model)

!pip install torchsummary
from torchsummary import summary
summary(model, input_size=(3, 32, 32))
```

```
Requirement already satisfied: torchsummary in /usr/local/lib/python3.11/dist-packages (1.5.1)
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 32, 30, 30]             896
              ReLU-2           [-1, 32, 30, 30]               0
            Conv2d-3           [-1, 32, 28, 28]           9,248
              ReLU-4           [-1, 32, 28, 28]               0
         MaxPool2d-5           [-1, 32, 14, 14]               0
           Dropout-6           [-1, 32, 14, 14]               0
            Conv2d-7           [-1, 64, 12, 12]          18,496
              ReLU-8           [-1, 64, 12, 12]               0
            Conv2d-9           [-1, 64, 10, 10]          36,928
             ReLU-10           [-1, 64, 10, 10]               0
        MaxPool2d-11             [-1, 64, 5, 5]               0
          Dropout-12             [-1, 64, 5, 5]               0
          Flatten-13                 [-1, 1600]               0
           Linear-14                  [-1, 512]         819,712
             ReLU-15                  [-1, 512]               0
          Dropout-16                  [-1, 512]               0
           Linear-17                   [-1, 10]           5,130
================================================================
Total params: 890,410
Trainable params: 890,410
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 1.20
Params size (MB): 3.40
Estimated Total Size (MB): 4.61
----------------------------------------------------------------
```

6.6 딥러닝의 규제

- 딥러닝 학습 알고리즘이 최적화해야 하는 매개변수는 방대함
  - 예를들어 [프로그램 6-4]을 실행한 결과는 매개변수가 89만 개 이상임
  - 컨볼루션층 4개와 완전연결층 2개로 구성된 깊지 않은 신경망이지만 방대한 수의 매개변수가 있음

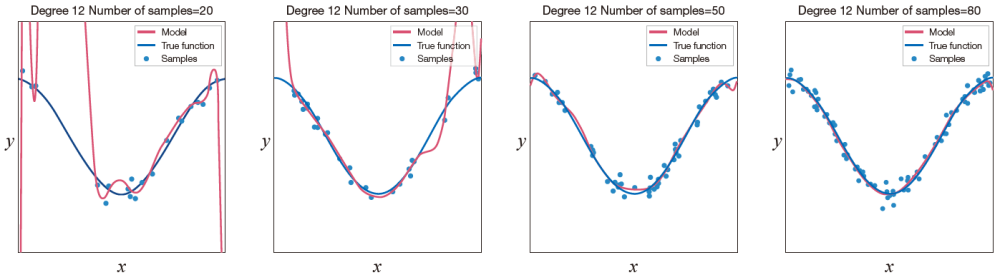
- 매겨변수가 많으면 학습 알고리즘이 과잉 적합overfitting 에 빠질 가능성에 있음.
- 그러나 딥러닝에서는 과잉 적합을 방지하려 굳이 신경망을 작게 만들지 않고, 다양한 규제 기법regulation 을 적용해 과잉 적합을 방지하는 전략을 사용함

- 이번 장에서는 자주 사용하는 규제 기법인 데이터 증대data augmentation, 드롭아웃 dropout, 가중치 감쇠weight decay 에 대해 설명하겠음

6.6 딥러닝의 규제

6.6.1 데이터 증대 (Data Augmentation)

- 딥러닝에서 과잉 적합을 방지하는 가장 확실한 방법은 **큰 훈련 집합을 사용**하는 것
- [그림 5-14]는 데이터가 커지면 자연스럽게 과잉 적합 문제가 해소되는 현상을 예시함



(a) 훈련 집합 크기: 20 (b) 훈련 집합 크기: 30 (c) 훈련 집합 크기: 50 (d) 훈련 집합 크기: 80

그림 5-14 데이터 양을 늘려 과잉 적합을 해소

- 인공지능 제품을 만드는 프로젝트를 수행할 때 〈**데이터를 추가로 수집**〉할 수 있는 상황이라면, 추가 수집이 현명한 해결법임. 하지만 대부분의 상황에서 데이터를 늘리는 일에는 **많은 시간과 비용이 따름**
- 딥러닝은 〈**주어진 데이터를 적절하게 변형**〉해 인위적으로 증대하는 전략을 쓰는데 **이 규제 기법을 데이터 증대**라 부름
- 영상의 경우 **1) 상하좌우 방향으로 조금 이동하거나, 2) 약간 회전하거나, 3) 좌우 또는 상하로 반전하거나, 4) 명암을 조절**하는 등의 **여러가지 변환을 조합해 영상의 개수를 늘림**

6.6 딥러닝의 규제

6.6.1 데이터 증대 (Data Augmentation)

- [프로그램6-7]은 증대한 영상을 단순히 보여주는 프로그램임
  - CIFAR에서 일부 데이터를 가져오고 다양한 영상 증대 기법 적용해보기

[프로그램 6-7] torchvision.transforms을 이용한 데이터 증대

```
from torch.utils.data import DataLoader
dataloader = DataLoader(CIFAR_training_data, batch_size=256, shuffle=False, drop_last=False)
cifar_sample, cifar_sample_label = next(iter(dataloader))  # CIFAR 데이터의 일부를 가져옴
```

```
import matplotlib.pyplot as plt
class_names = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(16, 2))
plt.suptitle("First 12 images in the train set")
for i in range(12):
    plt.subplot(1, 12, i+1)
    plt.imshow(cifar_sample[i].permute(1, 2, 0).numpy())  # 시각화를 위해 torch.tensor를 numpy.array로 변환
    plt.xticks([]); plt.yticks([])
    plt.title(class_names[int(cifar_sample_label[i])])
```

원본 CIFAR 데이터셋 시각화 해보기

***** 영상 증대를 위한 transform 설계**

[주의] 데이터 증대는 랜덤하게 적용되므로, 랜덤 시드를 고정하지 않으면 실험 결과가 변화함에 유의할 것!

```
from torchvision.transforms import Compose, RandomAffine, RandomVerticalFlip, RandomHorizontalFlip, RandomCrop, Resize

transform = Compose([
    RandomAffine(degrees=(-20, 20), translate=(0.2, 0.2)),  # 주어진 범위 내에서 랜덤하게 회전 혹은 이동을 적용
    RandomCrop((24, 24)),  # 랜덤한 영역을 크롭
    Resize((32, 32)),  # 크롭한 이미지의 크기를 늘려 확대
    RandomHorizontalFlip(p=0.5),  # 랜덤하게 수평 회전
    RandomVerticalFlip(p=0.5)  # 랜덤하게 수직 회전
])
```

3행 : Compose() 내부의 변환들을 차례로 수행
4행 : 랜덤하게 회전과 이동을 수행 (아핀 변환)
5-6행 : 이미지의 랜덤한 영역을 확대
7-8행 : 50% 확률로 수평/수직 회전 적용

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p7-codingnote

6.6 딥러닝의 규제

6.6.1 데이터 증대 (Data Augmentation)

[프로그램 6-7] torchvision.transforms을 이용한 데이터 증대

```
plt.figure(figsize=(16, 3))
plt.suptitle("Data Augmentation Result")
for i in range(6):
    plt.subplot(1, 6, i+1)
    plt.imshow(transform(cifar_sample[i]).permute(1, 2, 0).numpy())  # 시각화를 위해 torch.tensor를 numpy.array로 변환
    plt.xticks([]); plt.yticks([])
    plt.title(class_names[int(cifar_sample_label[i])])
```

transform을 이용해 변형된 데이터셋 시각화



6.6 딥러닝의 규제

6.6.1 데이터 증대 (Data Augmentation)

[프로그램 6-7] torchvision.transforms을 이용한 데이터 증대

```
# 뒤에 6개 영상에 대한 변형 (6~11)

plt.figure(figsize=(16, 3))
plt.suptitle("Data Augmentation Result 2")
for i in range(6):
    plt.subplot(1, 6, i+1)
    plt.imshow(transform(cifar_sample[6+i]).permute(1, 2, 0).numpy())  # 시각화를 위해 torch.tensor를 numpy.array로 변환
    plt.xticks([]); plt.yticks([])
    plt.title(class_names[int(cifar_sample_label[6+i])])
```

transform을 이용해 변형된 데이터셋 시각화



6.6 딥러닝의 규제

6.6.1 데이터 증대 (Data Augmentation)

- 증대한 영상을 신경망 학습에 실제로 사용해보고 성능 향상이 이루어지는지 확인할 예정임
- 증대한 영상을 학습에 활용하는 방식에는 오프라인 방식과 온라인 방식의 두 종류가 있음

- 오프라인 방식은 적절한 양의 데이터를 생성해 하드디스크에 저장한 후 학습할 때 읽어 훈련 집합으로 활용하는 것
- 온라인 방식은 학습을 진행하면서 실시간으로 샘플을 생성하는 것

- **딥러닝에서는 온라인 방식을 주로 활용함**

6.6 딥러닝의 규제

6.6.1 데이터 증대 (Data Augmentation)

- [프로그램 6-4]와 동일하게 30 에포크 학습 진행 후 성능 비교해보기

[프로그램 6-8] CNN으로 CIFAR-10 인식 : 데이터 증대 적용

```
1  import numpy as np
2  import torch
3  from torchvision import datasets
4  from torchvision.transforms import Compose, RandomAffine, RandomHorizontalFlip, ToTensor
5
6  # 데이터 증대 방식 정의
7  transform = Compose([
8      ToTensor(),
9      RandomAffine(0, translate=(0.1, 0.1)),  # 주어진 범위 내에서 랜덤하게 회전 혹은 이동을 적용
10     RandomHorizontalFlip(p=0.5),  # 랜덤하게 수평 회전
11 ])
12
13 # 데이터 증대를 학습 데이터에 적용
14 # CIFAR 읽고 텐서 구조 확인 (학습 데이터)
15 CIFAR_training_data = datasets.CIFAR10(
16     root="data",
17     train=True,
18     download=True,
19     transform=transform
20 )
21 # CIFAR 읽고 텐서 구조 확인 (평가 데이터)
22 CIFAR_test_data = datasets.CIFAR10(
23     root="data",
24     train=False,
25     download=True,
26     transform=ToTensor()  # 테스트 데이터에는 증대를 적용하면 안됩니다!!!
27 )
```

7-11행 : 사용하고자 하는 데이터 증대 기법들로 transform 구성

19행 : 데이터셋 선언 시, 앞서 선언한 transform 적용

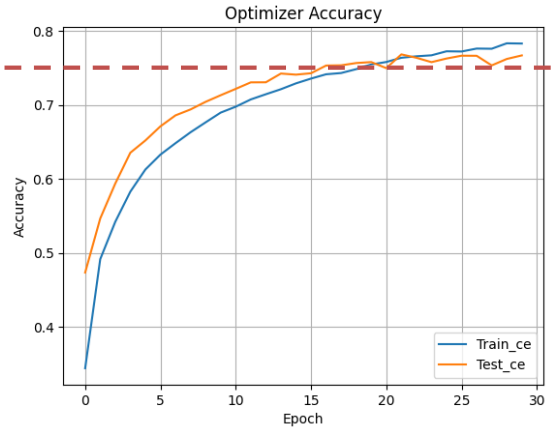
26행 : 테스트 데이터셋에는 데이터 증대를 위한 변형을 적용해선 안됨

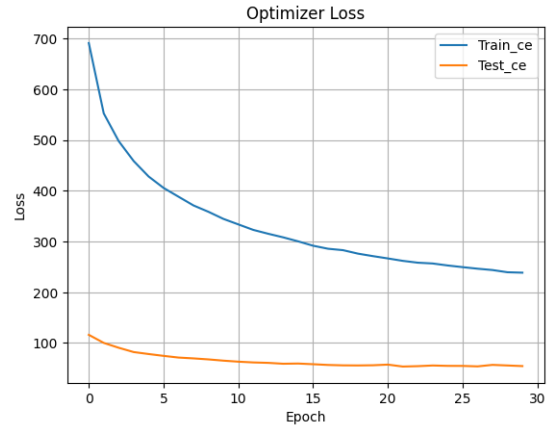
https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p8-codingnote/

6.6 딥러닝의 규제

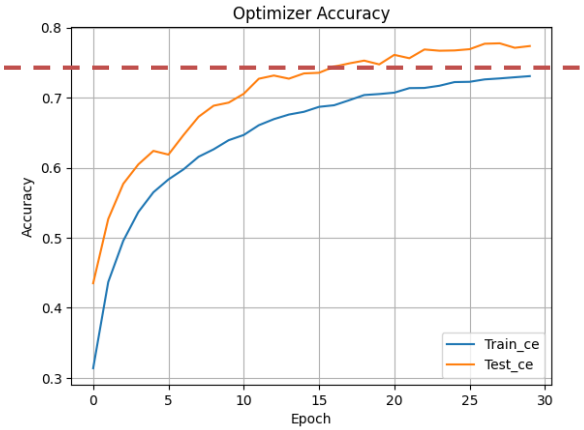
[프로그램 6-4] 컨볼루션 신경망으로 CIFAR-10 인식

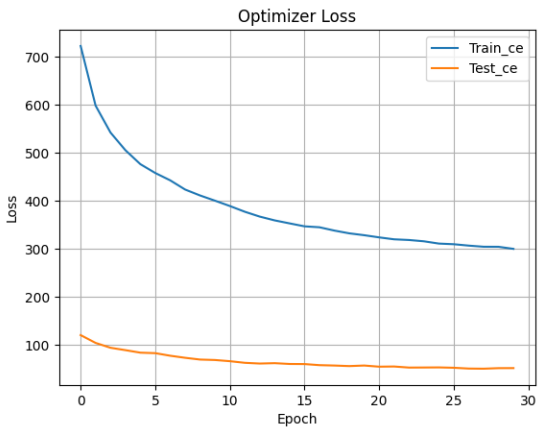
VS **[프로그램 6-8] 컨볼루션 신경망으로 CIFAR-10 인식 with 데이터 증대**





[프로그램 6-4] 결과



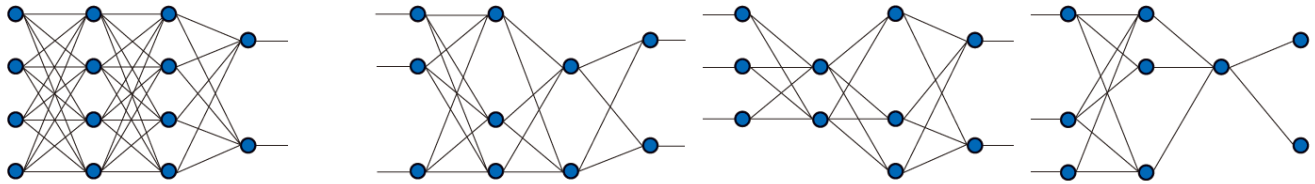


[프로그램 6-8] 학습데이터 증대 결과

6.6 딥러닝의 규제

6.6.2 드롭아웃(Dropout)

- 과대 적합 (Overfitting)을 해소하는 데 매우 효과적이라고 알려져 있음
- 드롭아웃은 일정 비율의 가중치를 임의로 선택해 불능으로 만들고 학습하는 단순한 아이디어임
- [그림 6-15]는 드롭아웃의 원리를 설명함. [그림 6-15(a)]는 원래 신경망 구조이고, [그림 6-15(b)]는 드롭아웃된 3개의 신경망임



(a) 원래 신경망(4-4-4-2 구조) (b) 드롭아웃된 3개의 신경망 예시

그림 6-15 드롭아웃의 원리

- 드롭아웃은 일정한 비율의 에지를 불능으로 만들어 학습에 참여하지 않게 함
- 구체적으로는 에지의 값(가중치)을 0으로 설정해 계산에 영향을 미치지 못하게 함
- 드롭아웃 비율은 불능으로 만들 에지의 비율에 해당하는데, 예를 들어 0.25로 설정하면 전체 가중치의 25%를 불능으로 만듦
- 불능이 될 에지는 샘플마다 독립적으로 정하는데, 난수를 이용해 랜덤하게 선택함

6.6 딥러닝의 규제

6.6.2 드롭아웃(Dropout)

[프로그램 6-9] 드롭아웃의 성능 향상 효과 측정

```
3  class CNN(nn.Module):
4      def __init__(self, dropout_rate):
5          super(CNN, self).__init__()
6
7          self.input_layer = nn.Sequential(
8              nn.Conv2d(in_channels=3, out_channels=32, kernel_size=(3,3), stride=1, padding=1),
9              nn.ReLU(),
10         )
11         self.hidden_layer_1 = nn.Sequential(
12             nn.Conv2d(in_channels=32, out_channels=32, kernel_size=(3, 3), stride=1, padding=1),
13             nn.ReLU(),
14         )
15         self.maxpool = nn.MaxPool2d(kernel_size=(2, 2))
16         self.dropout_1 = nn.Dropout(p=dropout_rate[0])
17         self.hidden_layer_2 = nn.Sequential(
18             nn.Conv2d(in_channels=32, out_channels=64, kernel_size=(3, 3), stride=1, padding=1),
19             nn.ReLU(),
20             nn.Conv2d(in_channels=64, out_channels=64, kernel_size=(3, 3), stride=1, padding=1),
21             nn.ReLU(),
22         )
23         self.dropout_2 = nn.Dropout(p=dropout_rate[1])
24         self.flatten = nn.Flatten()
25         self.hidden_layer_3 = nn.Sequential(
26             nn.Linear(in_features=64*8*8, out_features=512),
27             nn.ReLU()
28         )
29         self.dropout_3 = nn.Dropout(p=dropout_rate[2])
30         self.classifier = nn.Linear(in_features=512, out_features=10)
```

4행 모델 선언 시, dropout_rate를 입력 받음

16 행 : 입력 받은 dropout_rate를 기반으로 모델 구성

23 행 : 입력 받은 dropout_rate를 기반으로 모델 구성

29행 : 입력 받은 dropout_rate를 기반으로 모델 구성

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p9-codingnote

6.6 딥러닝의 규제

6.6.2 드롭아웃(Dropout)

[프로그램 6-9] 드롭아웃의 성능 향상 효과 측정

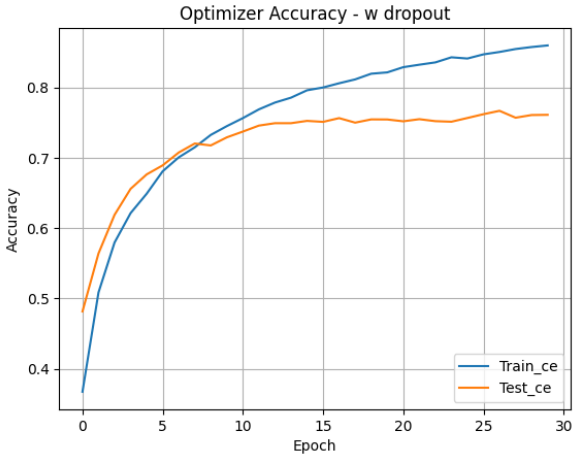
forward 함수 정의

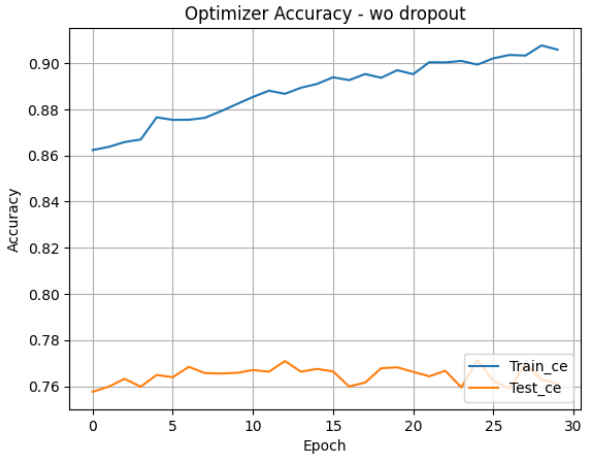
```
32      def forward(self, input):
33          """
34          input shape: (3, 32, 32)
35          output shape: (10)
36          """
37          x = self.input_layer(input)
38          x = self.hidden_layer_1(x)
39          x = self.maxpool(x)
40          x = self.dropout_1(x)
41          x = self.hidden_layer_2(x)
42          x = self.maxpool(x)
43          x = self.dropout_2(x)
44          x = self.flatten(x)
45          x = self.hidden_layer_3(x)
46          x = self.dropout_3(x)
47          x = self.classifier(x)
48          return x
```

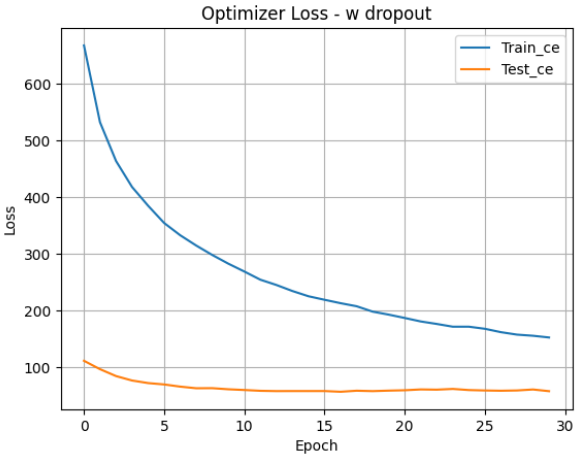
6.6 딥러닝의 규제

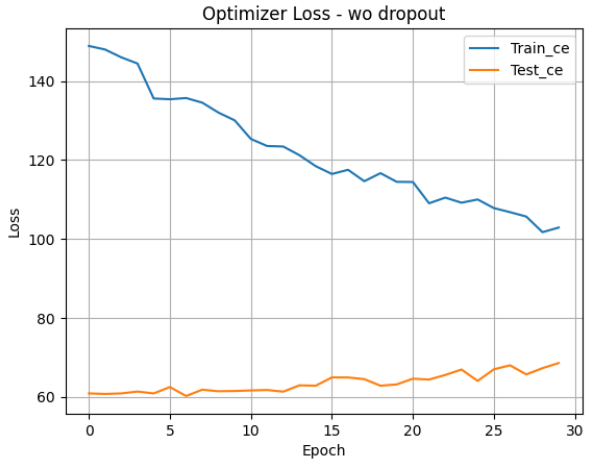
6.6.2 드롭아웃(Dropout)

[프로그램 6-9] 드롭아웃의 성능 향상 효과 측정









안정적인 학습

불안정한 학습과 과적합 증상

6.6 딥러닝의 규제

6.6.3 가중치 감쇠

- 큰 가중치 값이 과잉 적합을 일으키는 주요 원인에 해당함
  - 예를들어, [그림 5-12]의 (맨 오른쪽) 12차 다항식 모델로 학습한 결과는

    $y=1005.7x^{12}-27774.4x^{11}+\cdots$ 처럼 계수 즉 가중치 값이 수천~수만을 넘음

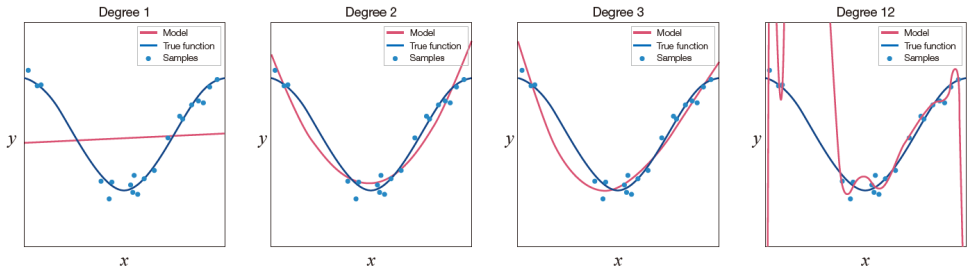


그림 5-12 과소 적합과 과잉 적합

  - 따라서 성능을 유지한 채로 가중치의 크기를 낮출 수 있다면 과잉 적합을 어느 정도 막을 수 있음

- 가중치 감쇠weight decay는 모델의 오류를 최소화하면서 동시에 가중치 값을 작게 유지하는 규제 기법임

6.6 딥러닝의 규제

U: 신경망 가중치
Y : 정답
O : 예측 값

6.6.3 가중치 감쇠 (weight decay)

- 식(6.4)는 깊은 다층 퍼셉트론의 손실 함수를 다시 쓴 것
  - 표준 손실 함수(평균제곱오차 MSE)

$$J\left(\mathbf{U}^1,\mathbf{U}^2,\cdots,\mathbf{U}^L\right)=\frac{1}{|M|}\sum_{\mathbf{x}\in M}\|\mathbf{y}-\mathbf{o}\|^2 \qquad (6.4)$$

- 식(6.5)는 L1 가중치 감쇠가 사용된 표준 손실 함수
  - 절댓값을 사용하는 규제 기법
  - L1을 라쏘 정규화 lasso regularization 라 부름 (L1 정규화)
  - λ 는 규제 항을 얼마나 중요하게 취급할 것인지 결정

$$J\left(\mathbf{U}^1,\mathbf{U}^2,\cdots,\mathbf{U}^L\right)=\frac{1}{|M|}\sum_{\mathbf{x}\in M}\|\mathbf{y}-\mathbf{o}\|^2+\lambda|\mathbf{U}|$$
$$=\frac{1}{|M|}\sum_{\mathbf{x}\in M}\|\mathbf{y}-\mathbf{o}\|^2+\lambda\left(\left|u_{10}^1\right|+\left|u_{11}^1\right|+\cdots++\cdots+\left|u_{n_Ln_{L-1}}^L\right|\right) \qquad (6.5)$$

- 식(6.6)은 L2 가중치 감쇠가 사용하는 표준 손실 함수
  - 제곱을 사용하는 기법
  - L2를 리지 정규화 ridge regularization 라 부름 (L2 정규화)
  - λ 는 규제 항을 얼마나 중요하게 취급할 것인지 결정
  - 파이토치에서 제공하는 최적화 방법론 대부분 L2 가중치 감쇠 사용
  - ➔ SGD, Adam, RMSprop, Adagrad 등에서 사용

$$J\left(\mathbf{U}^1,\mathbf{U}^2,\cdots,\mathbf{U}^L\right)=\frac{1}{|M|}\sum_{\mathbf{x}\in M}\|\mathbf{y}-\mathbf{o}\|^2+\lambda\|\mathbf{U}\|^2$$
$$=\frac{1}{|M|}\sum_{\mathbf{x}\in M}\|\mathbf{y}-\mathbf{o}\|^2+\lambda\left(\left(u_{10}^1\right)^2+\left(u_{11}^1\right)^2+\cdots+\left(u_{10}^2\right)^2+\cdots+\left(u_{n_Ln_{L-1}}^L\right)^2\right) \qquad (6.6)$$

6.6 딥러닝의 규제

6.6.4 성능 평가 쟁점

- 인공지능 제품은 사람의 수준에 가깝거나 사람의 의사결정을 보조할 수 있을 정도의 성능을 갖추어야 시장에서 경쟁력을 갖출 수 있음
- 인공지능 프로젝트를 수행하는 사람은 제품을 시장에 내놓기 전에 엄밀한 성능 평가를 해야 함

- 딥러닝으로 인공지능을 구현하는 경우에는 모델의 구조를 무한정 다르게 설계할 수 있고 하이퍼 매개변수도 무한정 다르게 설정할 수 있으므로 최적의 모델을 알아내고 최적의 하이퍼 매개변수 설정을 알아내는 일은 불가능에 가까움

- 결국, 여러 사람이 받아들이는 〈합리적인 모델 구조〉와 〈합리적인 하이퍼 매개변수〉 설정을 사용하며, 중요한 하이퍼 매개변수에 대해 최적화를 시도하는 접근 방법을 써야함

6.6 딥러닝의 규제

6.6.4 성능 평가 쟁점

- 하이퍼 매개변수 최적화를 할 때 주의해야 할 두 가지 사항으로 교차 검증과 제거 조사를 소개하겠음
- 중요한 인공지능 프로젝트와 인공지능 분야의 좋은 논문은 대부분 교차 검증과 제거 조사를 요구함

6.6.4.2 제거 조사 (ablation study)

- 제거 조사는 옵션에서 하나씩 고 성능 실험을 진행하여 옵션의 기여 여부를 조사하는 기법임

데이터증대 (적용 or 미적용)	드랍아웃 (0 or 0.25)	L2 정규화 가중치 감쇠 (0 or 0.001)	성능
X	X	X	0.71424
X	X	O	0.70496
X	O	X	0.74072
X	O	O	0.70704
O	X	X	0.75652
O	X	O	0.70064
O	O	X	0.71050
O	O	O	0.66916

파라미터 값이 성능향상에 도움이 되지 않은 듯

6.6 딥러닝의 규제

[주의] 학습하는데 오래 걸림

6.6.4 성능 평가 쟁점

- [프로그램 6-10]은 데이터 증대, 드롭아웃, 가중치 감쇠의 세 가지 옵션의 조합에 대해 제거 조사를 실시함
- 이때 교차 검증을 적용함으로써 제거 조사 결과에 대한 신뢰를 높임

[프로그램 6-10] CIFAR10-교차 검증으로 제거 조사

```
1  from torch.optim import Adam
2  from torch.nn import CrossEntropyLoss
3
4  def train_model(model, dataloader, l2_reg):
5      # 학습을 위한 최적화 알고리즘과 손실 함수 정의
6      optimizer = Adam(model.parameters(), lr=1e-3, weight_decay=l2_reg)
7      loss_fn = CrossEntropyLoss().cuda()
8
9      for epoch in range(n_epoch):
10         for x_train_batch, y_train_batch in dataloader:
11             x_train_batch = x_train_batch.cuda()
12             y_train_batch = y_train_batch.cuda()
13
14             pred = model(x_train_batch)
15             loss = loss_fn(pred, y_train_batch)
16
17             optimizer.zero_grad()  # 옵티마이저 초기화
18             loss.backward()  # 오차 역전파
19             optimizer.step()  # 역전파된 오차를 이용한 파라미터 업데이트 수행
```

** 모델 정의까지는 [프로그램 6-9]와 동일

4행 : 모델 학습 시, l2 가중치 감쇠 가중치를 설정할 수 있도록 수정

6행 : 최적화 알고리즘에 weight_decay 를 추가해 가중치 감쇠 적용

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p10-codingnote

6.6 딥러닝의 규제

6.6.4 성능 평가 쟁점

[프로그램 6-10] CIFAR10-교차 검증으로 제거 조사

TensorDataset 대신 transform을 적용할 수 있는 CustomDataset 정의

```
1  class CustomDataset(torch.utils.data.Dataset):
2      def __init__(self, x, y, transform=None):
3          self.x = x
4          self.y = y
5          self.transform = transform
6
7      def __getitem__(self, idx):
8          x = self.x[idx]
9          if self.transform:
10             x = self.transform(x)
11         return x, self.y[idx]
12
13     def __len__(self):
14         return self.x.shape[0]
```

2-5행 : 데이터셋 초기화 함수

7-11행 : 데이터셋의 idx 번째 데이터를 불러올 때 사용

9-10행 : transform이 있을 경우 적용하여 반환

13-14행 : 데이터셋의 길이를 반환하는 함수

**파이썬 매직 메서드인 __getitem__, __len__ 함수는
각각 dataset[idx]와 len(dataset)을 사용했을 때 대응되는 함수!

6.6 딥러닝의 규제

6.6.4 성능 평가 쟁점

[프로그램 6-10] CIFAR10-교차 검증으로 제거 조사

```
1  from sklearn.model_selection import KFold
2  from torch.utils.data import DataLoader
3
4  # 드롭아웃 비율에 따라 교차 검증을 수행하고 정확률을 반환하는 함수
5  def cross_validation(data_aug, dropout_rate, l2_reg):
6      # 데이터 증대 방식 정의
7      if data_aug:
8          transform = Compose([
9              ToTensor(),
10             Pad(10, padding_mode="edge"),
11             RandomAffine(3.0, translate=(0.1, 0.1)),  # 주어진 범위 내에서 랜덤하게 회전 혹은 이동을 적용
12             CenterCrop((32, 32)),
13             RandomHorizontalFlip(p=0.5),  # 랜덤하게 수평 회전
14         ])
15     else:
16         transform = ToTensor()
17     test_transform = ToTensor()
18
19     accuracy = list()
20     for train_index, val_index in KFold(k).split(x_train):
21         # 훈련 집합과 검증 집합으로 분할
22         xtrain, xval = x_train[train_index], x_train[val_index]
23         ytrain, yval = y_train[train_index], y_train[val_index]
24
25         # 신경망 초기화
26         model = CNN(dropout_rate).cuda()
27
28         # 분할된 훈련 집합으로 데이터로더 준비
29         train_dataloader = DataLoader(CustomDataset(xtrain, ytrain, transform=transform), batch_size=batch_size, shuffle=True, drop_last=True)
30
31         # 학습 진행
32         train_model(model, train_dataloader, l2_reg)
33
34         # 분할된 검증 집합으로 데이터로더 준비
35         val_dataloader = DataLoader(CustomDataset(xval, yval, transform=test_transform), batch_size=batch_size, shuffle=False, drop_last=False)
```

7-16행 : data_aug가 True 이면 데이터 증대 적용

26행 : 주어진 dropout_rate에 따라 모델 선언

32행 : 주어진 l2 가중치 감쇠 가중치를 적용하여 학습 진행

6.6 딥러닝의 규제

6.6.4 성능 평가 쟁점

[프로그램 6-10] CIFAR10-교차 검증으로 제거 조사

**** 여러 가지 조건을 바꿔가며 제거 조사 수행**

```
# 여러 조건을 바꿔가며 평가
acc_000 = cross_validation(False, [0.0, 0.0, 0.0], 0.0)
acc_001 = cross_validation(False, [0.0, 0.0, 0.0], 0.01)
acc_010 = cross_validation(False, [0.25, 0.25, 0.5], 0.0)
acc_011 = cross_validation(False, [0.25, 0.25, 0.5], 0.01)
acc_100 = cross_validation(True, [0.0, 0.0, 0.0], 0.0)
acc_101 = cross_validation(True, [0.0, 0.0, 0.0], 0.01)
acc_110 = cross_validation(True, [0.25, 0.25, 0.5], 0.0)
acc_111 = cross_validation(True, [0.25, 0.25, 0.5], 0.01)
```

```
# 드롭아웃 비율에 따라 교차 검증을 수행하고 정확률을 반환하는 함수
def cross_validation(dropout_rate, data_aug, l2_reg):
```

**** 실험 결과를 박스 플롯으로 시각화**

```
import matplotlib.pyplot as plt

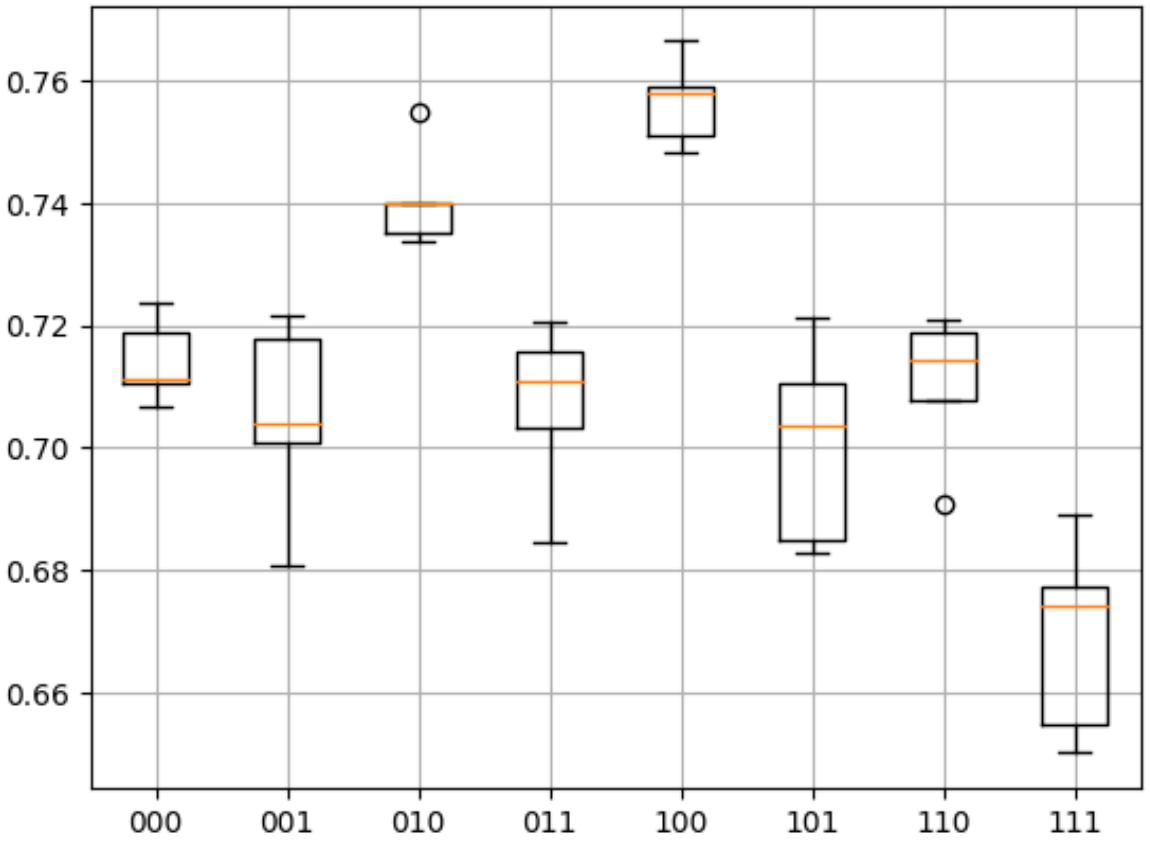
# 박스 플롯으로 정확률 표시
plt.grid()
plt.boxplot([acc_000, acc_001, acc_010, acc_011, acc_100, acc_101, acc_110, acc_111], labels=["000", "001", "010", "011", "100", "101", "110", "111"])
```

결과를 분석해보면?

6.6 딥러닝의 규제

6.6.4 성능 평가 쟁점

[프로그램 6-10] CIFAR10-교차 검증으로 제거 조사



제공받은 코드 직접 작성하기

- 6.5와 6.6에서 실습코드로 제공받은 내용을 반드시 직접 코딩해보기
- (상당한 분량의 코드임을 유의하여 연습 할 것)

6.7 전이 학습 (transfer learning)

- 심리학에서는 어떤 분야의 능숙함이 다른 분야의 학습에 크게 도움을 주는 사례를 학습의 전이 transfer learning 라고 함
  - 예를들어, 달리기를 잘 하는 사람은 축구도 잘하고, 피아노를 잘 치는 사람은 바이올린을 금방 배우며 채소를 잘 가꾸는 사람은 꽃도 잘 가꿈

- (개념정의) 기계학습 분야에서 전이 학습transfer learning 은 특정 도메인(A)에서 수집한 데이터로 학습한 모델을 다른 도메인(B)의 데이터를 인식하는데 활용해 성능을 향상하는 연구에 해당함
  - 예를들어, 자연 영상 Natural Image (실제 카메라로 촬영된 사진들)으로 구성된 ImageNet 데이터로 학습한 CNN 모델을 새의 종 데이터로 다시 학습해 새의 종을 높은 정확률로 인식하는 모델을 만들 수 있음

- (활용방안) 전이 학습은 주로 데이터가 부족하여 스크래치 학습 learn from scratch 으로 높은 성능을 달성하기 어려운 상황에서 활용함
  - 예를들어, CUB200-2011 데이터셋에는 200종의 새 영상이 있는데 클래스로 60장 가량의 영상이 있음. 훈련 집합은 클래스마다 30장뿐이라 과잉 적합이 일어날 가능성이 큼
  - 이런 상황에서는 대용량 데이터셋인 ImageNet으로 사전 학습 pre-trained 되어 있는 CNN 모델을 CUB200-2011 데이터로 전이하는 방안을 고려해 볼 수 있음 (*ImageNet 클래스 별 학습데이터 평균 1000장)

6.7 전이 학습

6.7.1 현대적인 컨볼루션 신경망 모델 : AlexNet, VGG, GoogLeNet, ResNet

- 전이 학습을 구현할 때는 대용량 데이터ImageNet로 이미 학습이 되어 있는 CNN 신경망 가중치를 사용함 (이미 학습된 가중치 Load 하여 사용)
- 딥러닝에서는 미리 학습되어 있는 모델을 사전 학습 모델pre-trained model이라 부름

- ImageNet-21K
  - ImageNet에는 21,841 클래스에 대해 1,400만 장 이상의 방대한 영상이 들어 있음
    - 클래스당 학습데이터 약 1000장
  - 영상의 크기는 영상마다 다른데 평균 469x387로 알려져 있음
  - ILSVRC 경진대회에서는 1000 클래스만 뽑고 훈련 집합 120만 장, 검증 집합 5만 장, 테스트 집합 15만 장을 사용함 (ImageNet-1K)

6.7 전이 학습

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- ImageNet을 사용해 스크래치 학습*을 할 수 있으면 좋겠지만 스크래치 학습을 하려면 많은 계산 자원과 시간 자원을 써야만 함 *스크래치 학습? 사전학습 가중치 없이 랜덤 초기화해서 처음부터 학습하는 것
- ILSVRC 대회에서 우승한 팀 역시 신경망의 구조 뿐만 아니라 가중치를 저장해 사전 학습 모델로 공개하여 제공하고 있음
- 딥러닝 라이브러리(pytorch 포함)는 사전 학습 모델을 쉽게 사용할 수 있게 클래스를 제공함
  - 모델 이름, ImageNet 테스트 집합에 대한 1순위와 5순위 정확률, 매개변수 개수 에 대한 정보를 같이 제공

**https://pytorch.org/vision/stable/models.html

Table of all available classification weights

Accuracies are reported on ImageNet-1K using single crops:

Weight	Acc@1	Acc@5	Params	GFLOPS	Recipe
AlexNet_Weights.IMAGENET1K_V1	56.522	79.066	61.1M	0.71	link
ConvNeXt_Base_Weights.IMAGENET1K_V1	84.062	96.87	88.6M	15.36	link
ConvNeXt_Large_Weights.IMAGENET1K_V1	84.414	96.976	197.8M	34.36	link
ConvNeXt_Small_Weights.IMAGENET1K_V1	83.616	96.65	50.2M	8.68	link
ConvNeXt_Tiny_Weights.IMAGENET1K_V1	82.52	96.146	28.6M	4.46	link
DenseNet121_Weights.IMAGENET1K_V1	74.434	91.972	8.0M	2.83	link
DenseNet161_Weights.IMAGENET1K_V1	77.138	93.56	28.7M	7.73	link
DenseNet169_Weights.IMAGENET1K_V1	75.6	92.806	14.1M	3.36	link
DenseNet201_Weights.IMAGENET1K_V1	76.896	93.37	20.0M	4.29	link
EfficientNet_B0_Weights.IMAGENET1K_V1	77.692	93.532	5.3M	0.39	link
EfficientNet_B1_Weights.IMAGENET1K_V1	78.642	94.186	7.8M	0.69	link
EfficientNet_B1_Weights.IMAGENET1K_V2	79.838	94.934	7.8M	0.69	link
EfficientNet_B2_Weights.IMAGENET1K_V1	80.608	95.31	9.1M	1.09	link
EfficientNet_B3_Weights.IMAGENET1K_V1	82.008	96.054	12.2M	1.83	link
EfficientNet_B4_Weights.IMAGENET1K_V1	83.384	96.594	19.3M	4.39	link
EfficientNet_B5_Weights.IMAGENET1K_V1	83.444	96.628	30.4M	10.27	link
EfficientNet_B6_Weights.IMAGENET1K_V1	84.008	96.916	43.0M	19.07	link
EfficientNet_B7_Weights.IMAGENET1K_V1	84.122	96.908	66.3M	37.75	link
EfficientNet_V2_L_Weights.IMAGENET1K_V1	85.808	97.788	118.5M	56.08	link
EfficientNet_V2_M_Weights.IMAGENET1K_V1	85.112	97.156	54.1M	24.58	link
EfficientNet_V2_S_Weights.IMAGENET1K_V1	84.228	96.878	21.5M	8.37	link
GoogLeNet_Weights.IMAGENET1K_V1	69.778	89.53	6.6M	1.5	link
Inception_V3_Weights.IMAGENET1K_V1	77.294	93.45	27.2M	5.71	link
MNASNet0_5_Weights.IMAGENET1K_V1	67.734	87.49	2.2M	0.1	link
MNASNet0_75_Weights.IMAGENET1K_V1	71.18	90.496	3.2M	0.21	link
MNASNet1_0_Weights.IMAGENET1K_V1	73.456	91.51	4.4M	0.31	link
MNASNet1_3_Weights.IMAGENET1K_V1	76.506	93.522	6.3M	0.53	link
MaxVit_T_Weights.IMAGENET1K_V1	83.7	96.722	30.9M	5.56	link
MobileNet_V2_Weights.IMAGENET1K_V1	71.878	90.286	3.5M	0.3	link
MobileNet_V2_Weights.IMAGENET1K_V2	72.154	90.822	3.5M	0.3	link
MobileNet_V3_Large_Weights.IMAGENET1K_V1	74.042	91.34	5.5M	0.22	link
MobileNet_V3_Large_Weights.IMAGENET1K_V2	75.274	92.566	5.5M	0.22	link
MobileNet_V3_Small_Weights.IMAGENET1K_V1	67.668	87.402	2.5M	0.06	link
RegNet_X_16GF_Weights.IMAGENET1K_V1	80.058	94.944	54.3M	15.94	link
RegNet_X_16GF_Weights.IMAGENET1K_V2	82.716	96.196	54.3M	15.94	link
RegNet_X_1_6GF_Weights.IMAGENET1K_V1	77.04	93.44	9.2M	1.6	link
RegNet_X_1_6GF_Weights.IMAGENET1K_V2	79.668	94.922	9.2M	1.6	link
RegNet_X_32GF_Weights.IMAGENET1K_V1	80.622	95.248	107.8M	31.74	link
RegNet_X_32GF_Weights.IMAGENET1K_V2	83.014	96.288	107.8M	31.74	link
RegNet_X_3_2GF_Weights.IMAGENET1K_V1	78.364	93.992	15.3M	3.18	link
RegNet_X_3_2GF_Weights.IMAGENET1K_V2	81.196	95.43	15.3M	3.18	link
RegNet_X_400MF_Weights.IMAGENET1K_V1	72.834	90.95	5.5M	0.41	link
RegNet_X_400MF_Weights.IMAGENET1K_V2	74.864	92.322	5.5M	0.41	link
RegNet_X_800MF_Weights.IMAGENET1K_V1	75.212	92.348	7.3M	0.8	link
RegNet_X_800MF_Weights.IMAGENET1K_V2	77.522	93.826	7.3M	0.8	link
RegNet_X_8GF_Weights.IMAGENET1K_V1	79.344	94.686	39.6M	8	link
RegNet_X_8GF_Weights.IMAGENET1K_V2	81.682	95.678	39.6M	8	link
RegNet_Y_128GF_Weights.IMAGENET1K_SWAG_E2E_V1	88.228	98.682	644.8M	374.57	link
VGG11_BN_Weights.IMAGENET1K_V1	70.37	89.81	132.9M	7.61	link
VGG11_Weights.IMAGENET1K_V1	69.02	88.628	132.9M	7.61	link
VGG13_BN_Weights.IMAGENET1K_V1	71.586	90.374	133.1M	11.31	link
VGG13_Weights.IMAGENET1K_V1	69.928	89.246	133.0M	11.31	link
VGG16_BN_Weights.IMAGENET1K_V1	73.36	91.516	138.4M	15.47	link
VGG16_Weights.IMAGENET1K_V1	71.592	90.382	138.4M	15.47	link
VGG16_Weights.IMAGENET1K_FEATURES	nan	nan	138.4M	15.47	link
VGG19_BN_Weights.IMAGENET1K_V1	74.218	91.842	143.7M	19.63	link
VGG19_Weights.IMAGENET1K_V1	72.376	90.876	143.7M	19.63	link
ViT_B_16_Weights.IMAGENET1K_V1	81.072	95.318	86.6M	17.56	link
ViT_B_16_Weights.IMAGENET1K_SWAG_E2E_V1	85.304	97.65	86.9M	55.48	link
ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1	81.886	96.18	86.6M	17.56	link
ViT_B_32_Weights.IMAGENET1K_V1	75.912	92.466	88.2M	4.41	link
ViT_H_14_Weights.IMAGENET1K_SWAG_E2E_V1	88.552	98.694	633.5M	1016.72	link
ViT_H_14_Weights.IMAGENET1K_SWAG_LINEAR_V1	85.708	97.73	632.0M	167.29	link
ViT_L_16_Weights.IMAGENET1K_V1	79.662	94.638	304.3M	61.55	link
ViT_L_16_Weights.IMAGENET1K_SWAG_E2E_V1	88.064	98.512	305.2M	361.99	link
ViT_L_16_Weights.IMAGENET1K_SWAG_LINEAR_V1	85.146	97.422	304.3M	61.55	link
ViT_L_32_Weights.IMAGENET1K_V1	76.972	93.07	306.5M	15.38	link
Wide_ResNet101_2_Weights.IMAGENET1K_V1	78.848	94.284	126.9M	22.75	link
Wide_ResNet101_2_Weights.IMAGENET1K_V2	82.51	96.02	126.9M	22.75	link
Wide_ResNet50_2_Weights.IMAGENET1K_V1	78.468	94.086	68.9M	11.4	link

6.7 전이 학습

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- 모델의 원조는 2012년에 ILSVRC 에서 우승을 차지한 AlexNet 임
- 이후에 나온 모델은 AlexNet의 성능을 압도하므로 AlexNet은 이제 전설이 되었음
- 앞서 제시된 사전 학습 모델 테이블은 현재 널리 쓰이고 있는 모델의 목록에 해당되며, 특히 2013년 이후 ILSVRC에서 우승을 차지한 VGG, GoogLeNet, ResNet이 유명함
- MobileNet은 높은 성능을 유지한 채로 신경망 구조를 작게 만들어 임베디드 소프트웨어나 스마트폰 앱에 맞게 개조한 모델임

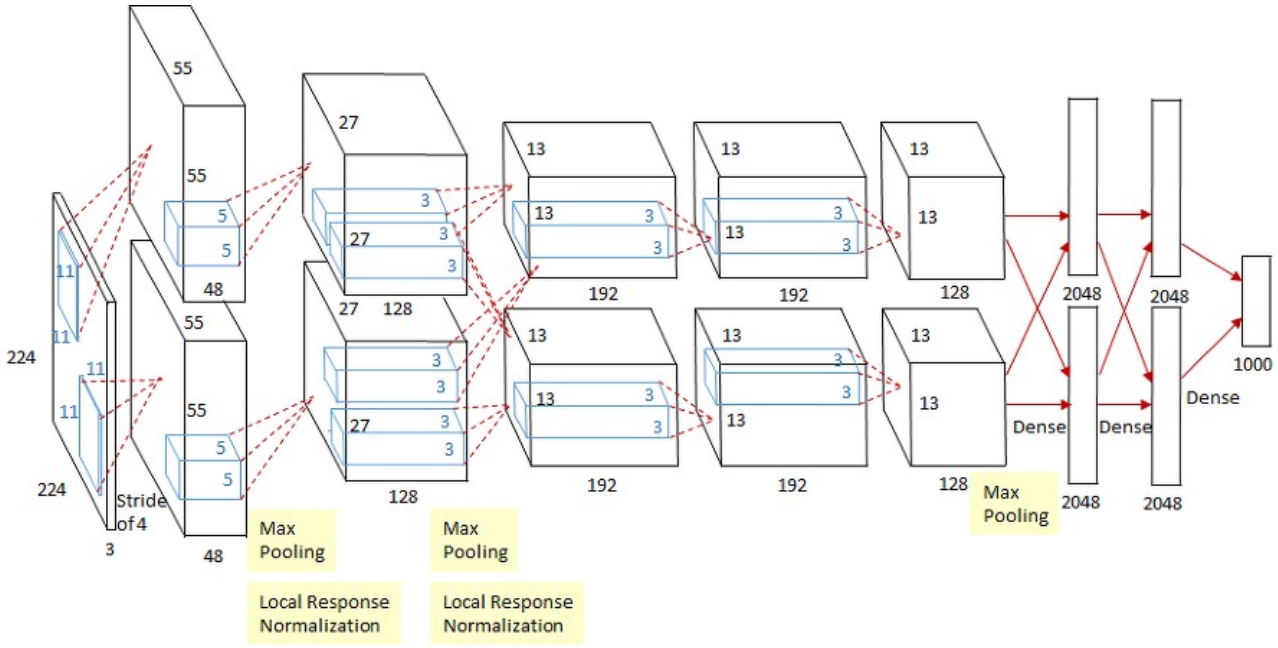
- VGG, GoogLeNet, ResNet은 공통적으로 컨볼루션층을 충분히 많이 쌓아 신경망의 층을 깊게 만듦
- 층을 깊게 하면 그레디언트 소멸 등으로 학습이 잘 안 되는 문제가 있는데 이 모델들은 여러 아이디어를 사용해 문제를 해결함
- VGG는 작은 마스크를 사용해 층을 깊게 만들고, GoogLeNet은 네트워크 속의 네트워크 라는 아이디어로 층을 깊게 만들며, ResNet은 지름길 연결 이라는 아이디어로 층을 더욱 깊게 만듦

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- AlexNet 모델 (2012)
  - 6천만 개의 매개변수가 존재하며, 2개의 GTX 580 3GB GPU에서 학습하는데 5~6일 소요됨
  - AlexNet은 ReLU 비선형성을 사용하여 빠르게 훈련할 수 있음을 보여줌(Tanh의 5배 빠름) *하이퍼볼릭 탄젠트
  - 과적합 방지를 위해 Data Augmentation(Mirroring, Random Crops)과 Dropout을 도입함



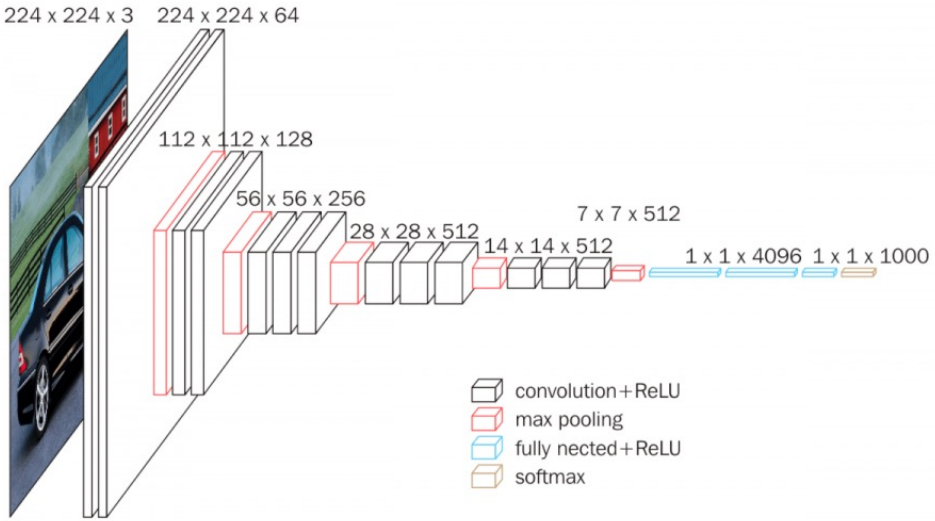
https://wikidocs.net/164787

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- VGG 모델 (VGG-16, VGG-19) (2014)
  - VGG는 2014년 ILSVRC에서 준우승을 차지함
  - VGG 는 옥스포트 대학 Visual Geometry Group의 약자, VGG 뒤의 숫자는 컨볼루션 레이어의 수를 의미함
  - VGG-16 : 13개의 CNN 층 + 3개의 FC 층
  - 큰 커널 크기의 필터(5x5, 7x7)를 여러 3x3 커널 크기 필터로 교체하여 AlexNet에 비해 성능 개선 성공
  - 단순하고 반복적인 획일화된 구조가 매력적임
  - VGG-16의 매개변수는 1억 3800만개로 Titan Black 6GB GPU 를 사용하여 몇 주간 학습



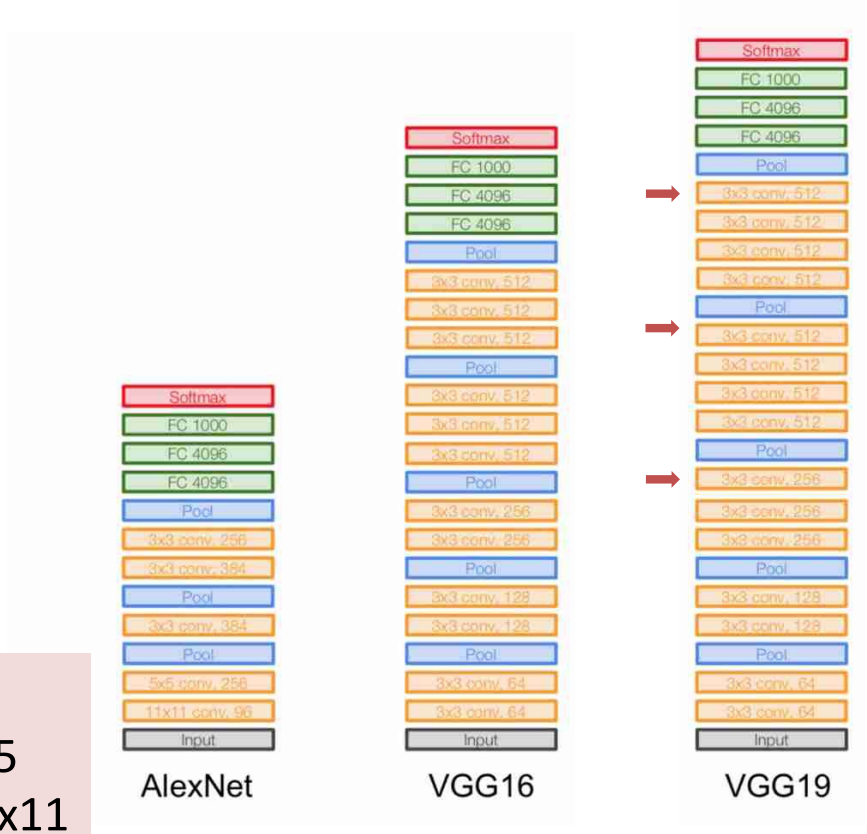
https://wikidocs.net/164796

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- VGG 모델 (VGG-16, VGG-19) (2014)
  - 입력은 고정된 크기 필요 (224x224), 학습데이터의 평균 값을 빼는 부분 필수!



➔
5x5
11x11

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
max-pool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
max-pool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
max-pool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
max-pool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
max-pool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

https://wikidocs.net/164796

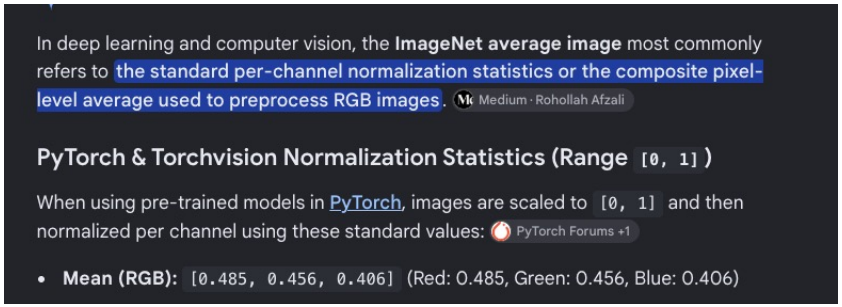
6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- VGG 모델 (VGG-16, VGG-19) (2014)
  - 입력은 고정된 크기 필요 (224x224), 학습데이터의 평균 값을 빼는 부분 필수!

  - 평균영상의 역할 (pixel-wise 평균)
    - 조명과 색 밝기의 편차 제거
    - 각 픽셀 값을 "0"중심으로 정렬하여 학습 안정화
    - 모델이 학습한 입력 분포와 추론 시 입력 분포를 동일하게 만드는 역할
    - 모델 학습 시 평균 값 빼는 부분이 반영되었다면, 추론 시 데이터 정규화도 반드시 진행되어야 함



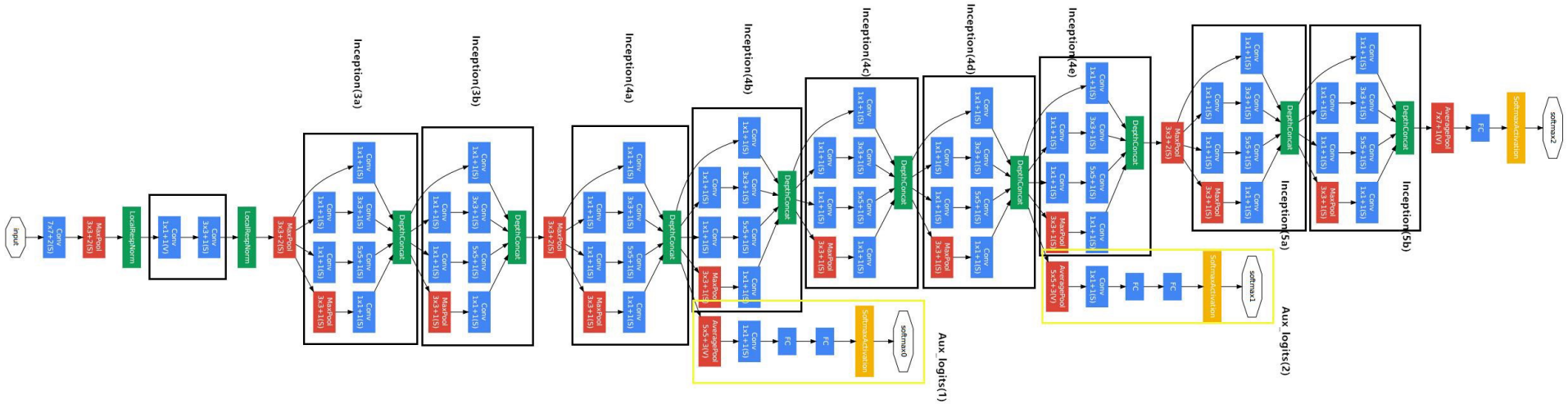
https://wikidocs.net/164796

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- Inception 모델 (2014)
  - 2014년 ILSVRC 대회에서 VGG 를 근소한 차이로 우승을 차지한 모델 (InceptionNet == GoogLeNet)
  - 속도와 정확성을 고려하여 여러 버전이 탄생 (V1, V2, V3, -ResNet)
  - 깊게만 쌓지 말고, (설계방향) 깊이와 너비 모두 깊게 쌓아보자
  - 층이 깊어 발생하는 그레디언트 소실 문제는 보조 분류기(Auxiliary classifier)로 해결
  - 복잡한 컨볼루션 연산을 decompose하는 효율적인 연산법 적용 (다음 슬라이드에서..)



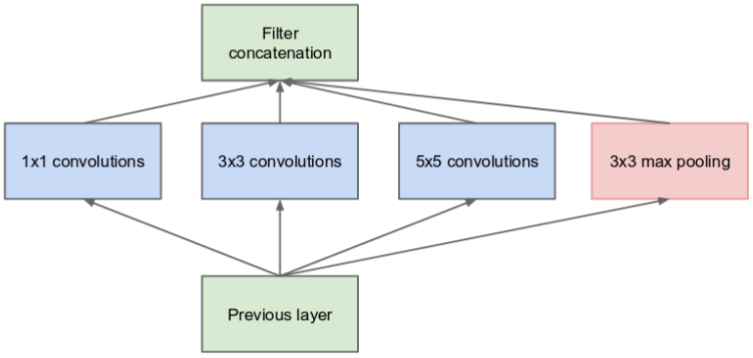
https://wikidocs.net/164794

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- Inception 모델 (2014)
  - 네트워크를 깊게 하면 표현력이 좋아지지만 연산량이 많아지고, 얇게 쌓아 연산량을 줄이면 표현력이 떨어지는 문제가 발생. ➔ 희소성(Sparsity) 구조을 통해 해당 문제를 해결 하려함
  - 희소성(Sparsity)이란 CNN의 부분연결성과 유사한 개념으로,연결이 적지만 효율적인 구조를 의미함
  - 그림(a)와 같이 병렬 Conv를 통해 sparse 연결을 dense 연산으로 근사
  - 이런 병렬 Conv 구조는 연산량 대비 표현력 향상, 다양한 공간적 특징 학습 가능함을 보였음



(a) Inception module, naïve version

입력 feature map (filter concatenation 출력)의
일부는 작은 receptive field (1×1)에,
일부는 큰 receptive field (5×5)에 반응하도록 되어
전체 네트워크가 "희소하게 활성화"되는 효과를 가져옴

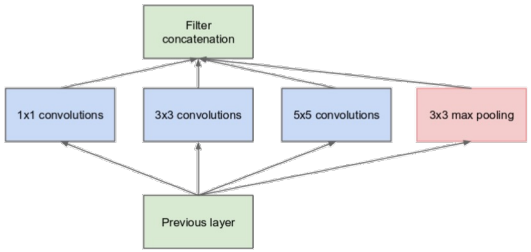
https://wikidocs.net/164794

6.7 전이 학습

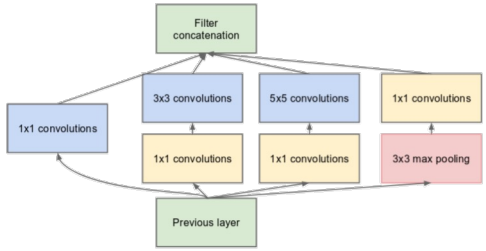
```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- **Inception V1 (2014)**
  - 값비싼 3x3 및 5x5 convolution 전에 <u>1x1 convolution을 사용하여 차원 축소</u>



(a) Inception module, naïve version



(b) Inception module with dimension reductions

예를 들어 입력이 **28×28×64**라면:

1. 각 위치에는 **64개 채널 값**, 즉 64차원 벡터가 있습니다.
2. 1×1 필터 하나의 실제 크기는 **1×1×64**입니다. 64개 값을 가중합해 **값 하나**를 만듭니다.
3. 이런 필터를 **16개** 사용하면 각 위치에서 값 16개가 나옵니다.

따라서 다음처럼 변합니다. *(stride=1 기준)*

$$\underbrace{28 \times 28 \times 64}_{\text{입력}} \xrightarrow{1\times1 \text{ 필터 } 16\text{개}} \underbrace{28 \times 28 \times 16}_{\text{출력}}$$

- **Inception V2**
  - 5x5 컨볼루션을 2개의 3x3컨볼루션 연산으로 인수분해하여 계산 속도 향상
  - 3x3 컨볼루션은 3x1다음에 1x3 을 통해 동일한 표현 효과 가능

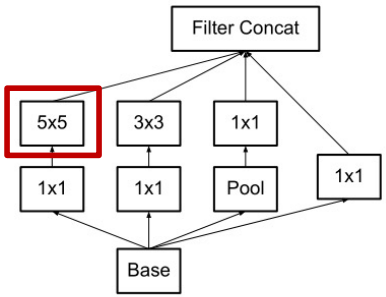


Figure 4. Original Inception module as described in [20].

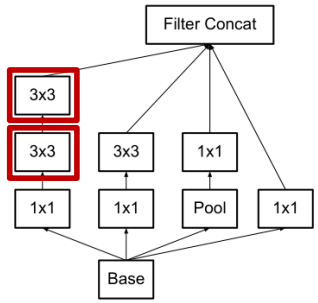
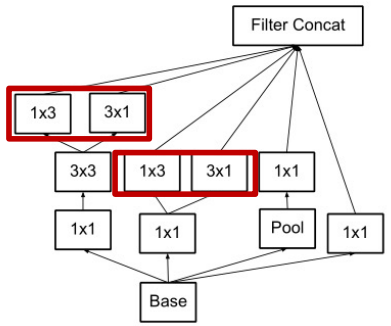


Figure 5. Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle 3 of Section 2



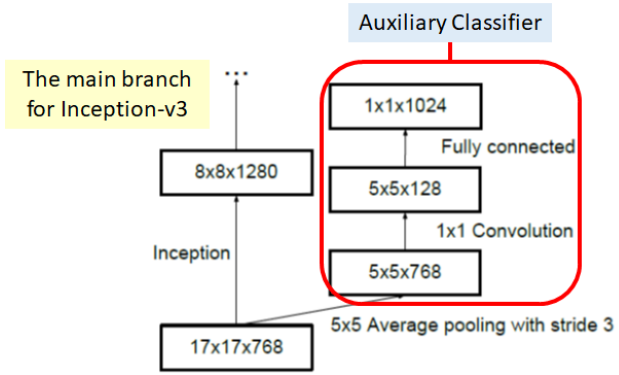
https://wikidocs.net/164794

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- Inception V3
  - (2개의 보조 분류기를 사용하는 대신) 마지막 17x17 레이어의 상단에 **1개의 보조 분류기만** 사용함
    - 아래 그림과 같이
  - V1에서 2개의 보조 분류기는 더 깊은 신경망 설계(그레디언트 소실 방지)를 위해 사용되었으나, V3에서 **1개의 보조 분류기는 정규화(regularization)**를 위해 사용되었음
    - 중간 특징과 뒤쪽 층이 함께 최종 정답을 맞히도록 손실함수 설계 $L_{\text{total}} = L_{\text{main}} + \lambda L_{\text{aux}}$



Inception v3 저자들이 정규화 역할을 강조한 것은 **실험 결과** 때문입니다.

- 보조 분류기가 있어도 **학습 초반의 수렴 속도는 거의 같았음**
- **학습 후반에 더 높은 정확도**에 도달했음
- 보조 분기에 **Batch Normalization**이나 **Dropout**을 적용하면 **주 분류기의 성능도 향상**되었음

그래서 저자들은 원래 기대했던 **기울기 소실 완화보다 정규화 효과가 주요 역할**이라고 해석했습니다. 이는 실험에 근거한 해석입니다. openaccess.thec...

보조 분류기는 **학습 중 중간 특징을 지도하는 역할**을 하고, 추론할 때는 주 분류기만 사용합니다.

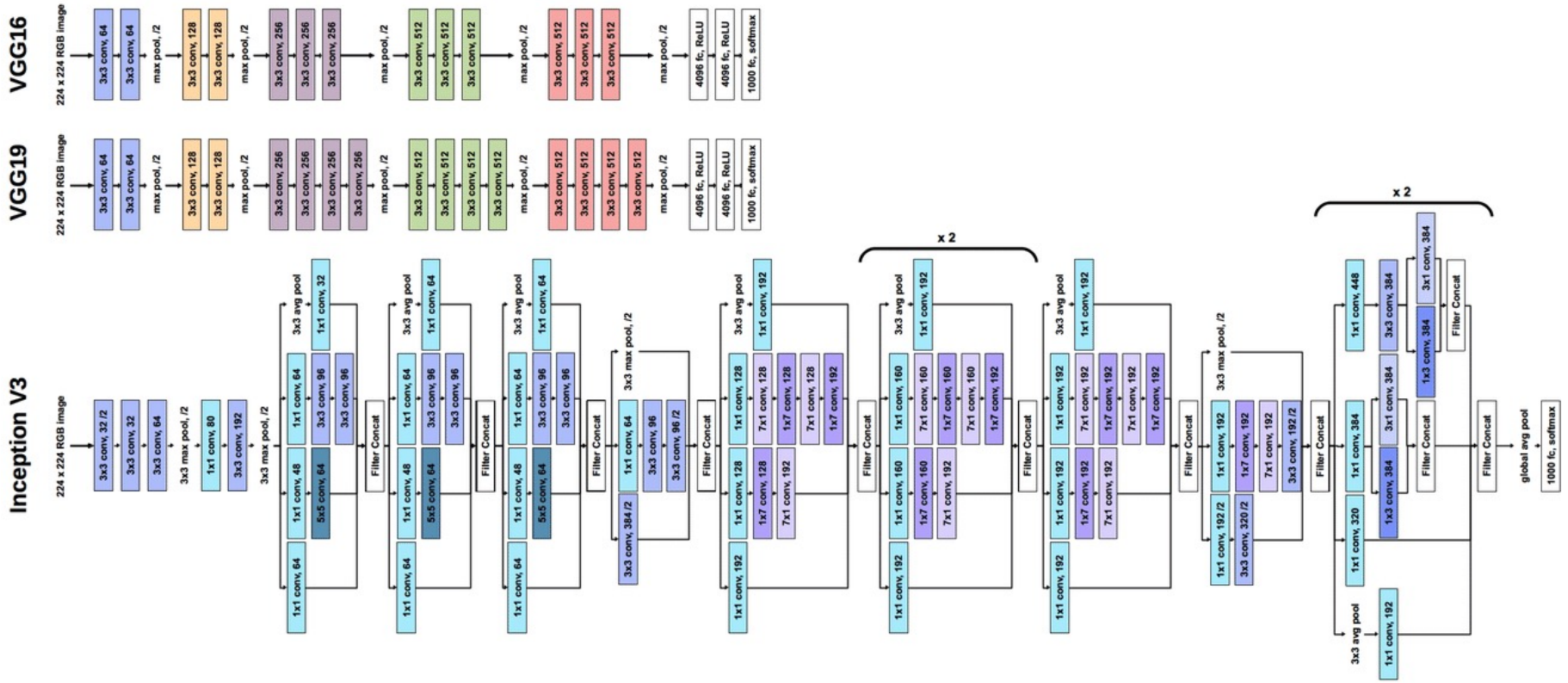
https://wikidocs.net/164794

6.7 전이 학습

```
LeNet (1998)
   ↓
AlexNet (2012)
   ↓
VGG (2014)
   ↓
GoogLeNet → Inception 시리즈 (2014~2016)
   ↓
ResNet (2015) → DenseNet / ResNeXt (2017)
   ↓
MobileNet / Xception / SENet (2017~2018)
   ↓
EfficientNet (2019)
   ↓
ConvNeXt (2022)
```

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- Inception (2014)



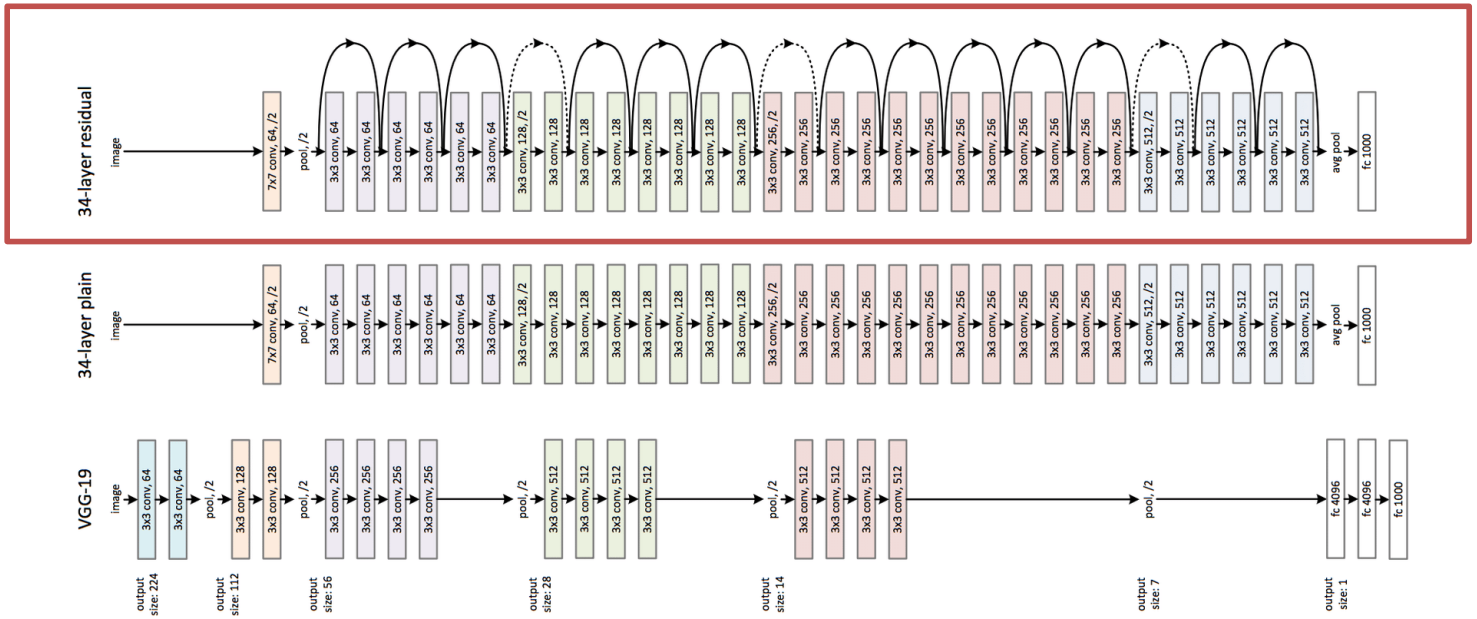
V3는 factorized convolution 사용을 확인 가능

https://wikidocs.net/164794

6.7 전이 학습

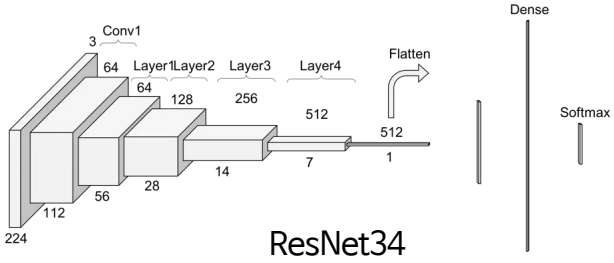
6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- ResNet 모델 (ResNet18, ResNet34, ResNet50, ResNet101, ResNet152)
  - 마이크로소프트에서 만든 ResNet은 2015년 ILSVRC 대회에서 3.5%의 5순위 오류율로 우승을 차지함
  - 아래 그림에서 확인할 수 있듯 이전 컨볼루션층의 결과를 현재 컨볼루션층의 결과를 받는 곳으로 보내 현재 컨볼루션창의 결과와 이전 컨볼루션층의 결과를 결합함
  - 이 아이디어를 지름길 연결shortcut connection 또는 건너뛰기 연결skip connection 이라고 부름
  - 지금 길 연결을 사용한 학습을 잔류 학습residual learning 이라고 함



https://wikidocs.net/164800

6.7 전이 학습



ResNet34

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- ResNet 모델
  - 모델의 종류
    - ResNet18, ResNet34 ← Basic Block
    - ResNet50, ResNet101, ResNet152, ResNet200 ← Bottleneck Block

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	[3×3, 64; 3×3, 64] ×2	[3×3, 64; 3×3, 64] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3
conv3_x	28×28	[3×3, 128; 3×3, 128] ×2	[3×3, 128; 3×3, 128] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×8
conv4_x	14×14	[3×3, 256; 3×3, 256] ×2	[3×3, 256; 3×3, 256] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×23	[1×1, 256; 3×3, 256; 1×1, 1024] ×36
conv5_x	7×7	[3×3, 512; 3×3, 512] ×2	[3×3, 512; 3×3, 512] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Basic Block

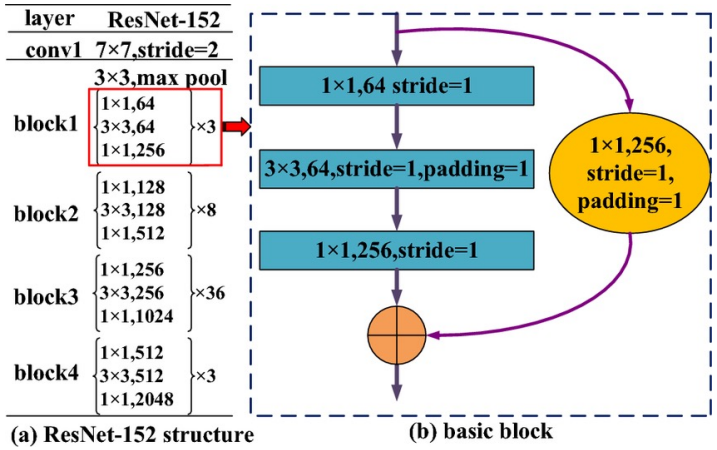
Bottleneck Block

https://wikidocs.net/164800

6.7 전이 학습

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망

- ResNet 모델
  - 모델의 종류
    - ResNet18, ResNet34 ← Basic Block
    - ResNet50, ResNet101, ResNet152, ResNet200 ← Bottleneck Block
  - ResNet-152 ← ILSVRC 우승 모델
    - Conv1 (앞) + 3개 레이어 * 50 blocks + FC (뒤)



Bottleneck block은 채널 수를 줄인 상태에서 3×3 convolution을 수행하고, 다시 채널 수를 늘리는 ResNet 블록이에요. 앞서 설명한 1×1 convolution의 차원 축소를 이용해 계산량을 줄이는 구조입니다.

예를 들어, 입력과 출력이 256채널인 블록은 다음처럼 동작합니다.

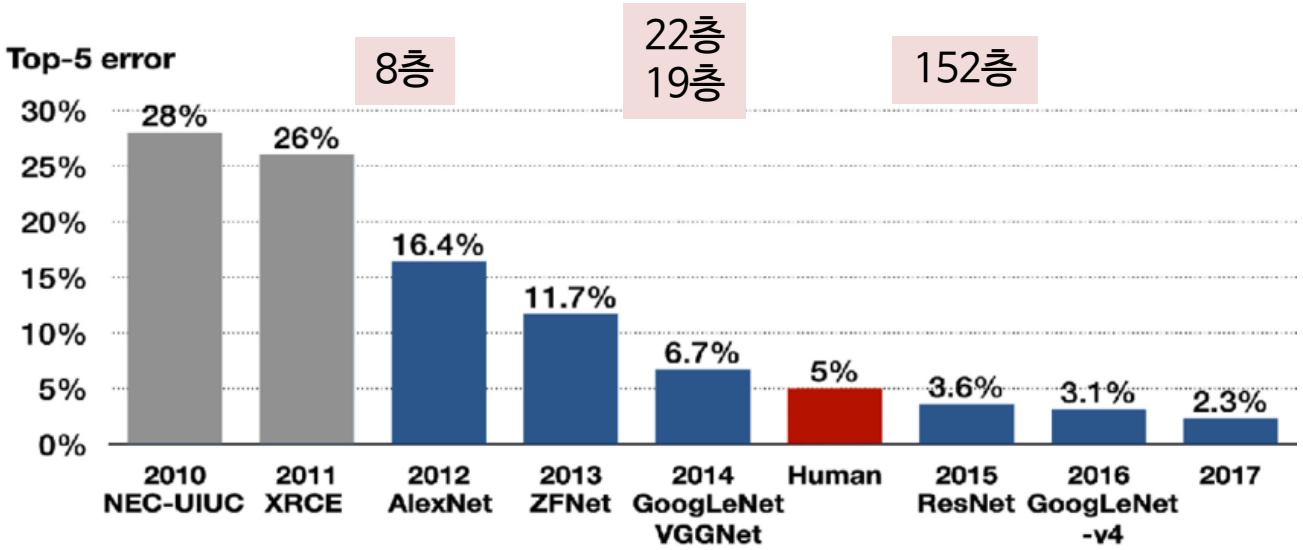
순서	연산	채널 수 변화	역할
①	1×1 Conv	256 → 64	채널 축소
②	3×3 Conv	64 → 64	적은 계산량으로 공간 특징 추출
③	1×1 Conv	64 → 256	채널 확장
④	입력 더하기	256 + 256 → 256	Skip connection

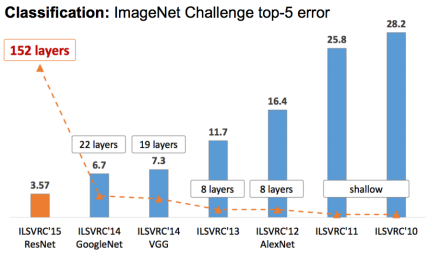
중간 부분의 채널이 256 → 64 → 256으로 좁아졌다가 넓어져서, 병의 목처럼 생겼다는 의미로 bottleneck이라고 부릅니다.

https://wikidocs.net/164800

6.7 전이 학습

6.7.1.1 딥러닝 라이브러리가 제공하는 현대적인 컨볼루션 신경망





6.7 전이 학습 (실습)

6.7.2 전이 학습 프로그래밍: 새의 품종 종류

- 전이 학습은 데이터셋이 작아 스크래치 학습으로 높은 성능을 얻기 어려운 상황에서 주로 사용함

6.7.2.1 ImageNet으로 학습한 ResNet 모델을 새 품종 인식 문제로 전이

- [그림 6-17]은 CUB200-2011 데이터셋에서 뽑은 예제 영상임
- 데이터셋은 칼텍 caltech에서 제작했으며 200종의 새 영상 11,788장이 담겨 있고, 종별로 평균 약 60장임
- 서로 다른 종이지만 부리 모양만 다르거나 발의 색깔만 달라 높은 정확률을 달성하기 어려운 미세 분류 문제fine-grained classification 임



그림 6-17 cub200-2011 데이터셋의 예제 영상

← 부류 내 변화가 아주 큼 within-class variance

← 부류 간 유사도가 아주 큼 between-class similarity

6.7 전이 학습

6.7.2.2 미세 조정 방식[1]과 동결 방식[2]

- 학습률을 낮게 유지하여 조심스럽게 조금씩 “전체 가중치”를 수정하는 전이 학습 방식을 미세 조정(fine-tuning)이라고 함
- 또 다른 학습 방식으로 동결(freezing)이 있으며, 동결 방식에서는 컨볼루션층의 가중치를 고정하여 수정이 일어나지 못하게 제한한 채 완전연결층(FC, 분류기)만 수정함

- 일반적으로 동결 방식보다 미세 조정 방식의 정확률이 뛰어남

6.7 전이 학습

6.7.2 전이 학습 프로그래밍

- 실습 시 메모리 사용량을 줄이기 위해, 200개의 클래스 중 20개만 사용

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

- 커스텀 데이터셋 사용하기
- 사전학습 모델 사용하기 (Pre-trained Weight Load)
- 사전학습 모델 수정하기 (FC 1000 ➔ FC 20)
- 미세조정 혹은 가중치동결 기법 다루기

https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p11-codingnote

6.7 전이 학습

6.7.2 전이 학습 프로그래밍

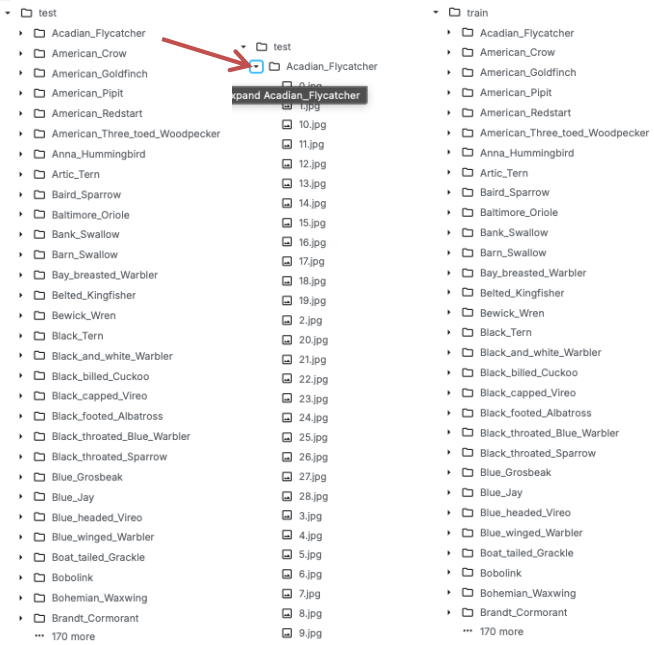
- 실습 시 메모리 사용량을 줄이기 위해, 200개의 클래스 중 20개만 사용

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

```
1  import json
2  from itertools import islice
3
4  # 클래스명과 정수 ID의 변환을 위한 딕셔너리 읽어오기
5  with open("/kaggle/input/cub200-2011-with-traintest-split/class_dict.json", "r") as f:
6      class_dict = json.load(f)
7
8  class_dict = {v:k for k,v in class_dict.items()}  # 딕셔너리의 키와 벨류값 바꿔주기
9  class_dict = dict(islice(class_dict.items(), 20))  # 데이터의 수를 줄이기 위해 20개 클래스의 데이터만 사용
10 class_dict
```

5-6행 JSON 형식의 파일을 읽어 딕셔너리 형태로 저장하기

```
{'Black_footed_Albatross': '0',
 'Laysan_Albatross': '1',
 'Sooty_Albatross': '2',
 'Groove_billed_Ani': '3',
 'Crested_Auklet': '4',
 'Least_Auklet': '5',
 'Parakeet_Auklet': '6',
 'Rhinoceros_Auklet': '7',
 'Brewer_Blackbird': '8',
 'Red_winged_Blackbird': '9',
 'Rusty_Blackbird': '10',
 'Yellow_headed_Blackbird': '11
 'Bobolink': '12',
 'Indigo_Bunting': '13',
 'Lazuli_Bunting': '14',
 'Painted_Bunting': '15',
 'Cardinal': '16',
 'Spotted_Catbird': '17',
 'Gray_Catbird': '18',
 'Yellow_breasted_Chat': '19'}
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p11-codingnote

6.7 전이 학습

6.7.2 전이 학습 프로그래밍

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

- __init__ 은 클래스(객체)가 처음 생성될 때 단 한번만 호출되는 함수
- __getitem__은 파이토치 DataLoader가 미니배치를 만들 때, Dataset에서 한 개의 데이터 샘플을 꺼내오는 순간 호출되는 함수
- __len__은 데이터셋의 총 길이(샘플 개수)가 필요할 때 호출되는 함수

```
1  import os
2  from PIL import Image  # 이미지를 읽어오기 위한 라이브러리
3
4  # CUB200 커스텀 데이터셋 정의
5  class CUB200(torch.utils.data.Dataset):
6      def __init__(self, root, class_dict, split="train", transform=None):
7          self.root = root
8          self.split = split
9          self.transform = transform
10         self.class_dict = class_dict
11         self.x = list()  # 이미지들의 경로를 담을 리스트
12         self.y = list()  # 라벨을 담을 리스트
13
14         data_dir = os.path.join(self.root, self.split)
15         for cls in self.class_dict:
16             cls_dir = os.path.join(data_dir, cls)  # 각 클래스의 이미지들이 들어있는 경로
17             imgs = os.listdir(cls_dir)  # 각 이미지의 이름
18             imgs = [os.path.join(cls_dir, img) for img in imgs]  # 이미지의 이름을 경로로 변환
19             self.x += imgs  # 해당 클래스의 이미지들을 저장
20             self.y += [int(self.class_dict[cls])] * len(imgs)  # 추가한 이미지들의 라벨 추가
21
22         self.x = np.array(self.x)
23         self.y = np.array(self.y)
24
25     def __getitem__(self, idx):
26         img = self.x[idx]  # 읽어올 이미지의 경로
27         img = Image.open(img)  # 이미지 읽어오기
28         if self.transform:
29             img = self.transform(img)
30         return img, self.y[idx]
31
32     def __len__(self):
33         return self.x.shape[0]
```

/train/[클래스명] 형태의 폴더에 저장된 이미지들을 불러오기 위한 커스텀 데이터셋 클래스

15-20행 사용할 클래스의 이미지들의 경로 저장해두기

26-27행 이미지를 사용할 때, 경로로부터 PIL 라이브러리를 통해 이미지 읽기

6.7 전이 학습

6.7.2 전이 학습 프로그래밍

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

서로 다른 크기의 이미지들을 224x224 크기로 통일해주는 transform 정의 ⇔ ResNet50의 입력이 224x224로 받기 때문

```
from torchvision.transforms import ToTensor, Resize, InterpolationMode, Compose
transform = Compose([
    ToTensor(),  # PIL.Image를 torch.Tensor로 변환하고, 255를 나눠 scale 조정
    Resize((224, 224), interpolation=InterpolationMode.NEAREST)  # 이미지를 224x224의 고정된 크기로 변환
])
```

앞서 정의한 데이터셋 선언

```
train_dataset = CUB200("/kaggle/input/cub200-2011-with-traintest-split", class_dict,"train", transform=transform)
test_dataset = CUB200("/kaggle/input/cub200-2011-with-traintest-split", class_dict,"test", transform=transform)
```

6.7 전이 학습

6.7.2 전이 학습 프로그래밍

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

In [25]:

```
from torch.optim import Adam
from torch.nn import CrossEntropyLoss
from torch.utils.data import DataLoader

# 학습을 위한 최적화 알고리즘과 손실 함수 정의
# [주의**] 전이 학습을 수행할 때는 일반적으로 낮은 학습률을 사용하는 것이 좋습니
다.
optimizer = Adam(model.parameters(), lr=2e-5)
loss_fn = CrossEntropyLoss().cuda()

# 데이터를 배치 단위로 불러오기 위한 데이터로더 정의
train_dataloader = DataLoader(train_dataset, batch_size=16, shu
ffle=True, drop_last=True)
val_dataloader = DataLoader(test_dataset, batch_size=16, shuffl
e=False, drop_last=False)
```

** 전이 학습을 수행할 시, 모델을 스크래치부터 학습할 때보다 낮은 학습률 (learning rate) 사용

6.7 전이 학습

6.7.2 전이 학습 프로그래밍

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

```
from torch import nn
from torchvision import models

# 위의 ResNet50 모델을 보면, FC가 2048 => 1000 의 구조를 가지고 있는데, 이를 2048=> 1024 => 20 으로 변경 할 예정
# 물론 2048 => 20 으로 적용해도 문제는 없는데, 단계적으로 적용하는게 일반적임

class ResNet50(nn.Module):
    def __init__(self, freeze_resnet=False):
        super(ResNet50, self).__init__()
        self.pretrained_resnet = models.resnet50(pretrained=True)  # 사전학습된 ResNet50 모델을 불러오기

        if freeze_resnet:  # 만약 사전학습된 계층들은 학습하지 않고 동결하고자 할 경우
            #for child in self.pretrained_resnet.children():  # resnet50의 각 계층에 접근 - 층의 이름 확인 불필요할 때 사용 하면 됨
            for name, child in self.pretrained_resnet.named_children():
                print(name)
                #if name in ['layer4', 'fc']:
                for param in child.parameters():  # 각 계층의 가중치와 편향에 접근
                    param.requires_grad = False   # 해당 파라미터는 기울기를 계산하여 학습하지 않음


        # 예측에 사용되는 ResNet의 마지막 계층들을 새로이 정의
        self.pretrained_resnet.fc = nn.Sequential(
            nn.Linear(2048, 1024),
            nn.ReLU(inplace=True),
            nn.Linear(1024, 20)
        )

    def forward(self, input):
        return self.pretrained_resnet(input)

#model = ResNet50(freeze_resnet=False).cuda()  # 전체 모델을 다시 미세 조정하는 방식
model = ResNet50(freeze_resnet=True).cuda()  # ResNet을 freeze하는 실험도 해보세요!
```

```
conv1
bn1
relu
maxpool
layer1
layer2
layer3
layer4
avgpool
fc
```

특정 층을 freeze 하고 싶을 때,
If 문과 같이 "층 이름"으로 필터링

ResNet50은 기본적으로 모든 학습 파라미터의 requires_grad=True입니다.

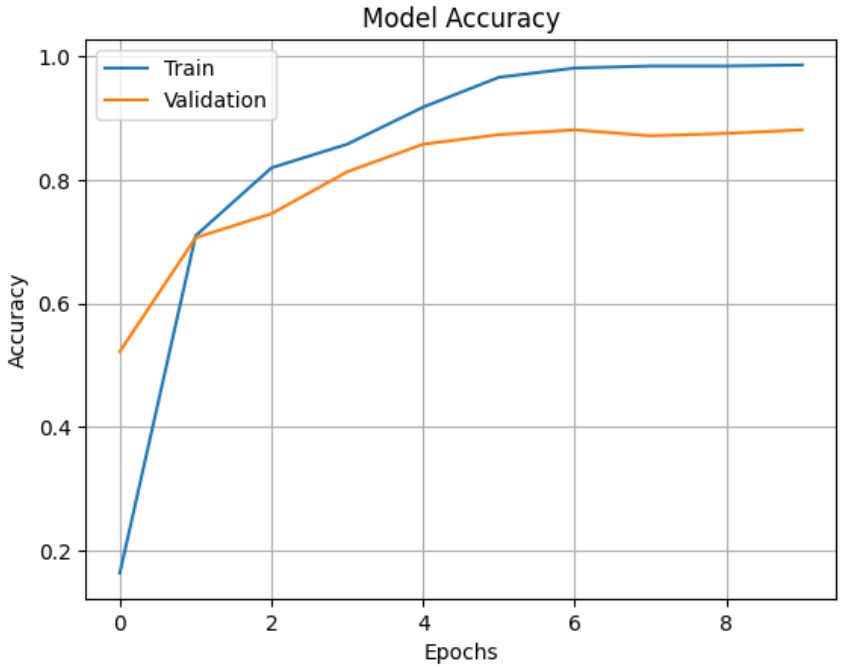
pretrained=True로 사전학습 가중치를 불러와도 자동으로 동결되지는 않습니다.

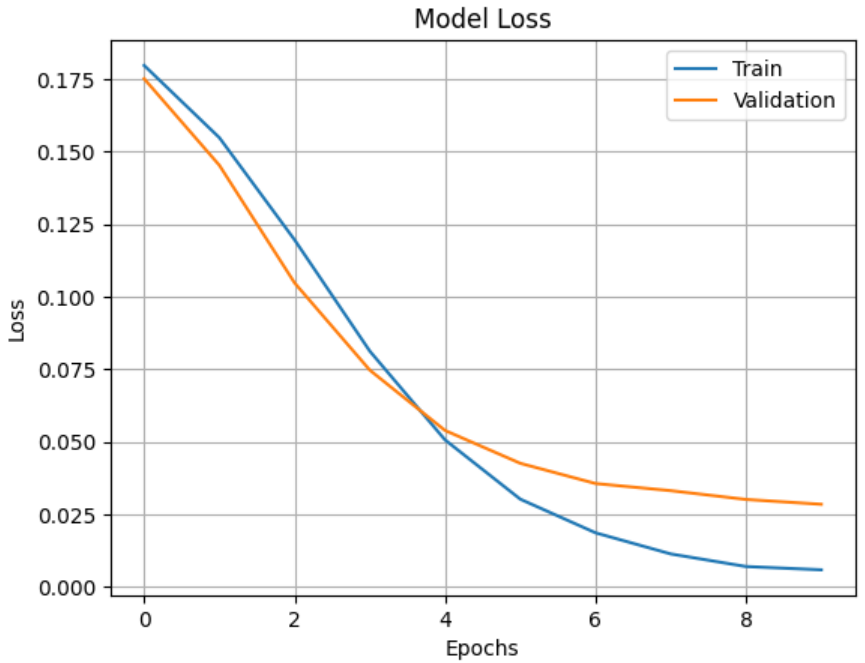
6.7 전이 학습

6.7.2 전이 학습 프로그래밍

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

```
model = ResNet50(freeze_resnet=False).cuda()  # 전체 모델을 다시 미세 조정하는 방식
```



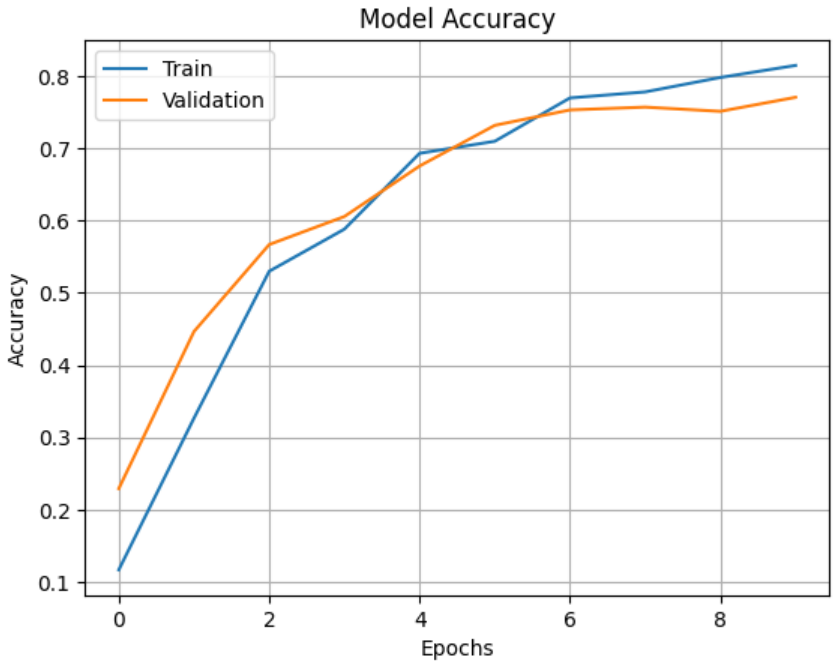


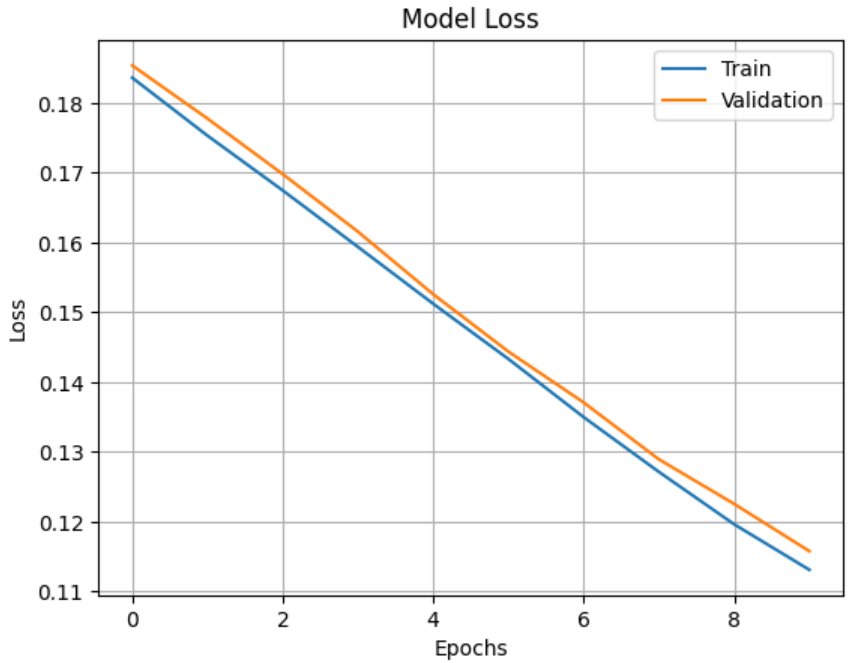
6.7 전이 학습

6.7.2 전이 학습 프로그래밍

[프로그램 6-11] ImageNet으로 학습된 ResNet50을 CUB200으로 전이 학습

```
model = ResNet50(freeze_resnet=True).cuda()  # ResNet을 freeze하는 실험 (새로 추가한 층만 학습)
```





정확도가 확실히 떨어지고, loss 그림 상 아직 수렴하지 않은 상황으로 보여짐

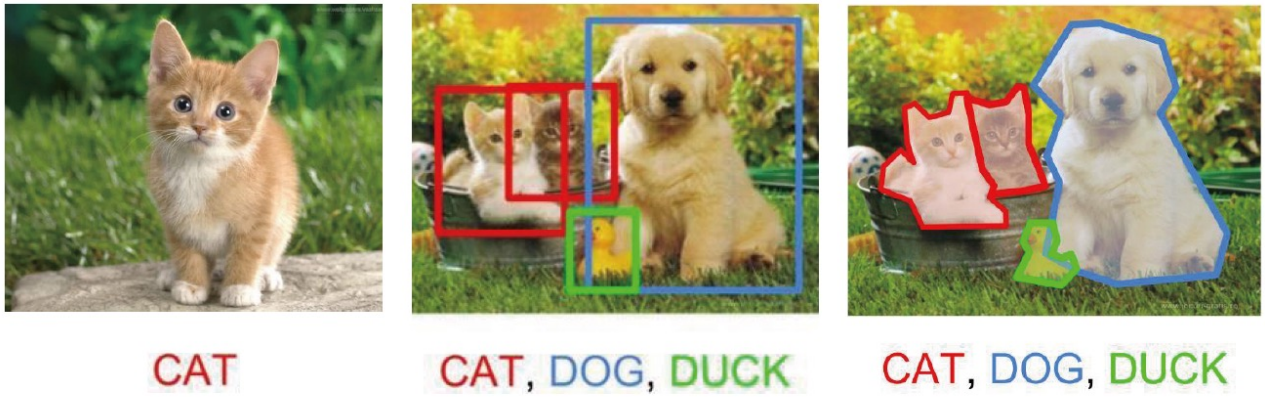
CUB200 ResNet50 사전학습 모델 활용하기 실습 과제

- 제출 방식 : 이메일로 보고서 제출
  - 마감 : 11월 09일 (월) 23시 59분
  - 이메일 주소 : admin@rcv.sejong.ac.kr
  - 이메일 제목 : [2026-2] [인공지능] 10주차 과제 제출 (학번_이름)
  - 보고서 분량 제한 없음

- [문제1] 사전학습 모델을 이용하여
  - ResNet50 사전학습 모델을 이용하여 문제를 해결하시오.
  - 가장 적절한 옵티마이저와 파라미터 값을 찾아 현재 설정된 베이스라인을 해결하시오.
  - https://www.kaggle.com/t/ecb58be92b9f4d1b9a87a2fe01c43a84

6.8 물체 검출

- **컴퓨터 비전 내 다양한 응용**
  - 분류 문제 (classification)
  - 검출 문제 (detection)
  - 분할 문제 (segmentation)



(a) 분류 (b) 검출 (c) 분할

그림 6-18 컴퓨터 비전의 세 가지 문제

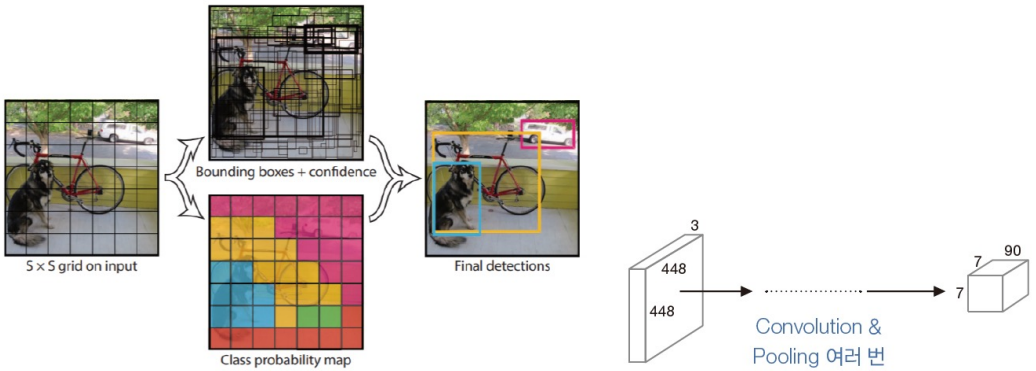
6.8 물체 검출

- **물체 검출을 위한 딥러닝 모델**
  - R-CNN
  - Fast R-CNN
  - Faster R-CNN
  - **YOLO (You Only Look Once)**
    - R-CNN계열과 달리 Yolo는 One-Stage Detector 임
    - 초당 30FPS 이상 처리할 수 있는 속도를 보장해 실시간 물체 검출에 널리 애용됨
    - 이 책에서는 원리가 간단하면서 빠르고 프로그래밍하기에 편리한 욜로에 대해 소개할 예정임

6.8 물체 검출

▪ 6.8.1 욜로를 이용한 물체 검출

- 입력 영상을 SxS 격자로 나눈 S^2 개의 격자 방grid cell 각각에 대해 B개의 바운딩 박스를 생성함
  - 바운딩 박스는 (x,y,w,h,o)의 5개 값으로 표현하는데, x와 y는 중심점, w와 h는 너비와 높이, o는 물체일 가능성을 의미함
- 또한, 격자 방 각각에 대해 C개의 클래스 확률을 계산함
  - MS COCO 데이터셋에는 사람, 자전거, 자동차 등 80개의 물체 클래스가 있으므로 C=80임
- 논문에는 입력 영상 448x448 로 변환하며, S=7과 B=2를 사용하였음.
  - 요약하면 49개 격자 방 각각은 자신이 어떤 물체 부류에 속하는지를 80차원 벡터로 표현하며, 바운딩 박스 2개를 5*2=10차원 벡터로 표현함

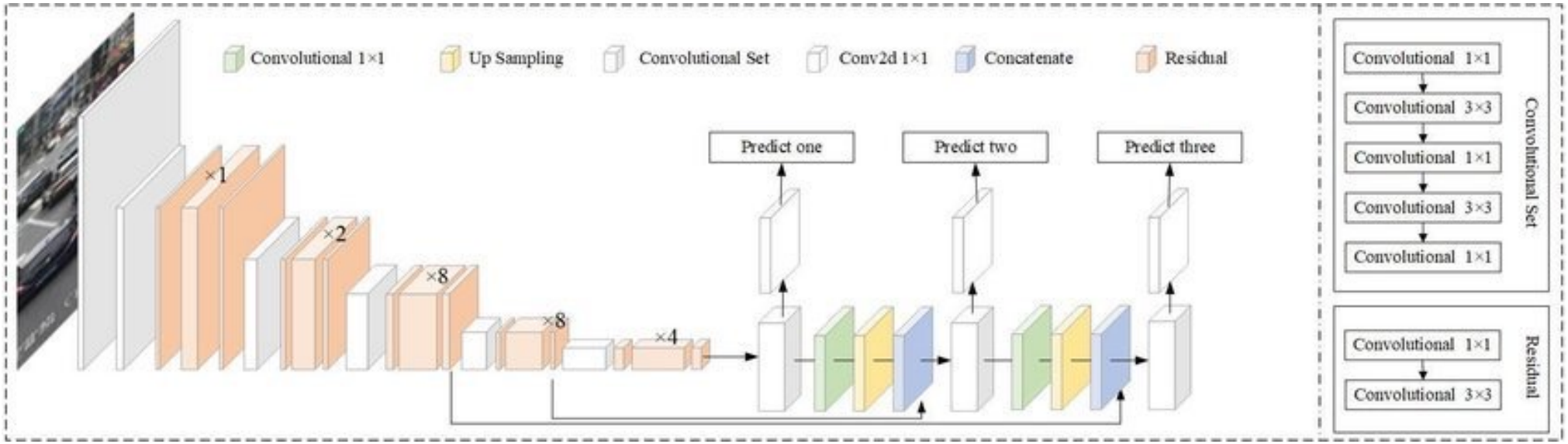


(a) 처리 절차 (b) 컨볼루션 신경망의 구조

그림 6-19 욜로의 원리(출처: [Redmon2016])

6.8 물체 검출

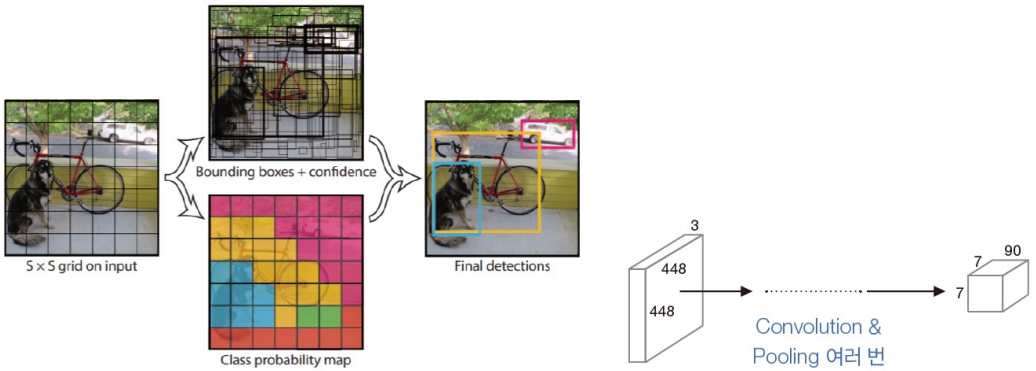
- 6.8.1 욜로를 이용한 물체 검출



YOLO v3 architecture

6.8 물체 검출

- **6.8.1 욜로를 이용한 물체 검출**
  - [그림 6-19(b)]는 448x448x3텐서를 입력으로 받아, [Conv+Pool]을 반복하다가 7x7x90 텐서를 출력하는 욜로의 신경망 구조를 보여줌
  - 7x7은 격자 구조를 나타내며, 90 차원은 1)클래스 확률을 표현하는 80차원 벡터와 2)바운딩 박스 2개를 표현하는 10차원 벡터를 결합한 것
  - [그럼 6-19(a)]의 중간에 있는 영상은 신경망이 출력한 바운딩 박스와 클래스 확률의 예시임



(a) 처리 절차 (b) 컨볼루션 신경망의 구조

그림 6-19 욜로의 원리(출처: [Redmon2016])

6.8 물체 검출

- **6.8.1 욜로를 이용한 물체 검출**
  - [그림 6-20]은 YOLOv3이라 부르는 버전3의 신경망 구조를 보여줌
  - 버전3에서는 14x14, 28x28, 56x56의 3개 스케일을 사용해 물체 검출 능력을 향상함
  - 각각 yolo_82와 yolo_94라는 이름의 중간 층, yolo_106이라는 이름의 마지막 층에서 텐서를 추출할 수 있음

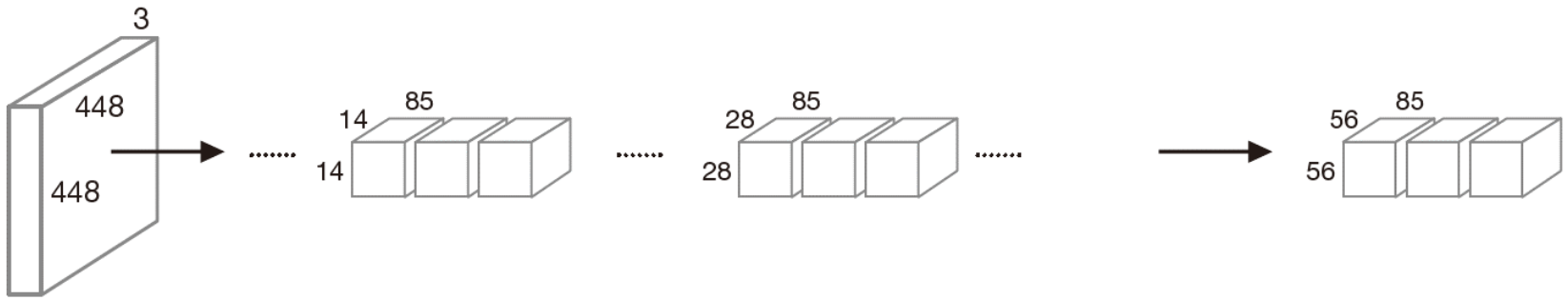


그림 6-20 YOLOv3의 신경망 구조

6.8 물체 검출

- **6.8.1 욜로를 이용한 물체 검출**
  - [그림 6-20]은 YOLOv3이라 부르는 버전3의 신경망 구조를 보여줌
  - 버전3에서는 14x14, 28x28, 56x56의 3개 스케일을 사용해 물체 검출 능력을 향상함
  - 각각 yolo_82와 yolo_94라는 이름의 중간 층, yolo_106이라는 이름의 마지막 층에서 텐서를 추출할 수 있음

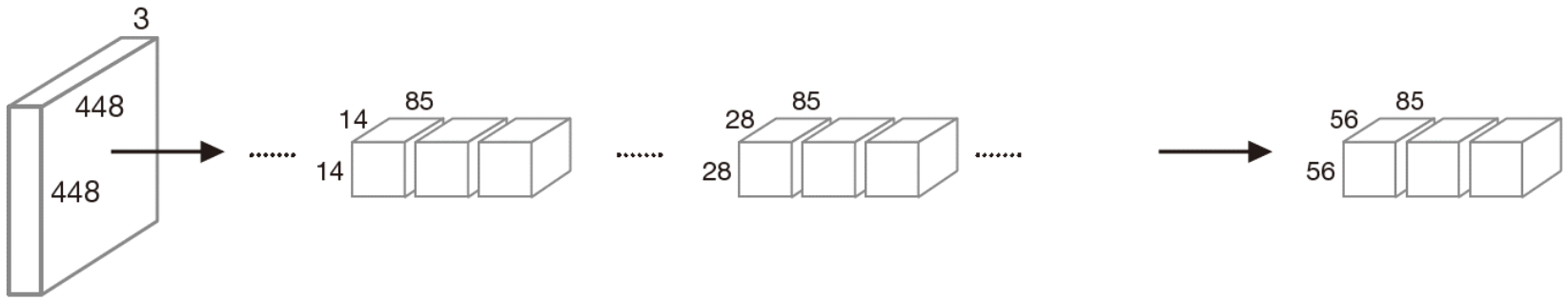


그림 6-20 YOLOv3의 신경망 구조

6.8 물체 검출

- 6.8.1 욜로를 이용한 물체 검출

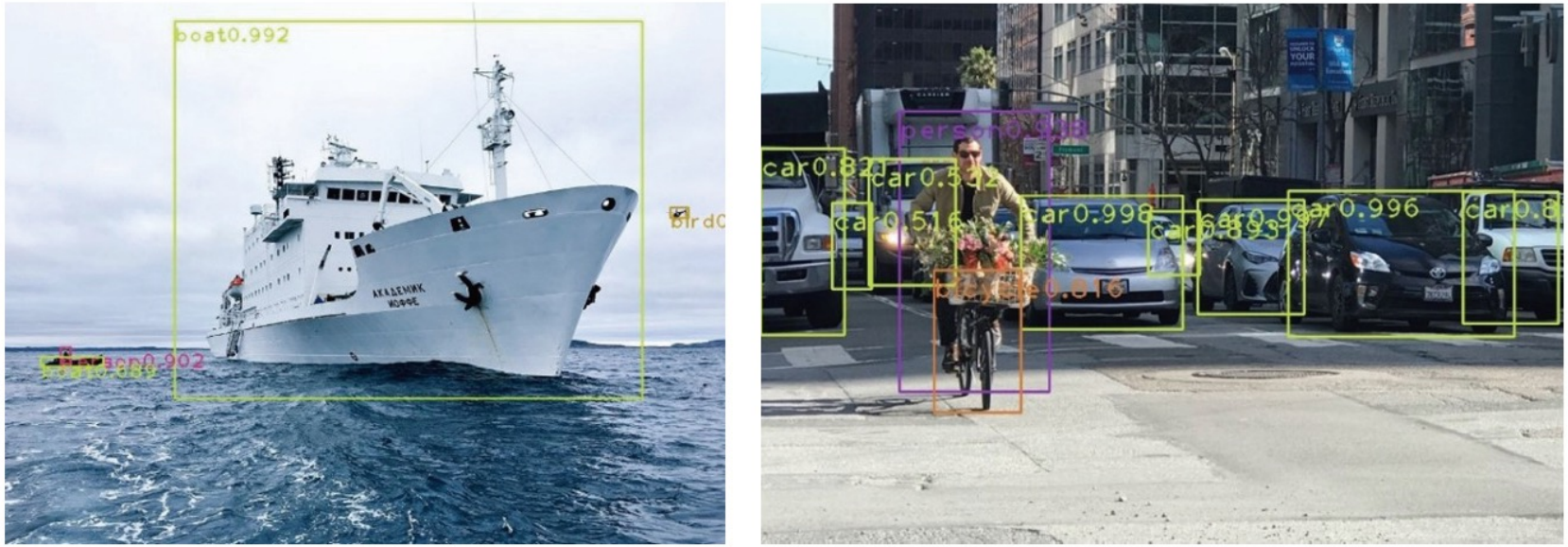


그림 6-21 YOLOv3를 이용한 물체 검출과 분류

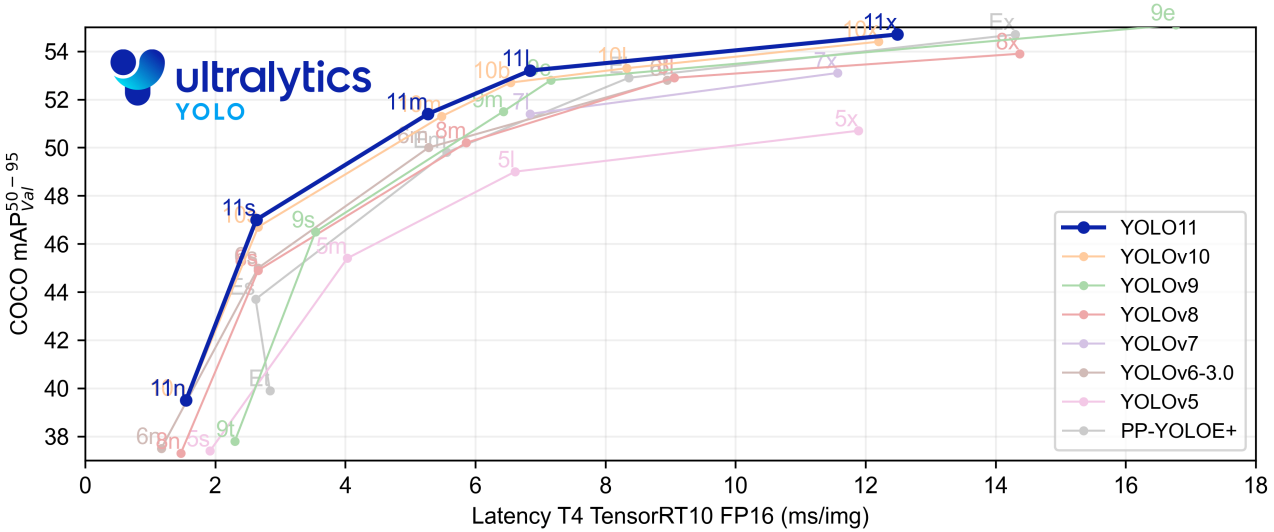
6.8 물체 검출

▪ 6.8.1 욜로를 이용한 물체 검출

1) Ultralytics는 뭐하는 회사인가?

- 컴퓨터비전 모델(특히 Object Detection) 개발
- 기업/연구자/학생이 바로 사용 가능한 패키지화된 모델/SDK 제공
- YOLO 시리즈를 지속적으로 발전시키고 유지보수하는 핵심 팀

즉, YOLO를 연구 모델에서 산업용 모델로 정말 잘 "써먹을 수 있게" 만든 팀이라고 보면 됨.



https://colab.research.google.com/github/ultralytics/ultralytics/blob/main/examples/tutorial.ipynb#scrollTo=yq26lwpYK1lq

6.8 물체 검출

- 6.8.1 욜로를 이용한 물체 검출

라이브러리 사용을 위한 패키지 설치

[1]:
```
!uv pip install ultralytics
import ultralytics
ultralytics.checks()
```

```
Ultralytics 8.3.227 🚀 Python-3.11.13 torch-2.6.0+cu124 CPU (Intel Xeon CPU @ 2.20GHz)
Setup complete ✅ (4 CPUs, 31.4 GB RAM, 6565.6/8062.4 GB disk)
```

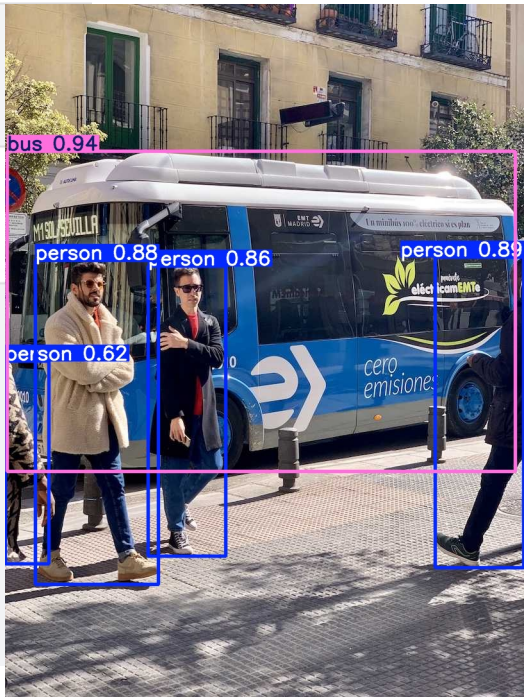
파이썬으로 욜로 모델 테스트

[2]:
```
from ultralytics import YOLO

# Load a model
model = YOLO('yolo11n.yaml')  # build a new model from scratch
model = YOLO('yolo11n.pt')  # load a pretrained model (recommended for training)

# Use the model
results = model('https://ultralytics.com/images/bus.jpg')  # predict on an image
results[0].show()

#results = model.export(format='onnx')  # export the model to ONNX format
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p14-codingnote

6.8 물체 검출

- 6.8.1 욜로를 이용한 물체 검출

검출기 전이 학습 (다른 데이터셋으로 fine tuning)

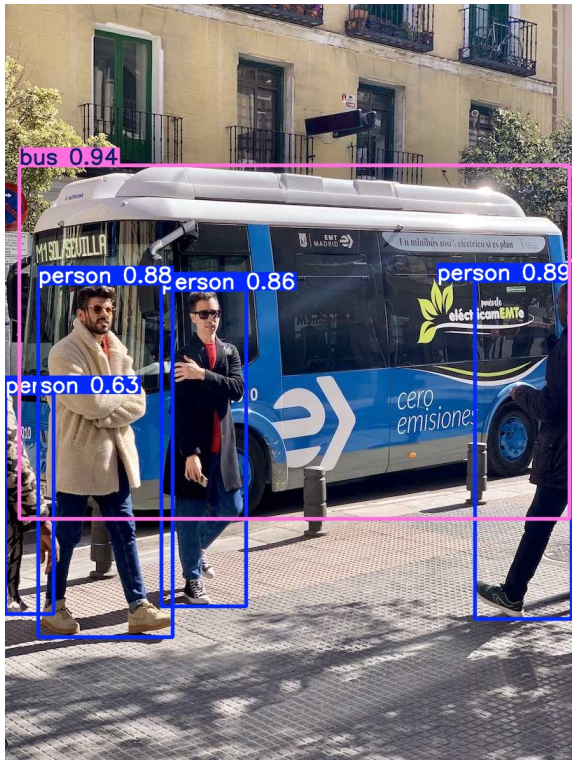
Detection 전이학습 (fine tuning)

- YOLO11 detection models have no suffix and are the default YOLO11 models, i.e. yolo11n.pt and are pretrained on COCO. See Detection Docs for full details.

```
# Load YOLO11n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO

model = YOLO('yolo11n.pt')  # load a pretrained YOLO detection model
model.train(data='coco8.yaml', epochs=3)  # train the model
results=model('https://ultralytics.com/images/bus.jpg')  # predict on an image
results[0].show()
```

```
Ultralytics 8.3.227 🚀 Python-3.11.13 torch-2.6.0+cu124 CPU (Intel Xeon CPU @ 2.20GHz)
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p14-codingnote

6.8 물체 검출

- 6.8.1 욜로를 이용한 물체 검출

검출기 전이 학습 (다른 데이터셋으로 fine tuning)

Oriented Bounding Boxes (OBB) 모델 전이학습 (fine tuning)

- YOLO11 OBB models use the -obb suffix, i.e. yolo11n-obb.pt and are pretrained on the DOTA dataset. See OBB Docs for full details.

12]:

```
# Load YOLO11n-obb, train it on DOTA8 for 3 epochs and predict an image with it
from ultralytics import YOLO

model = YOLO('yolo11n-obb.pt')  # load a pretrained YOLO OBB model
model.train(data='dota8.yaml', epochs=3)  # train the model
results=model('https://ultralytics.com/images/boats.jpg')  # predict on an image
results[0].show()
```



https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w9-p14-codingnote